

OTIMIZAÇÃO COMBINATÓRIA



Sumário

I	Heurísticas Gulosas e Busca Local	4
1	Introdução	5
1.1	TSP	5
1.1.1	Dificuldade	6
1.1.2	Abordagens	7
1.2	Preliminares	8
1.2.1	Grafos	8
1.2.2	Representação do TSP através de grafos	9
1.2.3	Leitor de instâncias	9
1.2.4	<i>Standard Template Library</i>	10
1.2.5	Valores ótimos	11
2	Abordagem meta-heurística para o TSP	12
2.1	<i>Framework</i> ILS	12
2.2	Construção()	13
2.2.1	Inserção mais barata	13
2.3	BuscaLocal()	16
2.3.1	Movimentos	16
2.3.2	Estruturas de vizinhança	16
2.3.3	<i>Best Improvement</i>	17
2.3.4	Estruturas de vizinhança utilizadas	18
2.3.5	RVND	19
2.4	Perturbação()	21
2.5	Parâmetros	23
2.6	<i>Benchmark</i>	23
3	Concatenação de subsequências	25
3.1	MLP	25
3.1.1	Subsequências	26

SUMÁRIO

3.1.2	Concatenação de subsequências	27
3.1.3	Estruturas auxiliares	29
3.1.4	Atualização de subsequências	31
3.1.5	Aplicando o ILS ao MLP	32
II	Algoritmos Exatos	35
4	Branch-and-Bound combinatório	36
4.1	Relaxações	36
4.2	Limitantes	38
4.3	Algoritmo <i>Branch-and-Bound</i>	38
4.4	Branch-and-bound para o TSP	41
4.4.1	Transformação do TSP em um AP	41
4.4.2	Regra de <i>branching</i>	43
4.4.3	Estratégias de <i>branching</i>	44
4.5	Implementação	46
4.6	<i>Benchmark</i>	48
5	Relaxação Lagrangiana	50
5.1	Relaxação de um Problema Linear Inteiro	50
5.1.1	Exemplo com ILP simples	52
5.2	Dual lagrangiano	53
5.3	Subgradiente	54
5.4	Método do subgradiente	57
5.5	Qualidade do Dual Lagrangiano	58
5.6	Relaxação e dual lagrangiano do TSP	60
5.6.1	Resolução do MS1TP	63
5.6.2	Exemplo numérico	65
5.7	Unindo a Relaxação Lagrangiana ao BnB	66
6	Planos de Corte	69
6.1	Resolvendo ILPs com o BnB	69
6.2	Resolvendo ILPs com <i>cutting planes</i>	71
6.2.1	O Problema de Separação	72
6.2.2	Descrição do Algoritmo	73
6.3	O Algoritmo <i>Branch-and-Cut</i>	76
6.4	O problema da Separação para o TSP	76
6.5	Heurística <i>Max-Back</i>	79
6.5.1	Um Algoritmo Exato para obter o min-cut	82

Parte I

Heurísticas Gulosas e Busca Local

1

Introdução

A otimização combinatória é uma importante área relacionada à pesquisa operacional. Um problema de otimização combinatória consiste em encontrar uma solução ótima dentre um conjunto finito de soluções. Neste capítulo, o Problema do Caixeiro Viajante é apresentado, junto a uma breve discussão acerca de sua importância e dificuldade de resolução. Além disso, para que o problema possa ser expresso de forma simples e consistente, alguns conceitos básicos sobre grafos são abordados.

1.1 TSP

O Problema do Caixeiro Viajante (*Traveling Salesman Problem*, TSP) é um dos mais famosos problemas de otimização combinatória na literatura. Dado um conjunto de cidades, o TSP consiste em encontrar uma rota que visite todas elas de forma a minimizar a distância total percorrida¹ (ou o tempo total de viagem). A Figura 1.1 apresenta um exemplo de instância do TSP, na qual há seis cidades dispostas em um espaço bidimensional. No exemplo, a distância entre as cidades 1 e 4 é de 118 km, enquanto a das cidades 1 e 3 é de 174 km. De posse das distâncias entre cada par de cidades, obtém-se algo semelhante à Tabela 1.1.

Uma solução válida (o termo “viável” será utilizado daqui em diante) para um TSP é uma sequência que visita cada cidade exatamente uma vez (um *tour*). As figuras 1.2a e 1.2b mostram exemplos de soluções viáveis para a mesma instância. Ao verificar a Tabela 1.1, percebe-se que a distância total percorrida² na solução da Figura 1.2a é de $129 + 114 + 250 + 186 + 105 + 118 = 902$ km. A solução da Figura 1.2b, por sua vez, possui uma distância total percorrida de $174 + 114 +$

¹Minimizar a distância total percorrida é a “função objetivo” do problema.

²Se a função objetivo do problema é minimizar a distância total percorrida, diz-se que a distância percorrida em uma solução é o seu “valor objetivo”.

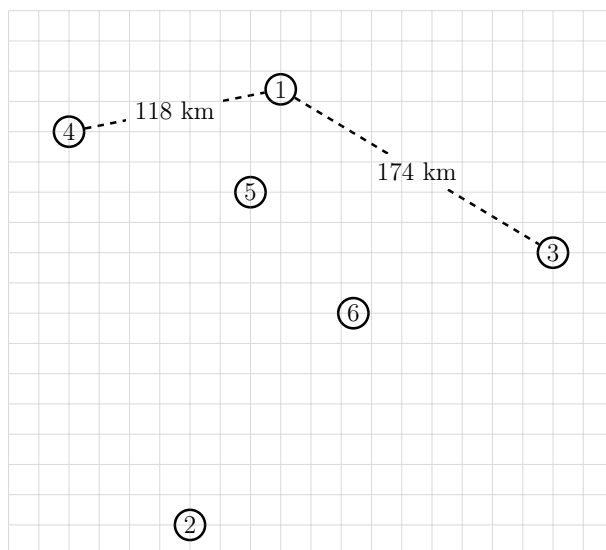


Figura 1.1: Exemplo de instância.

Tabela 1.1: Distâncias entre cada par de cidades (em km).

Cidade	1	2	3	4	5	6
1		245	174	118	59	129
2			250	226	186	147
3				274	169	114
4					105	185
5						87
6						

$147 + 226 + 105 + 59 = 825$ km, que é menor se comparada à solução da Figura 1.2a, tornando-a preferível.

1.1.1 Dificuldade

Uma maneira de determinar a solução com o menor custo de viagem para uma instância é examinar todas as soluções possíveis. Note que diferentes soluções podem ser vistas como permutações do conjunto de cidades. Além disso, (i) não importa de onde se inicia o *tour* (i.e., a sequência $(1, 3, 4, 5, 2, 1)$ é essencialmente igual à sequência $(2, 1, 3, 4, 5, 2)$); e (ii) percorrer um *tour* no sentido anti-horário é igual a percorrê-lo no sentido horário³ (i.e., a sequência $(1, 2, 3, 4, 1)$ é igual à

³A versão do TSP abordada pressupõe que partir da cidade i para a cidade j é igual a partir da cidade j para a cidade i . A variante na qual isso não é necessariamente verdade chama-se ATSP (*Assymetrical Traveling Salesman Problem*).

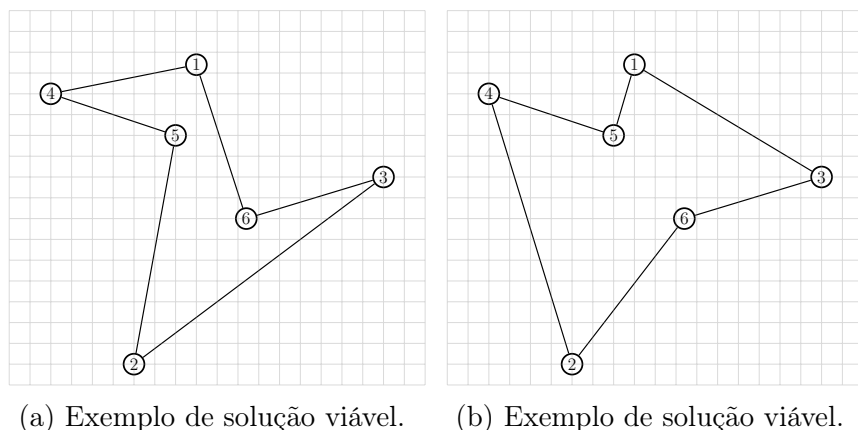


Figura 1.2: Exemplos de soluções viáveis.

sequência $(1, 4, 3, 2, 1)$.

Então, se n é o número de cidades de uma determinada instância, o número de soluções distintas é $(n - 1)!/2$. Dessa forma, resolver a instância descrita anteriormente exigiria examinar $(5!)/2 = 60$ soluções. Contudo, o TSP possui aplicações⁴ que envolvem instâncias com milhares (ou milhões) de cidades. Se $n \geq 60$, o número de soluções possíveis ultrapassa o número estimado de átomos no universo. Portanto, examinar cada uma das soluções para instâncias desse tipo é impraticável computacionalmente.

1.1.2 Abordagens

Há duas maneiras de resolver problemas de otimização combinatória como o TSP: os métodos exatos e os métodos heurísticos. O primeiro visa obter soluções comprovadamente ótimas, enquanto o segundo busca obter soluções boas em um tempo razoável. Normalmente, os métodos exatos costumam ser mais lentos, e seu desempenho depende da existência de bons limitantes superiores⁵, que normalmente são providenciados pelos métodos heurísticos.

Embora o TSP seja um problema bem resolvido tanto de forma exata quanto heurística, ele serve como ponto de partida para o estudo de problemas mais avançados, como o Problema de Roteamento de Veículos e suas variantes, por compartilhar muitas de suas características com eles. O capítulo seguinte descreve

⁴O TSP é um subproblema de tarefas como a criação de trilhas que conectam pontos em placas de circuitos, sequenciamento de DNA, etc.

⁵A solução da Figura 1.2b, por exemplo, possui uma distância total de 825 km. Isso significa que uma solução ótima para essa instância não pode ter uma distância total maior que 825. Diz-se, portanto, que 825 é um “limite superior”. Um método exato pode tirar proveito dessa informação. (*upper bound*).

uma abordagem heurística simples para resolver o TSP. A abordagem também é versátil, servindo como *framework* para a resolução de vários outros problemas de otimização combinatória.

1.2 Preliminares

O algoritmo heurístico descrito no próximo capítulo deve ser implementado na linguagem C++ para facilitar na compreensão do problema e no processo de implementação, alguns conceitos e observações são apresentados a seguir.

1.2.1 Grafos

Sejam V e E conjuntos de vértices e arestas, respectivamente. O par $G = (V, E)$ é considerado um grafo. Mais precisamente, E é um conjunto de pares $\{i, j\}$ tais que $i, j \in V, i \neq j$. Assim, qualquer conjunto de vértices ligados por arestas pode ser considerado um grafo. Um exemplo de grafo é mostrado na Figura 1.3a, nela, $V = \{1, 2, 3, 4, 5\}$ e $E = \{\{1, 4\}, \{1, 5\}, \{2, 3\}, \{3, 4\}, \{4, 5\}, \{2, 4\}, \{3, 5\}\}$. Se todos os vértices estiverem conectados uns aos outros por arestas, G é um grafo completo. Um grafo completo com cinco vértices é ilustrado na Figura 1.3b.

Grafo com pesos

Se cada aresta $e = \{i, j\} \in E$ de um grafo estiver associada a um peso c_{ij} (um número real qualquer), diz-se que esse grafo é um grafo com pesos. A Figura 1.3c ilustra um grafo com pesos em que $c_{2,3} = 3,14$, $c_{1,2} = 60$, $c_{3,5} = 42$, e $c_{4,5} = -1$.

Passeio

Dado um grafo $G = (V, E)$, a sequência alternada de vértices e arestas $v_0 e_0 v_1 e_1 \dots e_{k-1} v_k$ é considerada um passeio desde que toda aresta $e_i = \{v_i, v_{i+1}\}$ pertença a E . A Figura 1.3d contém um exemplo de um passeio percorrido sobre as arestas do grafo da Figura 1.3a. Observe que tanto vértices quanto arestas podem ser percorridos mais de uma vez.

Ciclo e ciclo Hamiltoniano

Um ciclo é um passeio que começa e termina no mesmo vértice. Se todos os vértices no grafo são visitados exatamente uma vez, o ciclo é dito hamiltoniano. As figuras 1.3e e 1.3f ilustram um ciclo e um ciclo hamiltoniano, respectivamente.

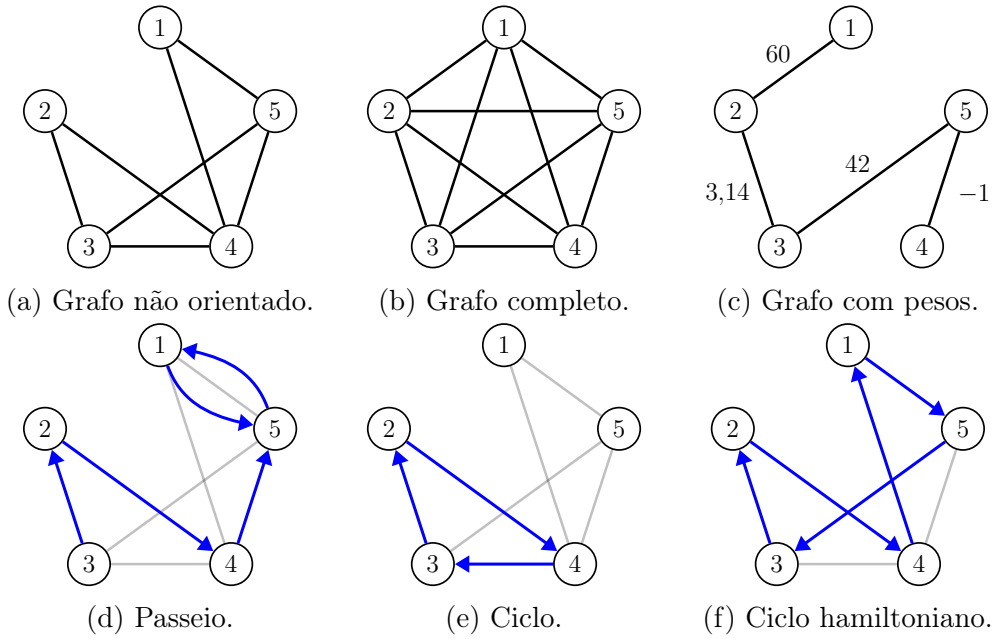


Figura 1.3: Exemplos de grafos, caminhos e ciclos.

1.2.2 Representação do TSP através de grafos

Uma instância do TSP pode ser vista como um grafo completo com pesos, no qual cada nó corresponde a uma cidade, e cada aresta está associada a um custo que representa a distância entre as duas cidades correspondentes. Resolver o TSP equivale a encontrar o ciclo hamiltoniano nesse grafo cujo custo total (a soma dos pesos das arestas no ciclo) é o menor possível.

O grafo citado pode, ainda, ser representado por uma matriz M , na qual o elemento $M_{ij} = c_{ij}$ equivale ao peso da aresta $\{i, j\}$. Por conter informações sobre as arestas, que consistem em pares de vértices adjacentes, diz-se que M é uma matriz de adjacência.

1.2.3 Leitor de instâncias

Alguns arquivos de instâncias e o código de um leitor de instâncias se encontram neste link: <https://github.com/cvneves/kit-opt/tree/master/GILS-RVND-TSP/leitor-instancias>. O leitor computa a matriz de adjacência da instância, representando-a através do *array* bidimensional `matrizAdj`. Note que os custos são indexados a partir de 1. Logo, se uma instância contém 14 cidades, a distância entre a primeira e a última cidade é `matrizAdj[1][14]` (ou `matrizAdj[14][1]`).

1.2.4 *Standard Template Library*

O uso do paradigma de programação orientada a objetos não é obrigatório. Contudo, para facilitar a implementação das estruturas de dados e rotinas necessárias no algoritmo, recomenda-se o estudo dos componentes **vector** e **sort()** da *Standard Template Library* (STL). O trecho de código abaixo mostra como as soluções do TSP podem ser representadas.

```

1  typedef struct Solucao{
2      vector<int> sequencia;
3      double valorObj;
4  } Solucao;
5
6  // Exemplo de inicializacao
7  Solucao s1 = {{1,6,3,2,5,4,1}, 0};
8
9  // Iterando pelos vertices da solucao
10 void exhibirSolucao(Solucao *s)
11 {
12     for(int i = 0; i < s->sequencia.size() - 1; i++)
13         std::cout << s->sequencia[i] << " -> ";
14     std::cout << s->sequencia.back() << std::endl;
15 }
16
17 exhibirSolucao(&s1);
18 // RESULTADO: "1 -> 6 -> 3 -> 2 -> 5 -> 4 -> 1"
19
20 // Inicializando uma solucao vazia
21 Solucao s2 = {{}, 0.0};
22 // Adicionando vertices na solucao
23 s2.sequencia.push_back(1);
24 s2.sequencia.push_back(3);
25 s2.sequencia.push_back(1);
26 exhibirSolucao(&s2);
27 // RESULTADO: "1 -> 3 -> 1"
28 s2.sequencia.insert(s2.begin() + 1, 4);
29 s2.sequencia.insert(s2.end() - 1, 5);
30 s2.sequencia.insert(s2.begin() + 2, 2);
31 exhibirSolucao(&s2);
32 // RESULTADO: "1 -> 4 -> 2 -> 3 -> 5 -> 1"
33
34 void calcularValorObj(Solucao *s){
35     s->valorObj = 0;
36     for(int i = 0; i < s->sequencia.size() - 1; i++)
37         s->valorObj += matrizAdj[s->sequencia[i]][s->sequencia[i+1]];
38 }

```

1.2.5 Valores ótimos

O valor ótimo de custo das instâncias fornecidas já foi determinado por meio de métodos exatos. Uma lista com esses valores — que devem ser consultados para determinar a eficácia da implementação do algoritmo — se encontra neste link: <http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/STSP.html>.

Depois de implementado, o algoritmo deve ser capaz de obter o valor ótimo para instâncias de até 300 cidades na maioria das vezes em que é executado⁶.

⁶Além de não garantir a obtenção de uma solução ótima, o algoritmo não é determinístico. Por isso, determinar a robustez e velocidade de sua implementação exige testá-lo extensivamente.

2

Abordagem meta-heurística para o TSP

Uma meta-heurística é um método heurístico para resolver problemas de otimização combinatória de forma genérica. Uma única meta-heurística pode ser reutilizada para resolver vários problemas diferentes, exigindo, por vezes, pouco esforço de adaptação para cada um deles. Este capítulo descreve a meta-heurística *Iterated Local Search* (ILS), além das devidas adaptações necessárias para utilizá-la para resolver o TSP.

2.1 *Framework* ILS

Considere um problema de otimização combinatória qualquer. Assuma que o problema é de minimização. O código a seguir apresenta um algoritmo genérico para resolvê-lo:

```
1  Solution ILS(int maxIter, int maxIterIls)
2  {
3      Solution bestOfAll;
4      bestOfAll.cost = INFINITY;
5      for(int i = 0; i < maxIter; i++)
6      {
7          Solution s = Construc()();
8          Solution best = s;
9
10         int iterIls = 0;
11
12         while(iterIls <= maxIterIls)
13         {
14             BuscaLocal(&s);
15             if(s.cost < best.cost)
16             {
17                 best = s;
18                 iterIls = 0;
19             }
```



```

20     s = Perturbacao(best);
21     iterIls++;
22 }
23 if (best.cost < bestOfAll.cost)
24     bestOfAll = best;
25 }
26
27 return bestOfAll;
28 }

```

Primeiramente, algoritmo constrói uma solução baseando-se em palpites educados através do método Construção() (linha 7). Em seguida, tenta-se melhorar o custo dessa solução fazendo-se pequenas modificações através do método BuscaLocal() enquanto houver melhoras (linhas 14–18). Quando não for mais possível melhorar a solução, continua-se a partir de uma cópia levemente modificada da melhor solução da iteração corrente **best**, gerada pelo procedimento Perturbação() (linha 20). Continua-se até que um critério de parada seja atingido (linha 12). Se **best.cost** for menor que **bestOfAll.cost**, **best** passa a ser a melhor solução (linhas 23 e 24). Esse processo é repetido **maxIter** vezes (linha 5), e a melhor solução **bestOfAll** é retornada (linha 27).

Os procedimentos Construção(), BuscaLocal() e Perturbação() serão explicados em detalhes para o caso do TSP nas próximas seções. Recomenda-se que o leitor implemente-os na ordem em que são apresentados, para que então possa implementar o código acima.

2.2 Construção()

Uma heurística construtiva busca criar uma solução razoável para um problema de otimização. A solução criada pode então ser modificada e melhorada ao longo do tempo. Por esse motivo, é comum que o valor objetivo de soluções geradas por procedimentos esteja longe do valor ótimo. A heurística construtiva utilizada no método Construção() baseia-se no método *Greedy Randomized Adaptive Search Procedure* (GRASP) [REF](#).

2.2.1 Inserção mais barata

Considere uma instância do TSP dada pelo grafo completo com pesos $G = (V, E)$. Considere, ainda, um subconjunto arbitrário de vértices $V' \subset V$, e que s' é um *tour* que visita todos os vértices de V' ¹. Seja $CL = V \setminus V'$ uma lista de candidatos a serem inseridos em s' de forma a obter um *tour* completo.

¹Nesse caso, diz-se que s' é um *subtour*, já que visita apenas alguns vértices de V .

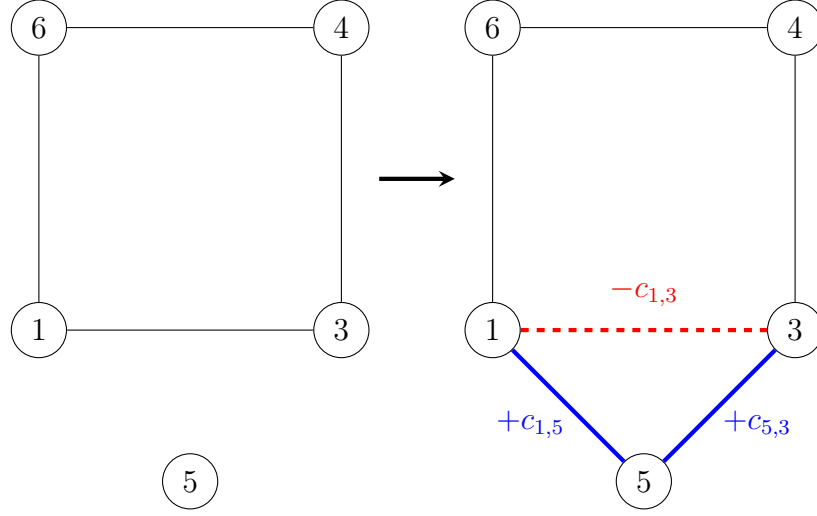


Figura 2.1: Inserção de um novo nó.

Analisemos o que ocorre quando um único vértice $k \in CL$ é inserido em s' . Note que para isso, é sempre necessário colocar k entre dois vértices adjacentes i e j . Essa operação implica na remoção da aresta $\{i, j\}$, e na adição de duas arestas $\{i, k\}$ e $\{k, j\}$. Suponha que s'' é um novo *subtour*, resultado da inserção de k em s' . A partir disso, deduz-se que o custo de inserção de k em s' é $\Delta = f(s'') - f(s') = c_{ik} + c_{kj} - c_{ij}$. Um exemplo dessa operação é mostrado na Figura 2.1, na qual o vértice 5 é inserido entre os vértices 1 e 3, resultando em um novo *subtour*.

É fácil perceber que escolher o vértice k e a aresta $\{i, j\}$ de forma a minimizar Δ significa diminuir o prejuízo no valor objetivo após a inserção. No entanto, escolher sempre o par $(k, \{i, j\})$ que minimiza Δ pode “viciar” o método, levando-o sempre a soluções parecidas, o que pode dificultar as buscas locais. Portanto, convém introduzir um pouco de aleatoriedade na escolha do par. Baseando-se nessas ideias, pode-se enumerar as etapas do método Construção() como segue.

1. Construir uma solução parcial s' de forma aleatória;
2. $V' \leftarrow$ vértices de s' e $CL \leftarrow V \setminus V'$;
3. Computar os pares $(k, \{i, j\})$ para todo $k \in CL$, $\{i, j\} \in s'$ e armazená-los em uma lista Ω ;
4. Ordenar os pares em Ω em ordem crescente de Δ ;
5. $\alpha \leftarrow$ número aleatório no intervalo $[0, 1]$;

6. Selecionar aleatoriamente um dos $\lfloor \alpha \times |\Omega| \rfloor$ primeiros pares em Ω ;
7. Sendo $(k, \{i, j\})$ o par escolhido, retirar a aresta $\{i, j\}$ de s' e inserir o vértice k entre i e j ;
8. Remover k da lista de candidatos CL ;
9. Se CL estiver vazio, retornar s' . Senão, voltar para o passo 3.

É suficiente construir o *subtour* inicial do passo 1 com 3 vértices. Para isso, pode-se iniciar s' sempre como $s' = \{1, 1\}$ e inserir 3 vértices de forma aleatória. Assim, o *subtour* inicial da Figura 2.1, por exemplo, seria $s' = \{1, 6, 4, 3, 1\}$.

Um exemplo de implementação do procedimento Construção() pode ser visto a seguir:

```

1  struct InsertionInfo
2  {
3      int noInserido; // no k a ser inserido
4      int arestaRemovida; // aresta {i,j} na qual o no k sera inserido
5      double custo; // delta ao inserir k na aresta {i,j}
6  };
7
8  std::vector<InsertionInfo> calcularCustoInsercao(Solution& s, std::vector<
    int>& CL)
9  {
10     std::vector<InsertionInfo> custoInsercao = std::vector<InsertionInfo>((s
        .size() - 1) * CL.size());
11     int l = 0;
12     for(int a = 0; a < s.sequence.size() - 1; a++) {
13         int i = s.sequence[a];
14         int j = s.sequence[a + 1];
15         for (auto k : CL) {
16             custoInsercao[l].custo = c[i][k] + c[j][k] - c[i][j];
17             custoInsercao[l].noInserido = k;
18             custoInsercao[l].arestaRemovida = a;
19             l++;
20         }
21     }
22     return custoInsercao;
23 }
24
25 Solution Construcão()
26 {
27     Solution s;
28     s.sequence = escolher3NosAleatorios();
29     std::vector<int> CL = nosRestantes();
30     /* Ex: V = {1,2,3,4,5,6,7,8,9,10}
31        s.sequence = {1,2,9,5,1}

```

```
32     CL = {3,4,6,7,8,10} */
33     while(!CL.empty()) {
34         std::vector<InsertionInfo> custoInsercao = calcularCustoInsercao(s, CL
            );
35         ordenarEmOrdemCrescente(custoInsercao);
36         double alpha = (double) rand() / RAND_MAX;
37         int selecionado = rand() % ((int) ceil(alpha * custoInsercao.size()));
38         inserirNaSolucao(s, custoInsercao[selecionado].noInserido);
39     }
40
41     return s;
42 }
```

No exemplo, armazena-se as informações de cada par $(k, \{i, j\})$ em uma estrutura do tipo `InsertionInfo`. Os custos de inserção de cada par $(k, \{i, j\})$ são calculados pela função `calcularCustoInsercao()`, que é chamada na função `Construcao()` até que CL esteja vazio.

2.3 BuscaLocal()

A etapa de busca local possui como objetivo melhorar a solução corrente no decorrer da execução do algoritmo. O procedimento é feito modificando-se as soluções e avaliando o impacto das modificações na função objetivo. Antes de descrever o procedimento `BuscaLocal()`, alguns conceitos são apresentados nas subseções que seguem.

2.3.1 Movimentos

Seja s uma solução viável qualquer para um problema de otimização, e m uma operação que altera s de alguma maneira. Diz-se que m é um “movimento”, e que $s' = s \oplus m$ é uma nova solução, obtida após executá-lo. Há varios tipos de movimentos possíveis, que dependem, por sua vez, do problema de otimização em questão. Um exemplo de movimento para o TSP é trocar as posições de dois vértices na sequência.

2.3.2 Estruturas de vizinhança

Sejam s uma solução qualquer, e M_k um conjunto de movimentos estruturalmente parecidos. O conjunto $\mathcal{N}_k(s)$ contém todas as possíveis soluções que se poderia obter executando em s os movimentos do conjunto M_k . Já que as soluções s e $s' \in \mathcal{N}_k(s)$ diferem em apenas um movimento, diz-se que elas são “vizinhas”. Por esse motivo, $\mathcal{N}_k$ é considerado uma “estrutura de vizinhança”.

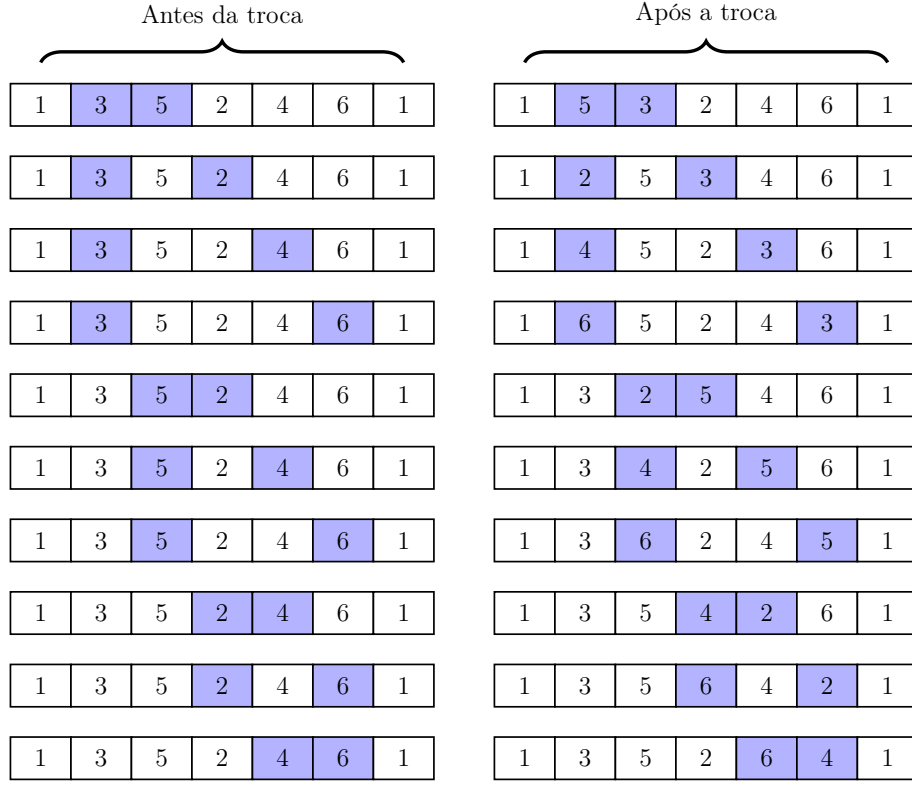


Figura 2.2: Exemplo da estrutura de vizinhança *swap*.

Para compreender melhor o conceito, consulte a Figura 2.2. Na figura, todas as combinações possíveis de movimentos da estrutura de vizinhança SWAP, que envolve trocar a posição de dois vértices quaisquer na sequência, foram listadas para a solução

$$s = (1, 3, 5, 2, 4, 6, 1).$$

2.3.3 Best Improvement

Dada uma solução s e uma estrutura de vizinhança $\mathcal{N}_k$, o método do melhor aprimoramento (*best improvement*) visa encontrar o vizinho de s com o menor custo possível. Em outras palavras, deseja-se encontrar o “melhor vizinho” de s considerando-se a estrutura de vizinhança $\mathcal{N}_k$. Se $f(s^*) < f(s)$, diz-se que houve uma melhora, e convém substituir s por s^* . O trecho de código a seguir mostra como utilizar o método *best improvement* para explorar toda a estrutura de vizinhança *swap*:

```

1  bool bestImprovementSwap(Solution *s)
2  {

```



```

3  double bestDelta = 0;
4  int best_i, best_j;
5  for(int i = 1; i < s->sequence.size() - 1; i++)
6  {
7      int vi = s->sequence[i];
8      int vi_next = s->sequence[i + 1];
9      int vi_prev = s->sequence[i - 1];
10     for(int j = i + 1; j < s->sequence.size() - 1; j++)
11     {
12         int vj = s->sequence[j];
13         int vj_next = s->sequence[j + 1];
14         int vj_prev = s->sequence[j - 1];
15         double delta = -c[vi_prev][vi] - c[vi][vi_next] + c[vi_prev][vj]
16                       + c[vj][vi_next] - c[vj_prev][vj] - c[vj][vj_next]
17                       + c[vj_prev][vi] + c[vi][vj_next];
18
19         if (delta < bestDelta)
20         {
21             bestDelta = delta;
22             best_i = i;
23             best_j = j;
24         }
25     }
26 }
27
28 if(bestDelta < 0)
29 {
30     std::swap(s->sequence[best_i], s->sequence[best_j]);
31     s->cost = s->cost + bestDelta;
32     return true;
33 }
34 return false;
35 }

```

No código, enumera-se todos os movimentos de troca entre dois nós possíveis, e o impacto de cada movimento na função objetivo é avaliado nas linhas 15–17. Note que não é necessário recalcular o custo da solução do zero, pois o custo de uma troca pode ser calculado com uma simples fórmula desde que as arestas a serem removidas e inseridas sejam conhecidas².

2.3.4 Estruturas de vizinhança utilizadas

As estruturas de vizinhança que devem ser utilizadas são descritas como segue.

- SWAP — $\mathcal{N}_1$: Troca a posição de dois vértices na sequência;

²Levar isso em consideração é crucial para o bom desempenho do algoritmo.

- 2-OPT — $\mathcal{N}_2$: Duas arestas não adjacentes da solução são removidas e o segmento entre elas é reinserido de maneira invertida, adicionando-se duas novas arestas para reconstruir a solução.
- REINSERTION — $\mathcal{N}_3$: Um único vértice é retirado de sua posição e inserido em outra;
- OR-OPT-2 — $\mathcal{N}_4$: Um bloco composto por dois vértices adjacentes é retirado de sua posição e inserido em outra;
- OR-OPT-3 — $\mathcal{N}_5$: Um bloco composto por três vértices adjacentes é retirado de sua posição e inserido em outra;

A Figura 2.3 apresenta exemplos de movimentos de cada uma das estruturas aplicados em uma solução com 10 vértices.

2.3.5 RVND

O procedimento BuscaLocal() consiste em uma implementação do método *Random Variable Neighborhood Descent* (RVND) utilizando-se as estruturas de vizinhança mencionadas. A ideia do procedimento é utilizar o método *best improvement* em diferentes estruturas de vizinhança (escolhidas de forma aleatória) enquanto houver melhora, eliminando estruturas de vizinhança que não provocaram melhoras. Isso pode ser visto no seguinte trecho de código:

```
1 void BuscaLocal(Solution *s)
2 {
3     std::vector<int> NL = {1, 2, 3, 4, 5};
4     bool improved = false;
5
6     while (NL.empty() == false)
7     {
8         int n = rand() % NL.size();
9         switch (NL[n])
10        {
11            case 1:
12                improved = bestImprovementSwap(s);
13                break;
14            case 2:
15                improved = bestImprovement20pt(s);
16                break;
17            case 3:
18                improved = bestImprovement0r0pt(s, 1); // Reinsertion
19                break;
20            case 4:
21                improved = bestImprovement0r0pt(s, 2); // Or-opt2
```

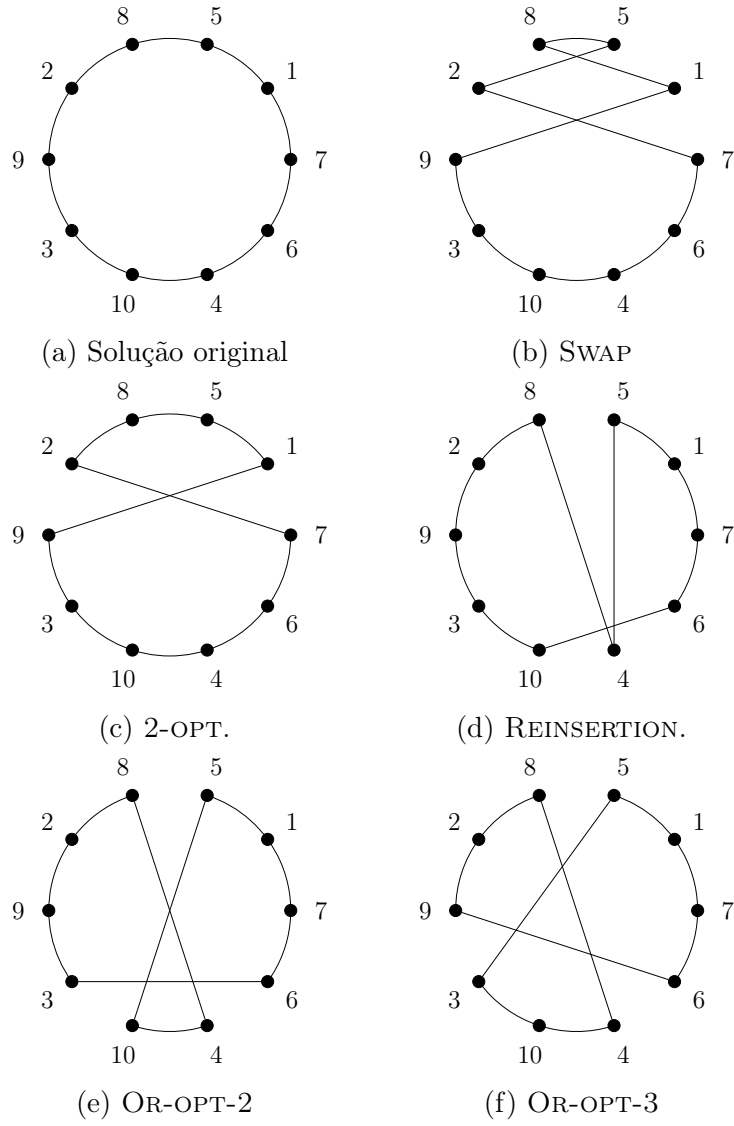


Figura 2.3: Exemplos das estruturas de vizinhança.

```

22     break;
23     case 5:
24         improved = bestImprovementOrOpt(s, 3); // Or-opt3
25         break;
26     }
27
28     if (improved)
29         NL = {1, 2, 3, 4, 5};
30     else
31         NL.erase(NL.begin() + n);
32 }
33 }

```

No código, o procedimento `BestImprovement()` deve ser implementado para cada uma das 5 estruturas de vizinhança. A implementação de cada um dos procedimentos é semelhante ao exemplo mostrado anteriormente para a estrutura de vizinhança SWAP.

Já que as estruturas de vizinhança REINSERTION, OR-OPT-2 e OR-OPT-3 são muito parecidas, encoraja-se que sejam implementadas através de uma única função, que possui como um de seus parâmetros um número de 1 a 3, que representa o tamanho do bloco a ser movido.

2.4 Perturbação()

Uma solução é um “ótimo local” se ela não pode mais ser melhorada pelo procedimento `BuscaLocal()`. O objetivo do procedimento `Perturbação()` é modificar levemente soluções desse tipo. Embora uma solução obtida após uma perturbação aleatória seja quase sempre pior que a original, espera-se que ela possa ser melhorada através do procedimento `BuscaLocal()`, conforme ilustrado na Figura 2.4.

Para facilitar a visualização, a função objetivo $f(s)$ na Figura 2.4 foi representada por meio de uma curva contínua, enquanto o eixo horizontal representa o espaço de soluções possíveis. As linhas tracejadas delimitam as soluções vizinhas que as buscas locais conseguem “enxergar”. Ao realizar uma busca local nas vizinhanças de s_0 , obtém-se a solução s_1 , que é um ótimo local. A solução s_1 é então perturbada, resultando em uma solução ligeiramente pior s_2 . Porém, ao realizar uma busca local nas vizinhanças de s_2 , obtém-se uma nova solução s_3 , que é o mínimo global da função.

Neste caso, o procedimento `Perturbação()` consiste na execução de um movimento chamado *double bridge*. O movimento troca as posições de dois segmentos da sequência. As figuras 2.5a e 2.5b ilustram um exemplo de uma solução antes e após esse movimento.

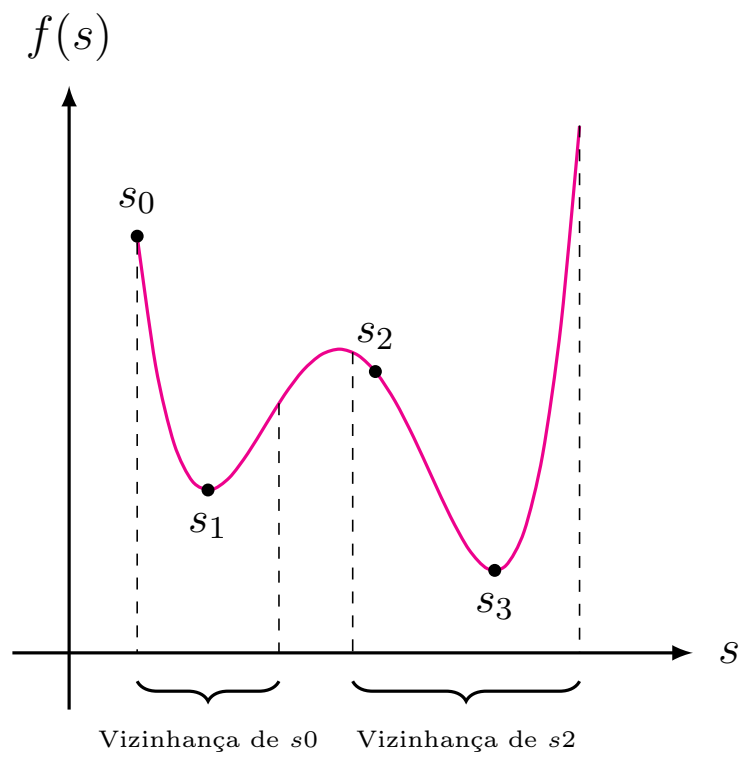


Figura 2.4: Visualização da busca local.

Uma chamada ao procedimento Perturbação(s) deve executar os seguintes passos:

1. $s' \leftarrow s$;
2. Escolher aleatoriamente dois segmentos não sobrepostos de s' de tamanhos entre 2 e $\lceil |V|/10 \rceil$;
3. Trocar a posição dos segmentos em s' ;
4. Retornar s' .

1	14	5	4	8	7	13	6	12	3	10	2	11	9	1
---	----	---	---	---	---	----	---	----	---	----	---	----	---	---

(a) Solução antes da perturbação.

1	14	3	10	2	11	8	7	13	6	12	5	4	9	1
---	----	---	----	---	----	---	---	----	---	----	---	---	---	---

(b) Solução após a perturbação.

Figura 2.5: Exemplo do movimento *double bridge* em uma sequência.

2.5 Parâmetros

Os parâmetros `maxIter` e `maxIterILS` influenciam no desempenho do algoritmo tanto em termos de tempo, quanto na qualidade das soluções. Neste caso, os valores dos parâmetros devem ser inicializados da seguinte forma:

1. `maxIter` $\leftarrow$ 50;
2. `maxIterILS` $\leftarrow$ $\begin{cases} |V|/2 & \text{se } |V| \geq 150 \\ |V| & \text{senão.} \end{cases}$

2.6 Benchmark

A Tabela 2.1 apresenta os valores médios de custo e tempo (em segundos) de resolução de cada instância após 10 execuções do algoritmo em um processador Intel® Core™ i7-7700 3.60GHz.

Abordagem meta-heurística para o TSP

Tabela 2.1: Tempo e custo médios obtidos para cada instância.

Instância	Resultados		Instância	Resultados	
	Tempo	Custo		Tempo	Custo
a280	76.51	2579	kroA150	9.395	26524
ali535	1927	202431	kroA200	28.83	29368
att48	0.229	10628	kroB100	3.128	22141
att532	1597	27731.9	kroB150	9.646	26130
bayg29	0.034	1610	kroB200	31.55	29437
bays29	0.06	2020	kroC100	2.985	20749
berlin52	0.291	7542	kroD100	3.014	21294
bier127	8.873	118282	kroE100	3.713	22068
brazil58	0.362	25395	lin105	3.231	14379
brg180	10.25	1950	lin318	171.5	42031.1
burma14	0.003	3323	linhp318	162.7	42042.4
ch130	8.664	6110	pcb442	572.9	50850.8
ch150	7.493	6528	pr107	3.506	44303
d198	24.14	15780	pr124	5.681	59030
d493	1015	35050.7	pr136	10.46	96772
dantzig42	0.199	699	pr144	8.863	58537
eil101	3.422	629	pr152	6.877	73682
eil51	0.293	426	pr226	33.48	80369
eil76	1.223	538	pr264	55.91	49135
fl417	315.1	11861	pr299	107.4	48191.6
fri26	0.022	937	pr76	1.109	108159
gil262	80.39	2378.1	rat195	24.77	2323
gr120	7.139	6942	rat99	3.089	1211
gr137	8.543	69853	rd100	3.323	7910
gr17	0.006	2085	rd400	481.5	15296.8
gr202	30.33	40160.2	si175	13.28	21407
gr21	0.012	2707	si535	882.9	48463.9
gr229	49.66	134613	st70	0.843	675
gr24	0.018	1272	swiss42	0.126	1273
gr431	644.2	171509	ts225	26.22	126643
gr48	0.233	5046	tsp225	41.83	3916
gr96	2.595	55209	u159	8.621	42080
hk48	0.237	11461	ulysses16	0.005	6859
kroA100	2.812	21282	ulysses22	0.015	7013

3

Concatenação de subsequências

O Problema da Mínima Latência (*Minimum Latency Problem*, MLP) [REF](#) e o Problema de Roteamento de Veículos com Janelas de Tempo (*Vehicle Routing Problem with Time Windows*, VRPTW) [REF](#) são exemplos problemas de otimização combinatória mais difíceis que o TSP. Parte dessa dificuldade reside em características especiais desses problemas, seja em termos de função objetivo ou de restrições. Tais diferenças tornam difícil avaliar o impacto de modificações na solução. No MLP, por exemplo, a aplicação de um movimento como o *swap* pode alterar drasticamente o seu valor objetivo, que não pode mais ser calculado apenas somando e subtraindo custos de arestas, como no caso do TSP. No VRPTW, aplicar o mesmo movimento pode violar restrições do problema, exigindo uma avaliação prévia da sua viabilidade, que pode demandar muitas operações.

Reavaliar o custo de movimentos ou checar a viabilidade de novas soluções durante as buscas locais de forma ingênua pode comprometer e, por vezes, inviabilizar o método. Este capítulo aborda o método de concatenação de subsequências, que permite realizar essas avaliações de maneira eficiente para uma vasta classe de problemas. Para isso, a aplicação do método será apresentada para o MLP e o VRPTW, definidos nas seções seguintes. Ao fim do capítulo, espera-se que o leitor seja capaz de empregar as mesmas técnicas do capítulo anterior para esses problemas de forma eficiente.

3.1 MLP

Seja $G = (V, E)$ um grafo em que cada aresta $\{\sigma_i, \sigma_j\} \in E$ está associada a um custo $t_{\sigma_i \sigma_j}$. Suponha, ainda, que $|V| = n$ e que o *tour* $s = (\sigma_1, \dots, \sigma_{n+1})$ é a sequência de vértices de um ciclo hamiltoniano em G , que começa e termina no nó $\sigma_1 = \sigma_{n+1}$. Considera-se $l(i) = \sum_{j=1}^{i-1} t_{\sigma_j \sigma_{j+1}}$ a “latência” do i -ésimo nó da sequência s e que $l(1) = 0$. Em outras palavras, se $t_{\sigma_i \sigma_j}$ é o tempo de viagem entre os nós σ_i e σ_j , então $l(i)$ pode ser pensado como o tempo total de viagem do início do *tour*

σ_1 até o nó σ_i . Assim, se $s = (1, 3, 6, 2, 4, 7, 5, 1)$, por exemplo, então

$$l(4) = t_{13} + t_{36} + t_{62}$$

é a latência do nó $\sigma_4 = 2$. O MLP visa encontrar um ciclo hamiltoniano s em G cuja latência total — ou custo acumulado — $f(s) = \sum_{i=1}^{n+1} l(i)$ é mínima.

Observe que, diferente do TSP, a escolha do primeiro vértice da sequência é importante, visto que afeta as latências dos vértices seguintes e consequentemente a latência total do ciclo, para fins de consistência, considere que a sequência começa e termina no vértice 1 (i.e., $\sigma_1 = \sigma_{n+1} = 1$).

Para compreender a motivação do problema, considere que um motorista deseja realizar uma sequência de entregas para n clientes distintos. Para encontrar uma rota satisfatória em termos de tempo de viagem, o entregador poderia tratar o seu problema como um TSP, visando minimizar o custo total da própria viagem. Para satisfazer os clientes (i.e., realizar as entregas em um tempo razoável para todos eles), por outro lado, o entregador poderia tratar o problema como um MLP, tentando minimizar o tempo de entrega de cada um. Nesse caso, a latência $l(i)$ pode ser vista como o tempo necessário para realizar a entrega do i -ésimo cliente. Portanto, numa abordagem mais “altruísta”, o MLP busca minimizar a soma de todos os tempos de entrega (latências).

Por ter uma estrutura similar à do TSP, o problema pode ser abordado com o mesmo método baseado em busca local do capítulo anterior. Observe, porém, que a aplicação de qualquer um dos movimentos mostrados acarretaria drásticas mudanças no custo acumulado. Isso ocorre porque, diferente do caso do TSP, modificar um segmento no *tour* implica em alterar a latência de todos os nós seguintes, que dependem diretamente da disposição dos nós anteriores. Dito isso, as próximas seções descrevem uma maneira de recalcular o custo acumulado em um número constante de operações.

3.1.1 Subsequências

Seja $s = (\sigma_1, \dots, \sigma_{n+1})$ uma solução viável para uma instância do MLP. Uma subsequência é uma sequência $\sigma = (\sigma_i, \dots, \sigma_j)$, em que $i, j \in \{1, \dots, n+1\}$. Além disso, considera-se $C(\sigma) = \sum_{k=i}^j l(\sigma_k)$ o custo acumulado da subsequência σ . Note que o número total de subsequências associadas a s , dentre as quais constam até mesmo sequências compostas por um único nó, é $(n+1)^2$.

Concatenação de subsequências

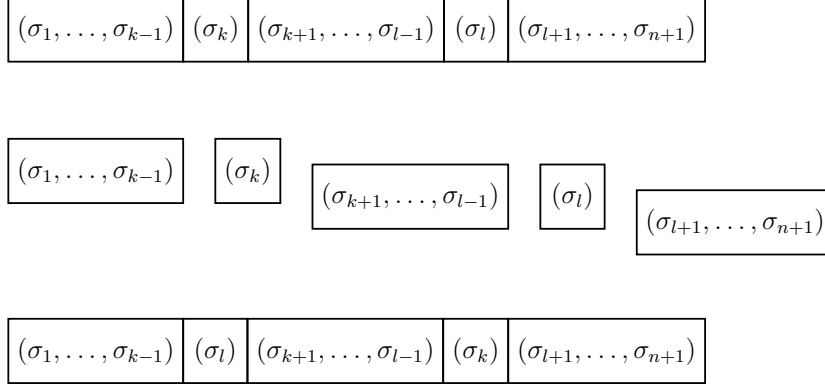


Figura 3.1: Quebra de uma solução em 5 subsequências e reconstrução durante o movimento *swap*.

3.1.2 Concatenação de subsequências

Sejam $\sigma' = (\sigma'_1, \dots, \sigma'_a)$ e $\sigma'' = (\sigma''_1, \dots, \sigma''_b)$ duas subsequências. A subsequência

$$\begin{aligned}\sigma &= \sigma' \oplus \sigma'' \\ &= (\sigma'_1, \dots, \sigma'_a, \sigma''_1, \dots, \sigma''_b)\end{aligned}$$

é resultado da concatenação de σ' e σ'' . A utilidade da concatenação de subsequências pode ser vista na Figura 3.1, na qual o movimento *swap* é aplicado entre os nós σ_k e σ_l de uma solução. A aplicação do movimento pode ser compreendida como a “quebra” da sequência da solução em um número fixo de subsequências — 5 neste caso —, que são então rearranjadas e concatenadas para formar uma nova solução. Todos os outros movimentos mostrados no capítulo anterior também podem ser vistos da mesma forma¹. Em outras palavras, se uma maneira eficiente de calcular o custo acumulado de uma subsequência obtida após a concatenação de outras duas subsequências for conhecida, o impacto dos movimentos de várias estruturas de vizinhança herdadas do TSP também poderá ser facilmente avaliado, permitindo que sejam reutilizadas.

Para compreender como isso pode ser feito, observe a Figura 3.2, que ilustra a concatenação de duas subsequências σ' e σ'' de tamanhos arbitrários $a \geq 1$ e $b \geq 1$, respectivamente. Na Figura 3.2a, as duas subsequências, bem como seus custos acumulados $f(\sigma')$ e $f(\sigma'')$, escritos de forma explícita, são exibidos. Enquanto isso, o custo acumulado da subsequência $\sigma = \sigma' \oplus \sigma''$ é mostrado de forma similar na Figura 3.2b. Note que tanto $f(\sigma')$ quanto $f(\sigma'')$ reaparecem no cálculo de $f(\sigma)$,

¹Observe que as subsequências podem ter a sua ordem invertida. Isso ocorre no caso do movimento *2-opt*, que divide a solução em três subsequências, inverte uma delas, e as concatena novamente para formar uma nova solução.

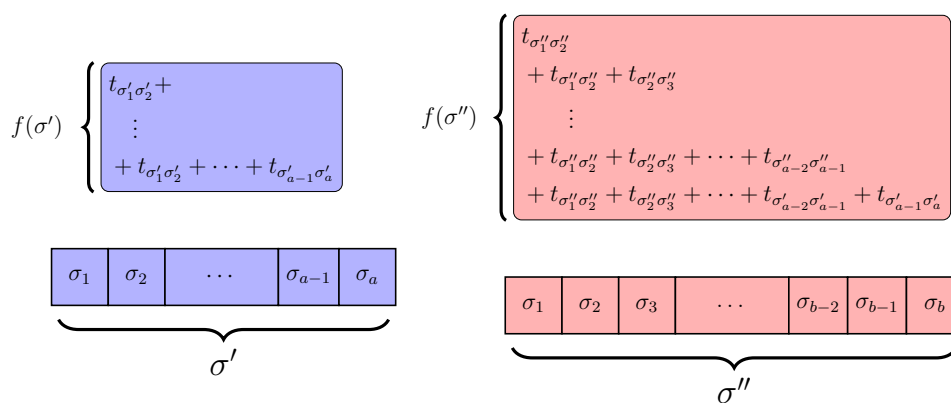
Concatenação de subsequências

sendo acompanhados pelo termo

$$\Delta = t_{\sigma'_1\sigma'_2} + \cdots + t_{\sigma'_{a-1}\sigma'_a} + t_{\sigma'_a\sigma'_1},$$

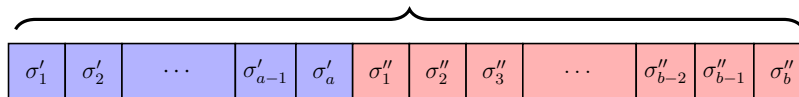
que aparece b vezes na soma. Isso significa que se $f(\sigma')$, $f(\sigma'')$ e Δ forem conhecidos previamente, pode-se calcular o custo acumulado da subsequência resultante da concatenação como

$$f(\sigma) = f(\sigma') + b\Delta + f(\sigma'').$$



(a) Antes da concatenação.

$$\sigma = \sigma' \oplus \sigma''$$



$$f(\sigma) \left\{ \begin{array}{l} \left\{ \begin{array}{l} t_{\sigma'_1 \sigma'_2} \\ \vdots \\ + t_{\sigma'_1 \sigma'_2} + \cdots + t_{\sigma'_{a-1} \sigma'_a} \end{array} \right\} \\ b\Delta \left\{ \begin{array}{l} + t_{\sigma'_1 \sigma'_2} + \cdots + t_{\sigma'_{a-1} \sigma'_a} + t_{\sigma'_a \sigma''_1} \\ + t_{\sigma'_1 \sigma'_2} + \cdots + t_{\sigma'_{a-1} \sigma'_a} + t_{\sigma'_a \sigma''_1} + t_{\sigma''_1 \sigma''_2} \\ + t_{\sigma'_1 \sigma'_2} + \cdots + t_{\sigma'_{a-1} \sigma'_a} + t_{\sigma'_a \sigma''_1} + t_{\sigma''_1 \sigma''_2} + t_{\sigma''_2 \sigma''_3} \\ \vdots \\ + t_{\sigma'_1 \sigma'_2} + \cdots + t_{\sigma'_{a-1} \sigma'_a} + t_{\sigma'_a \sigma''_1} + t_{\sigma''_1 \sigma''_2} + t_{\sigma''_2 \sigma''_3} + \cdots + t_{\sigma''_{b-2} \sigma''_{b-1}} \\ + t_{\sigma'_1 \sigma'_2} + \cdots + t_{\sigma'_{a-1} \sigma'_a} + t_{\sigma'_a \sigma''_1} + t_{\sigma''_1 \sigma''_2} + t_{\sigma''_2 \sigma''_3} + \cdots + t_{\sigma''_{b-2} \sigma''_{b-1}} + t_{\sigma''_{b-1} \sigma''_b} \end{array} \right\} \end{array} \right.$$

(b) Após a concatenação.

Figura 3.2: Concatenação de duas subsequências σ' e σ'' .

3.1.3 Estruturas auxiliares

Sejam $s = (\sigma_1, \dots, \sigma_{n+1})$ uma solução viável qualquer para uma instância do MLP, e σ uma subsequência de s . Além de seu custo acumulado $C(\sigma)$, a subsequência σ também é caracterizada pelos valores auxiliares $W(\sigma)$ (custo de atraso) e $T(\sigma)$ (duração). Caso σ seja uma subsequência composta por um único nó,

$$T(\sigma) = 0, \quad (3.1)$$

$$C(\sigma) = 0, \quad (3.2)$$

$$W(\sigma) = \begin{cases} 0 & \text{se } \sigma = (\sigma_1), \\ 1 & \text{caso contrário.} \end{cases} \quad (3.3)$$

Se, por outro lado, σ for resultado da concatenação de outras duas subsequências σ' e σ'' , então

$$T(\sigma' \oplus \sigma'') = T(\sigma') + t_{\sigma'_a \sigma''_1} + T(\sigma''), \quad (3.4)$$

$$C(\sigma' \oplus \sigma'') = C(\sigma') + W(\sigma'') (T(\sigma') + t_{\sigma'_a \sigma''_1}) + C(\sigma''), \quad (3.5)$$

$$W(\sigma' \oplus \sigma'') = W(\sigma') + W(\sigma''). \quad (3.6)$$

Conforme explicado na seção anterior, a equação (3.5) descreve o cálculo do custo acumulado da subsequência resultante a partir dos custos das duas subsequências envolvidas na operação de concatenação. Dessa forma, se $T(\sigma)$, $C(\sigma)$ e $W(\sigma)$ forem conhecidos previamente para todas as subsequências σ possíveis de uma solução s , qualquer outra solução vizinha s' obtida aplicando-se um dos movimentos do capítulo anterior pode ter seu custo facilmente avaliado através de algumas operações de soma.

Representação das estruturas auxiliares

O trecho de código a seguir mostra como as informações relevantes de uma subsequência podem ser armazenadas. Além dos valores já mostrados, o primeiro e último nós (**first** e **last**, respectivamente) de cada subsequência também devem ser armazenados para que se possa aplicar as equações (3.4) e (3.5). A partir de duas subsequências **sigma_1** e **sigma_2**, a função **Concatenate()** aplica as equações (3.4)–(3.6) e retorna a subsequência resultante **sigma**.

```

1 struct Subsequence
2 {
3     double T, C;
4     int W;
5     int first, last; // primeiro e ultimo nos da subsequencia
6     inline static Subsequence Concatenate(Subsequence &sigma_1,
                                           Subsequence &sigma_2)
```

```

7      {
8          Subsequence sigma;
9          double temp = t[sigma_1.last][sigma_2.first];
10         sigma.W = sigma_1.W + sigma_2.W;
11         sigma.T = sigma_1.T + temp + sigma_2.T;
12         sigma.C = sigma_1.C + sigma_2.W * (sigma_1.T + temp) + sigma_2.C;
13         sigma.first = sigma_1.first;
14         sigma.last = sigma_2.last;
15
16         return sigma;
17     }
18 };

```

Inicialização das estruturas auxiliares

O trecho de código a seguir mostra como os valores de todas as subsequências de uma solução podem ser inicializados. Os valores auxiliares de cada subsequência de uma solução são armazenados em `subseq_matrix`. O elemento `subseq_matrix[i][j]` armazena as informações da subsequência que começa no i -ésimo nó e termina no j -ésimo nó de s . No código, primeiramente, aplica-se as equações (3.1)–(3.3) para que as subsequências formadas por apenas um nó sejam obtidas. Em seguida, as outras subsequências são obtidas por meio de operações de concatenação.

```

1  // n: numero de nos da instancia
2  // s: solucao corrente
3  // subseq_matrix = vector<vector<Subsequence>>(n, vector<Subsequence>(n));
4
5  void UpdateAllSubseq(Solution *s, vector<vector<Subsequence>> &
6      subseq_matrix)
7  {
8      int n = s->sequence.size();
9
10     // subsequencias de um unico no
11     for (int i = 0; i < n; i++)
12     {
13         subseq_matrix[i][i].W = (i > 0);
14         subseq_matrix[i][i].C = 0;
15         subseq_matrix[i][i].T = 0;
16         subseq_matrix[i][i].first = s->sequence[i];
17         subseq_matrix[i][i].last = s->sequence[i];
18     }
19
20     for (int i = 0; i < n; i++)
21         for (int j = i + 1; j < n; j++)
22             subseq_matrix[i][j] = Subsequence::Concatenate(subseq_matrix[i][j-1], subseq_matrix[j][j]);

```



```

22
23     // subsequências invertidas
24     // (necessárias para o 2-opt)
25     for (int i = n - 1; i >= 0; i--)
26         for (int j = i - 1; j >= 0; j--)
27             subseq_matrix[i][j] = Subsequence::Concatenate(subseq_matrix [
                i][j+1], subseq_matrix[j][j]);
28 }

```

Após construir a matriz de subsequências a partir de uma solução inicial obtida através de um procedimento construtivo, pode-se, enfim, calcular o impacto de movimentos de maneira eficiente. Isso pode ser visto no trecho de código a seguir, no qual o custo de uma solução obtida após a aplicação do movimento *2-opt* foi obtido utilizando-se duas operações de concatenação. O valor `sigma_2.C` equivale ao custo acumulado da nova solução, que pode então ser comparado ao custo da solução anterior.

```

1  Solution *s = Construction();
2  subseq_matrix = vector<vector<Subsequence>>(n, vector<Subsequence>(n));
3  UpdateAllSubseq(s, subseq_matrix);
4  // movimento 2-opt remove as arestas {i-1, i} e {j, j+1},
5  // dividindo a solucao em tres subsequencias.
6  // no 2-opt a subsequencia que começa em i e termina em j e invertida
7  // por isso, ela foi acessada por meio de subseq_matrix[j][i], e nao
   subseq_matrix[i][j]
8  Subsequence sigma_1 = Subsequence::Concatenate(subseq_matrix[0][i-1],
   subseq_matrix[j][i]);
9  Subsequence sigma_2 = Subsequence::Concatenate(sigma_1, subseq_matrix[j
   +1][n]);
10 if (sigma_2.C < s->cost)
11 {
12     ApplyTwoOpt(s, i, j);
13     // A solucao foi modificada,
14     // logo, as subsequencias devem ser recalculadas
15     UpdateAllSubseq(s, subseq_matrix);
16 }

```

3.1.4 Atualização de subsequências

Sempre que uma nova solução é criada, a matriz de subsequências deve ser computada a partir do zero, exigindo $(n+1)^2 - n - 1$ operações de concatenação. Quando uma solução é modificada por um movimento, no entanto, algumas subsequências permanecem inalteradas. Isso pode ser visto mais facilmente na Figura 3.3, que mostra a matriz de subsequências de uma solução $s = (\sigma_1, \dots, \sigma_{15})$ alterada por um movimento qualquer entre os nós σ_9 e σ_{11} . A região cinza na figura representa os elementos na matriz que precisam ser atualizados. Os elementos restantes,

Concatenação de subsequências

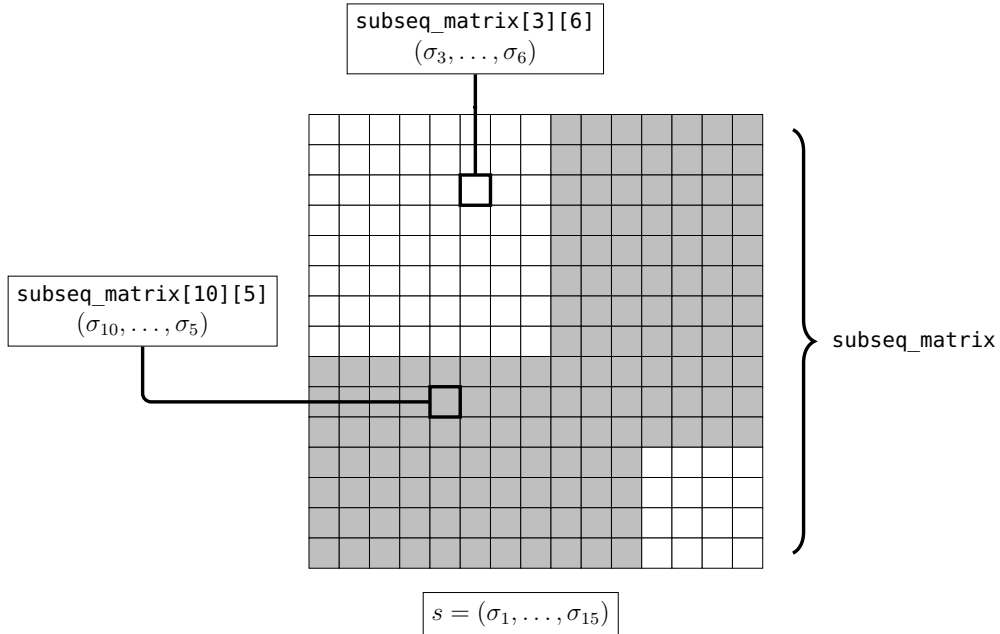


Figura 3.3: Matriz de subsequências de uma solução.

por sua vez, não precisam ser atualizados, já que as subsequências associadas não foram alteradas.

No geral, é sempre possível pensar em maneiras diferentes de atualizar a matriz de subsequências levando-se em conta as particularidades de cada movimento. Porém, isso exigiria a implementação de uma função de atualização de subsequências para cada movimento. Felizmente, a ideia mostrada na Figura 3.3 é aplicável a qualquer movimento, apresentando um bom equilíbrio entre facilidade de implementação e impacto no tempo de execução do algoritmo.

3.1.5 Aplicando o ILS ao MLP

Um algoritmo simples para resolver o MLP é descrito em [11]. Para resolver o problema, os autores utilizaram a meta-heurística ILS. Com exceção do procedimento construtivo, mostrado no Algoritmo 1, todos os procedimentos, incluindo perturbação, busca local e estruturas de vizinhança, foram implementados exatamente como descrito no capítulo anterior. Para fazer as buscas locais de maneira eficiente, as estruturas auxiliares descritas nas seções anteriores foram utilizadas. O algoritmo é um dos métodos heurísticos mais efetivos para resolver o MLP. Como exercício, o leitor é encorajado a replicá-lo. Para isso, convém utilizar como base o código desenvolvido no capítulo anterior, adicionando a estrutura **Subsequence** e a matriz de subsequências. Os resultados podem então ser comparados com os

do artigo citado².

Algorithm 1: Construção

```

1 Procedimento Construção()
2  $\alpha \leftarrow$  número aleatório em  $R$ 
3  $s \leftarrow \{1\}$ 
4 Inicialize a lista de candidatos  $CL$ 
5  $CL \leftarrow CL - \{1\}$ 
6  $r \leftarrow 1$ 
7 while  $CL \neq \emptyset$  do
8   Ordene  $CL$  em ordem crescente de acordo com sua distância ao nó  $r$ 
9   Escolha aleatoriamente  $c$  dentre os  $\alpha\%$  melhores candidatos de  $CL$ 
10   $s \leftarrow s \cup \{c\}$ 
11   $r \leftarrow c$ 
12   $CL \leftarrow CL - \{r\}$ 
13 return  $s$ 
14 end Construção

```

A Tabela 3.1 contém os valores médios de custo acumulado e tempo (em segundos) de resolução das instâncias após 10 execuções do algoritmo em um processador Intel® Core™ i7-7700 3.60GHz. Durante os experimentos, utilizou-se os parâmetros $I_{Max} = 10$, $I_{ILS} = \min\{100, n\}$ e $R = \{0, 1, 2, \dots, 25\}$.

²Os autores do artigo utilizaram um subconjunto de instâncias do TSP. Portanto, o mesmo leitor de instâncias pode ser usado.

Concatenação de subsequências

Tabela 3.1: Tempo e custo médios obtidos para cada instância.

Instância	Resultados		Instância	Resultados	
	Tempo	Custo		Tempo	Custo
att48	0.152	209320	kroA100	2.358	983128
bayg29	0.022	22230	kroA150	11.8	1825770
bays29	0.027	26862	kroB100	2.775	986008
berlin52	0.203	143721	kroB150	11.93	1786550
bier127	5.981	4545000	kroB200	28.93	2674354
brazil58	0.287	512361	kroC100	2.305	961324
brg180	16.85	174750	kroD100	2.881	976965
burma14	0.001	20315	kroE100	2.772	971266
ch130	5.787	349903.5	lin105	2.801	603910
ch150	10.22	444424	pr107	2.662	2026630
d198	27.71	1186728	pr124	3.701	3154350
dantzig42	0.111	12528	pr136	11.64	6199270
eil101	3.761	27513	pr144	6.214	3846140
eil51	0.233	10178	pr152	7.807	5064570
eil76	0.999	17976	pr76	0.784	3455240
fri26	0.015	10703	rat195	34.91	218673.8
gr120	5.93	363454	rat99	3.716	57986
gr137	6.401	4061500	rd100	2.671	340047
gr17	0.003	12994	si175	14.94	1808530
gr21	0.006	24345	st70	0.661	20557
gr24	0.01	13795	swiss42	0.083	22327
gr48	0.176	102378	u159	11.01	2972030
gr96	2.012	2097170	ulysses16	0.003	40392
hk48	0.145	247926	ulysses22	0.007	52064

Parte II

Algoritmos Exatos

4

Branch-and-Bound combinatório

O *Branch-and-Bound* (BnB) consiste em um algoritmo de enumeração implícita capaz de resolver problemas combinatórios como o TSP de maneira exata aproveitando-se da noção de *bounds*.

4.1 Relaxações

Todo problema de otimização combinatória possui um “espaço de soluções” associado. Aqui, entende-se por espaço de soluções o conjunto de todas as soluções válidas para uma certa instância do problema. O espaço de soluções para uma instância do TSP envolvendo o conjunto de vértices $\{1, 2, \dots, n\}$, por exemplo, é o conjunto de todas as permutações da sequência $(1, 2, \dots, n)$. Note que o espaço de soluções de um problema de otimização é diretamente moldado por suas restrições. Quanto mais restrições um problema de otimização combinatória possui, menor tende a ser o número de soluções que são válidas para ele. Em outras palavras, quanto mais restrito um problema de otimização combinatória é, menor é seu espaço de soluções associado. O TSP, por exemplo, possui duas restrições notórias: (i) cada uma das n cidades deve ser visitada exatamente uma vez; (ii) a sequência de visitas aos vértices deve formar um único *tour* (i.e., deve-se partir de uma cidade e retornar a ela). As figuras 4.1a e 4.1b apresentam soluções que não obedecem às restrições citadas, e que não pertencem ao espaço de soluções da versão original do TSP.

Ao retirar uma ou mais restrições de um problema de otimização, obtém-se um novo problema menos restrito. Diz-se que esse novo problema é uma “relaxação” do problema original. “Relaxar” uma restrição significa, portanto, removê-la do problema. Dito isso, há uma importante relação entre o espaço de soluções de um problema e o de suas relaxações. A Figura 4.2, por exemplo, ilustra o espaço de soluções de diferentes versões do TSP. Observe que o espaço de soluções da versão original do TSP está contido inteiramente no espaço de ambas as relaxações. Essa

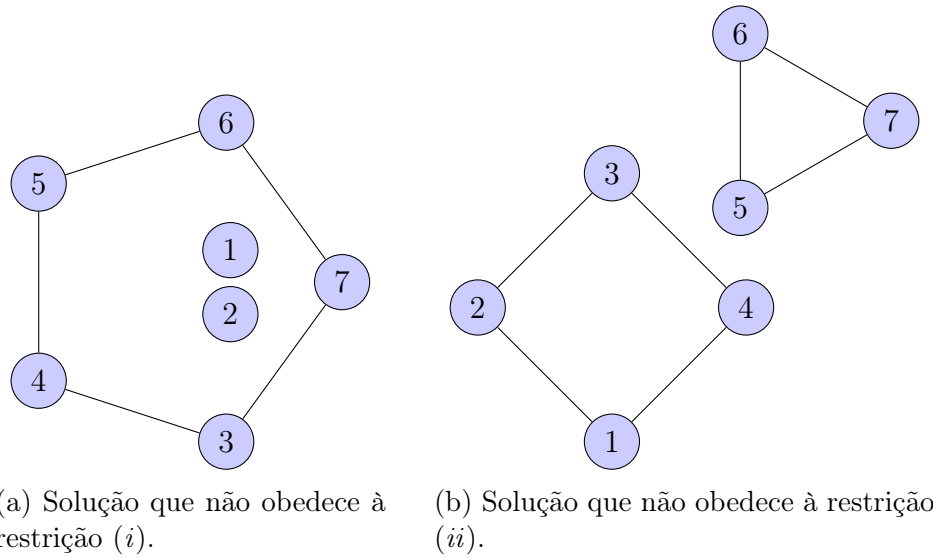


Figura 4.1: Exemplos de soluções inválidas.

propriedade é não apenas intuitiva — soluções para um problema mais restrito não deixam de ser válidas para um problema menos restrito —, como também é válida para qualquer problema de otimização, e será usada extensivamente nas próximas seções.

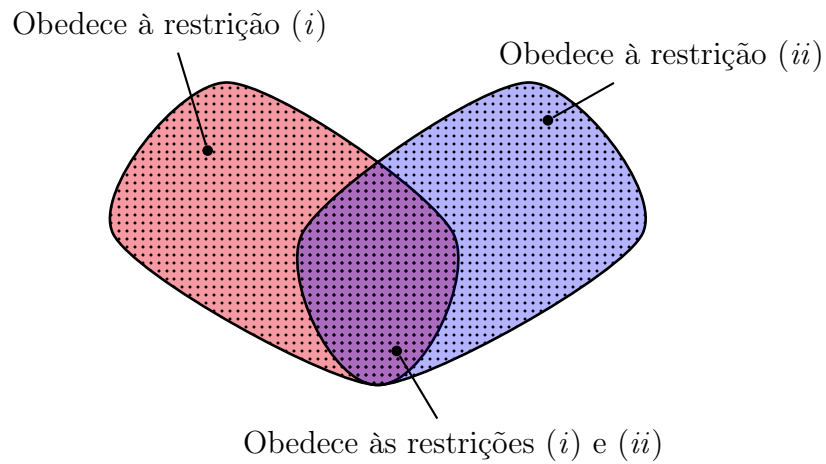


Figura 4.2: Espaços de soluções de diferentes versões do TSP.

4.2 Limitantes

Considere um problema de minimização qualquer e cuja solução ótima é $s^* \in \mathcal{S}$, em que $\mathcal{S}$ é seu espaço de soluções associado. Analogamente, considere que uma relaxação do mesmo problema está associada a um espaço de soluções $\mathcal{U}$ e uma solução ótima $u^* \in \mathcal{U}$. Já que o segundo problema consiste em uma relaxação do primeiro, então $\mathcal{S} \subseteq \mathcal{U}$, e portanto, a relação $f(u^*) \leq f(s^*)$ é sempre válida. Considere, agora, que $s' \in \mathcal{S}$ é uma solução qualquer — não necessariamente ótima — para o primeiro problema. Já que s^* é a solução ótima, então, por definição, $f(s^*) \leq f(s')$. Portanto, é garantido que o valor objetivo da solução ótima do primeiro problema se encontra dentro do intervalo $[f(u^*), f(s')]$. Por isso, diz-se que $f(u^*)$ é um limitante inferior (*lower bound*, LB) para o primeiro problema, enquanto $f(s')$ é um limitante superior (*upper bound*, UB).

O conceito de limitantes é útil, por exemplo, para determinar a qualidade de uma solução. Para ver isso, considere que s' é uma solução obtida de maneira heurística e que

$$100\% \times \frac{f(s') - f(u^*)}{f(u^*)}$$

é a diferença relativa entre $f(s')$ e $f(u^*)$, denominada *gap*. Note que se o *gap* for nulo, então $f(s') = f(u^*)$, o que significa que a solução s' é ótima, já que

$$f(u^*) \leq f(s^*) \leq f(s').$$

Por isso, quanto mais próximo de zero for o *gap*, maiores são as chances de que s' seja uma solução ótima para o problema.

O algoritmo *Branch-and-Bound* (BnB), apresentado neste capítulo, se baseia na ideia de diminuir o tamanho do intervalo $[f(u^*), f(s')]$ até que se possa garantir que a solução s' é ótima. O *gap* entre $f(s')$ e $f(u^*)$ funciona, portanto, como um critério de parada para o algoritmo, que finaliza apenas quando ele é nulo.

4.3 Algoritmo *Branch-and-Bound*

O Algoritmo 2 descreve o método *Branch-and-Bound* (BnB) para problemas de minimização. Primeiramente, o maior UB encontrado é inicializado com o valor ∞ (linha 2). Após isso, se uma heurística para o problema estiver disponível, ela é usada para gerar um UB válido (linhas 3–5). Em seguida, uma lista de subproblemas L é inicializada com o espaço $\mathcal{U}$, que corresponde ao espaço de soluções associado a uma relaxação qualquer do problema a ser resolvido. Após isso, um subproblema na lista L é escolhido e resolvido (linhas 8–10). Se a solução s' obtida for viável para o problema original, ela passa a ser a melhor solução,

e seu valor objetivo passa a ser o menor UB encontrado (linhas 11–14). Se, por outro lado, s' não for viável e seu valor objetivo não for maior que o melhor UB, o espaço associado a U é dividido em espaços mais restritos — e consequentemente menores —, que são então adicionados a L (linhas 15–18). O procedimento se repete enquanto a lista L não estiver vazia (linha 7).

Algorithm 2: BRANCH-AND-BOUND

```

Data: ..
Result:  $s^*$ 
1 begin
2    $UB \leftarrow \infty$ 
3   if heurística disponível then
4     Obter solução  $s'$  usando heurística
5      $UB \leftarrow f(s')$ 
6    $L \leftarrow \{\mathcal{U}\}$ 
7   while  $L \neq \emptyset$  do
8      $\mathcal{S}' \leftarrow \mathcal{U} \in L$ 
9      $L \leftarrow L \setminus \mathcal{S}'$ 
10     $s' \leftarrow \text{Resolver}(\mathcal{S}')$ 
11    if  $s'$  é viável then
12      if  $f(s') < UB$  then
13         $UB \leftarrow f(s')$ 
14         $s^* \leftarrow s'$ 
15    else
16      if  $f(s') \leq UB$  then
17        Dividir  $\mathcal{S}'$  em subproblemas mais restritos  $\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_k$ 
18         $L \leftarrow L \cup \{\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_k\}$ 
19  return  $s^*$ 

```

Para compreender a ideia geral por trás do Algoritmo 2, observe a Figura 2, que mostra o processo de subdivisão da região de soluções da relaxação de um problema de otimização qualquer. O espaço de soluções associados ao problema original e uma relaxação são representados por um círculo tracejado e uma região oval, respectivamente, na Figura 4.3a. Inicialmente, um LB foi obtido resolvendo-se a relaxação do problema na otimalidade, e apresenta o valor $f(u^*)$. Além disso, uma solução s' para o problema foi obtida de forma heurística, e possui um valor objetivo $f(s') = 110$. Nesse momento, a solução ótima do problema original (s^*) é desconhecida.

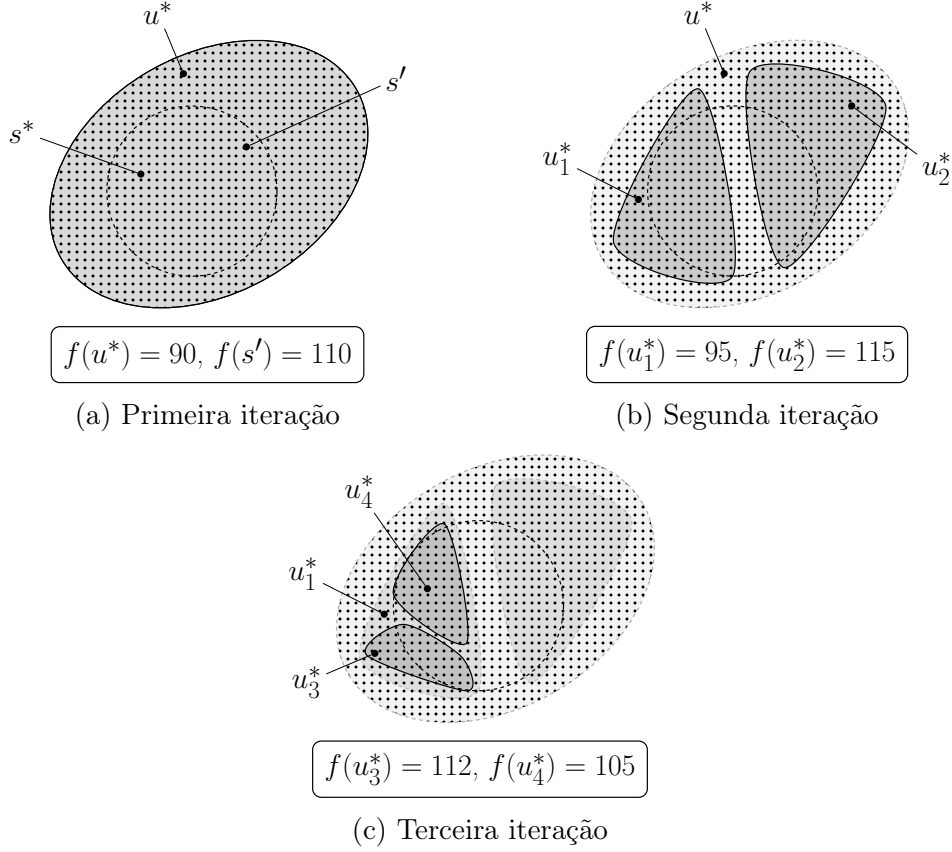


Figura 4.3: Exemplo ilustrativo de algumas das iterações do BnB.

Suponha, agora, que fôssemos capazes de dividir a região associada à relaxação do problema em duas novas regiões mais restritas que excluem a solução u^* . O processo, chamado de *branching*, é mostrado na Figura 4.3b, e deve ser feito de forma que a solução s^* ainda esteja dentro de ao menos uma das regiões resultantes. Novamente, os problemas associados às regiões resultantes são resolvidos na otimalidade, gerando as soluções inviáveis u_1^* e u_2^* . Note, no entanto, que a solução u_2^* possui um valor objetivo maior que o da melhor solução viável encontrada (s'). Portanto, o espaço da direita pode ser ignorado, e precisa-se explorar apenas a região associada à solução u_1^* .

Na terceira iteração, mostrada na Figura 4.3c, a região esquerda é dividida em duas novas regiões que excluem a solução u_1^* . Novamente, a solução u_3^* possui um valor objetivo maior que o melhor UB conhecido, e sua região pode ser ignorada. A solução u_4^* , por sua vez, é viável para o problema original, e possui um valor objetivo menor que o de s' , passando a ser o novo melhor UB encontrado. Observe, ainda, que não faz sentido dividir a região associada à solução u_4^* em regiões

menores. Fazer isso apenas geraria soluções cujo valor objetivo é maior ou igual ao atual melhor UB $f(u_4^*)$. Assim, já que não há mais nenhuma região a ser explorada, a solução u_4^* é a solução ótima para o problema.

Em suma, o BnB consiste em uma maneira inteligente de enumerar os elementos do espaço de soluções de um problema de otimização combinatória. O método é genérico, servindo como base para todos os algoritmos exatos apresentados no restante deste livro. Também é importante destacar o impacto positivo que o UB inicial $f(s')$ teve no exemplo mostrado. Se uma heurística não estivesse inicialmente disponível, e consequentemente o UB $f(s')$ não fosse conhecido, não teria sido possível ignorar a região associada à solução u_2^* . Consequentemente, determinar a solução ótima para o problema poderia exigir muito mais iterações. Por esse motivo, a elaboração de boas heurísticas é crucial para o bom desempenho dos algoritmos exatos.

4.4 Branch-and-bound para o TSP

Para aplicar o BnB a um problema de otimização combinatória, é necessário definir uma relaxação para o problema e um método de *branching*. Neste capítulo, será utilizado o *Assignment Problem* (AP), que é um problema de otimização combinatória que, se parametrizado de maneira apropriada, pode ser interpretado como uma relaxação para o TSP.

Tanto o AP quanto o método de *branching* necessários para aplicar o BnB ao TSP serão definidos nas próximas seções.

4.4.1 Transformação do TSP em um AP

Sejam U um conjunto de trabalhadores e W um conjunto de tarefas em que $|U| = |W|$. Designar um trabalhador $i \in U$ a uma tarefa $j \in W$ incorre um custo w_{ij} . O AP visa designar cada trabalhador a exatamente uma tarefa e cada tarefa a exatamente um trabalhador de forma que a soma total dos custos seja mínima. A Figura 4.4 mostra uma solução para uma instância do AP em que $U = \{1, 2, \dots, 5\}$ e $W = \{6, 7, \dots, 10\}$. O custo da solução é

$$w_{17} + w_{36} + w_{29} + w_{58} + w_{4,10}.$$

Para entender como o AP pode ser visto como uma relaxação do TSP, é necessário desenvolver um método que seja capaz de transformar uma instância arbitrária do TSP em um instância do AP. Considere, então, que o grafo completo $G = (V, E)$, em que $V = \{1, 2, \dots, n\}$, é uma instância qualquer do TSP. A ins-

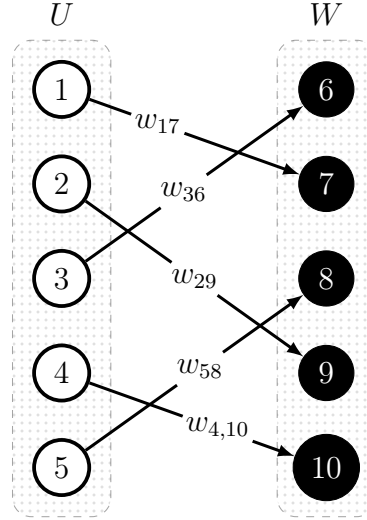


Figura 4.4: Exemplo de solução para uma instância do AP

tância do AP deve ser construída fazendo-se $U = V = W$ e

$$w_{ij} = \begin{cases} c_{ij} & \text{se } i \neq j \\ \infty & \text{se } i = j \end{cases}$$

para todo $i \in U$, $j \in W$ ¹. O resultado da transformação descrita é uma instância específica do AP que possui uma correspondência com a instância do TSP que a originou. Esse problema de otimização combinatória — que consiste em uma versão especial do AP — será chamado, daqui em diante, de AP_{TSP} .

A ligação entre o TSP e o AP_{TSP} pode ser compreendida observando-se as figuras 4.5a e 4.5b, que mostram duas soluções para uma instância do AP_{TSP} . As figuras 4.5c e 4.5d mostram uma representação alternativa das soluções das figuras 4.5a e 4.5b, respectivamente. É interessante notar que embora ambas as soluções sejam válidas para o AP_{TSP} , apenas a primeira delas é viável para o TSP. É fácil perceber, na verdade, que qualquer solução viável para o TSP também é viável para o AP_{TSP} . Por isso, o AP_{TSP} é uma relaxação do TSP.

A vantagem de usar o AP_{TSP} como relaxação para o TSP é que existem algoritmos que resolvem o AP em tempo polinomial. Um deles é o Algoritmo Húngaro, que é capaz de resolver o AP em tempo $\mathcal{O}(n^3)$, em que $n = |U| = |W|$. Uma implementação do Algoritmo Húngaro e do método de transformação do TSP no AP_{TSP} , acompanhada de um leitor de instâncias do TSP, pode ser obtida

¹Devido à natureza do AP, a transformação obriga cada nó a se conectar a algum outro nó. Fazer $w_{ij} = \infty$ para todo $i \in U$, $j \in W$, $i = j$ impede que um nó seja conectado a ele mesmo, já que o problema é de minimização.

no seguinte *link*: <https://github.com/carlosvinicius01/kit-opt/tree/master/BB-Combinat%C3%B3rio/algoritmo-hungaro>.

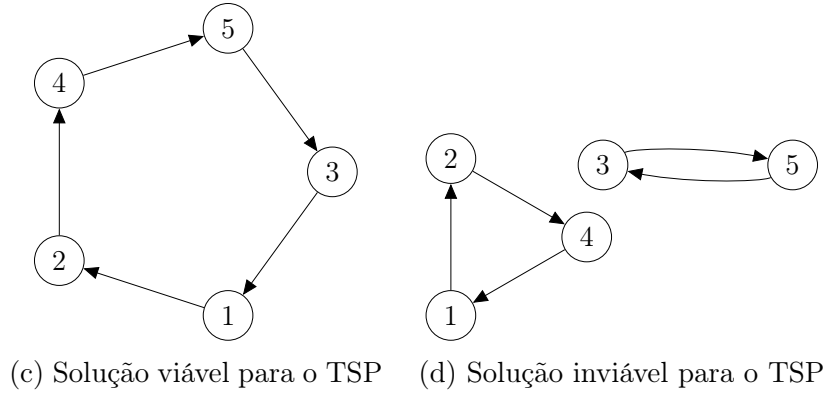
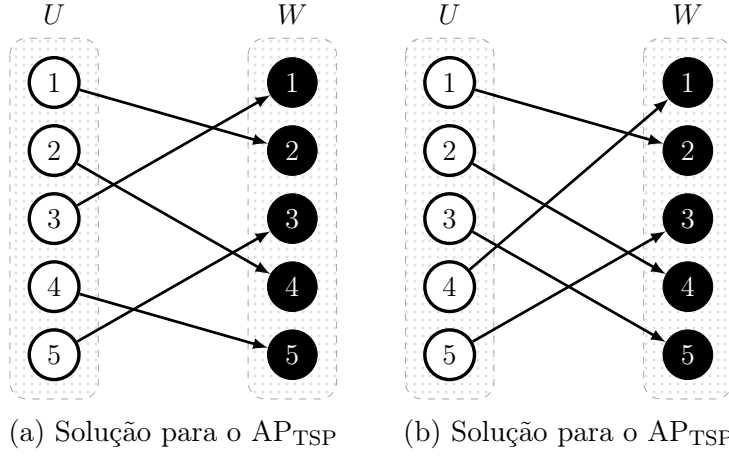


Figura 4.5: Soluções do AP_{TSP}

4.4.2 Regra de *branching*

Suponha que uma instância do TSP foi convertida em uma instância do AP_{TSP} , que foi então resolvida por meio do Algoritmo Húngaro. Sabe-se que se a solução obtida for viável para o TSP, ela é a solução ótima. Caso contrário, é necessário dividir a região de soluções em regiões diferentes, de forma que a solução obtida não esteja contida em nenhuma das regiões resultantes. Uma forma simples de garantir isso é escolher o menor *subtour* da solução e proibir cada um de seus arcos separadamente, conforme mostrado na Figura 4.6. Na figura, inicia-se com uma solução para o AP_{TSP} , que é inviável para o TSP por possuir dois *subtours*. Nesse caso, o menor *subtour* é aquele que contém os nós 3 e 5. Então, duas novas

instâncias do AP_{TSP} são geradas: a primeira delas proíbe o arco $(5, 3)$, enquanto a segunda proíbe o arco $(3, 5)$. Um arco a pode ser proibido fazendo-se $c_a = \infty$.

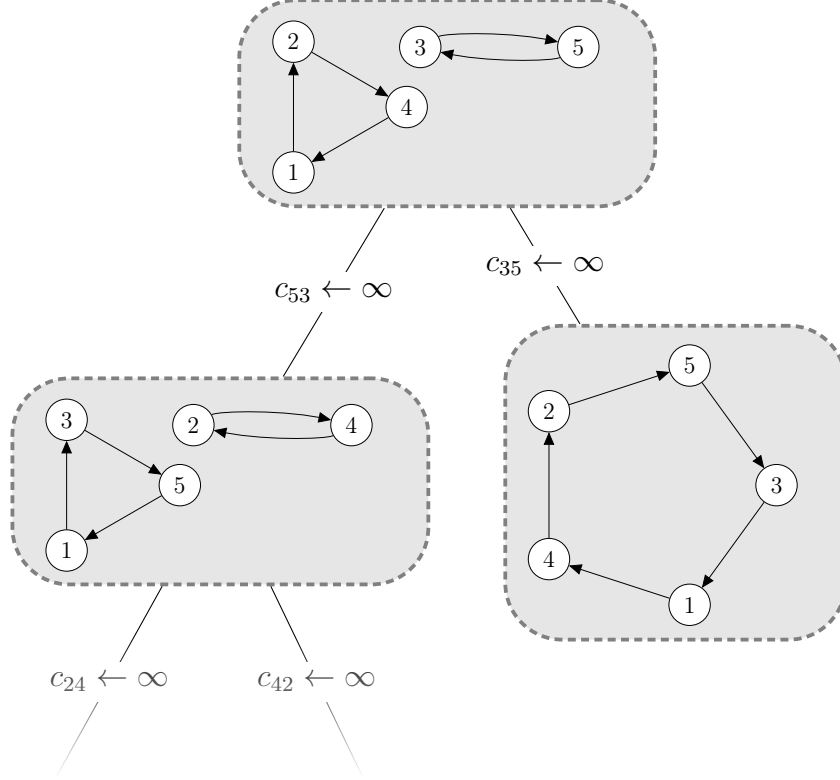


Figura 4.6: Iterações do BnB representadas por árvore

Na instância em que $c_{35} = \infty$, uma solução viável para o TSP foi obtida, e não é mais necessário subdividir o espaço de soluções nesse caso. A instância na qual $c_{53} = \infty$, por outro lado, gerou uma solução inviável. Por isso, o mesmo mecanismo é utilizado, dessa vez proibindo-se os arcos $(2, 4)$ e $(4, 2)$.

Assim, o BnB pode ser visto como um algoritmo que atua sobre uma árvore de decisões. Cada nó da árvore representa uma instância para um problema, que possui um espaço de soluções associado. Se, ao resolver o subproblema associado ao nó corrente na árvore, uma solução inviável for encontrada, deve-se gerar nós “filhos” restringindo-se ainda mais a instância associada. Além disso, os nós filhos sempre herdam as características dos “pais”, que, neste caso, são os arcos proibidos.

4.4.3 Estratégias de *branching*

Em tese, o próximo nó a ser visitado pode ser escolhido de forma arbitrária, desde que a árvore seja explorada por completo. No entanto, a ordem na qual os nós

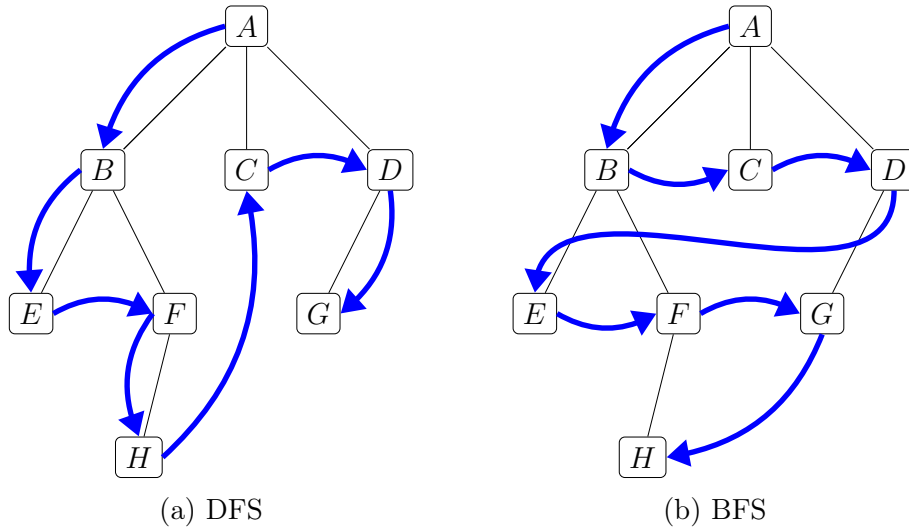


Figura 4.7: DFS e BFS

são explorados possui um impacto significativo na prática. O método utilizado para escolher o próximo nó a ser explorado dentre os nós restantes na árvore é denominado “estratégia de *branching*”.

As duas estratégias de *branching* mais simples são a busca por largura (*Breadth First Search*, *BFS*) e a busca por profundidade (*Depth First Search*, *DFS*), ilustradas na Figura 4.7, na qual uma árvore arbitrária é explorada de duas maneiras diferentes. A ordem na qual os nós são escolhidos é representada pelas setas azuis, e tem início no nó raiz *A*. Na *DFS*, a ordem na qual os nós são visitados se assemelha a um “mergulho”. A partir do nó raiz *A*, mergulha-se em direção ao filho esquerdo (ou direito) até que isso não seja mais possível, como no caso do nó *E*, que não possui filhos. Em seguida, retorna-se ao ancestral mais próximo em busca de um nó filho não visitado. O processo se repete até que toda a árvore seja visitada. Na *BFS*, os nós são ordenados com base na sua distância em relação ao nó raiz (i.e., sua altura ou nível).

Note que a estratégia de *branching* deve ser executada no decorrer do algoritmo, ao mesmo tempo em que a árvore é gerada. Observe, ainda, que uma vez que um nó é visitado e seus possíveis filhos são adicionados à árvore, ele pode ser deletado. No caso da *DFS*, por exemplo, inicia-se a árvore apenas com o nó raiz, e os nós filhos são exaustivamente explorados e deletados um galho (*branch*) por vez. Na *BFS*, por outro lado, sempre que os nós de uma determinada altura são visitados, todos os seus filhos são adicionados à árvore, causando, muitas vezes, um crescimento exponencial no número de nós não visitados. Por isso, a *DFS* tende a ser mais eficaz que a *BFS*, principalmente em termos de uso de memória.

Por fim, há, ainda, a estratégia de busca pelo menor LB (*Best Bound Search*,

BBS). Nela, o próximo nó da árvore a ser escolhido é o que possui o menor LB associado. Para que cada nó tenha um LB, pode-se fazer com que os nós filhos herdem o LB do pai, obtido logo após a resolução do AP_{TSP} .

4.5 Implementação

A implementação do Algoritmo Húngaro fornecida na seção anterior retorna a solução na forma de uma matriz. A solução da Figura 4.5b, por exemplo, é representada pela matriz

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix},$$

cujo elemento A_{ij} assume o valor 1 se e somente se o trabalhador $i \in U$ foi designado à tarefa $j \in W$. Dito isso, o leitor deve implementar um algoritmo que é capaz de detectar se a matriz representa uma solução que contém *subtours*. Caso a solução possua *subtours*, o algoritmo deve, ainda, ser capaz de listar todos os arcos do menor deles. O trecho de código a seguir apresenta uma sugestão de estrutura de dados para representar a solução mostrada anteriormente em função de seus *subtours*:

```
1  std::vector<std::vector<int>> subtours = {{1,2,4,1}, {3,5,3}};
```

Em posse de um algoritmo que é capaz de resolver o AP_{TSP} e listar os *subtours*, resta apenas a tarefa de implementar o BnB, que pode ser facilitada utilizando-se a representação por meio da árvore mostrada na seção anterior. Cada nó da árvore, que está associado a um AP_{TSP} , pode ser representado conforme o seguinte trecho de código:

```
1  struct Node
2  {
3      std::vector<pair<int, int>> forbidden_arcs; // lista de arcos
        proibidos do nó
4      std::vector<std::vector<int>> subtour; // conjunto de subtours da
        solucao
5      double lower_bound; // custo total da solucao do algoritmo húngaro
6      int chosen; // indice do menor subtour
7      bool feasible; // indica se a solucao do AP_TSP e viavel
8  };
```

É recomendado resolver e atualizar cada nó, utilizando o algoritmo húngaro, de forma separada, em uma função do tipo:

```

1 void updateNode(Node *node)
2 {
3     hungarian_problem_t p;
4
5     cost = ...; // matriz de custos atualizada com os arcos proibidos
6     n = ...; // dimensao da matriz
7
8     int mode = HUNGARIAN_MODE_MINIMIZE_COST;
9     hungarian_init(&p, cost, n, n, mode);
10
11     node->lower_bound = hungarian_solve(&p);
12
13     node->subtours = Subtours(p); // detectar o conjunto de subtours
14     node->choosen = getChoosen(p); // pegar o indice do menor subtour
15     node->feasible = isFeasible(p); // verificar viabilidade
16
17     hungarian_free(&p);
18
19 }

```

Observe que não é preciso criar uma matriz de custos para cada nó. É possível modificar a matriz de custos original com base na lista de arcos proibidos em cada nó. Para proibir o arco (2, 5), por exemplo, basta usar o seguinte trecho de código:

```

1 cost[2][5] = 99999999; /* usar std::limits<double>::infinity() pode causar
    erros de imprecisao numerica na implementacao do algoritmo hungaro
    usada */

```

O trecho de código a seguir contém sugestões de como implementar algumas rotinas do BnB utilizando-se uma lista encadeada para representar a árvore.

```

1 Node root; // no raiz
2 updateNode(root); // resolver e atualizar a raiz a partir da instancia
    original
3
4 /* criacao da arvore */
5 tree = std::list<Node>;
6 tree.push_back(root);
7
8 double upper_bound = stod(argv[2])+1; // passar como argumento o valor
    otimo da instancia + 1
9
10 while (!tree.empty())
11 {
12     auto node = branchingStrategy(); // escolher um dos nos da arvore e
        remove-lo
13
14
15

```

```

16  if (node->feasible)
17  {
18      if(node->lower_bound < upper_bound)
19      {
20          upper_bound = node->lower_bound;
21      }
22      continue;
23  }
24
25  /* Adicionando os filhos */
26  for (int i = 0; i < node.subtour[node.chosen].size() - 1; i++) //
      iterar por todos os arcos do subtour escolhido
27  {
28      Node n;
29      n.arcos_proibidos = node.arcos_proibidos;
30
31      std::pair<int,int> forbidden_arc = {
32          node.subtour[node.chosen][i],
33          node.subtour[node.chosen][i + 1]
34      };
35
36      n.forbidden_arcs.push_back(forbidden_arc);
37      updateNode(n);
38      if(n->lower_bound <= upper_bound)
39          tree.push_back(n); //inserir novos nos na arvore
40  }
41  }

```

A vantagem de representar a árvore do BnB por meio de uma lista encadeada é que tanto a BFS quanto a DFS podem ser implementadas de forma simples. A BFS pode ser implementada escolhendo-se sempre o primeiro elemento (nó) da lista, enquanto na DFS deve-se escolher sempre o último elemento da lista. A corretude desse método é garantida desde que (i) todo nó seja deletado da lista assim que for visitado e (ii) os novos filhos sejam adicionados sempre ao fim (cauda) da lista. O leitor é encorajado a conferir essas propriedades por conta própria através de exemplos pequenos.

A estratégia de busca pelo menor LB também pode ser implementada usando uma lista encadeada para representar a árvore. Nesse caso, no entanto, o uso da estrutura de dados *binary heap*, disponível em C++ através do *container priority_queue*, é mais apropriado.

4.6 Benchmark

Todas as estratégias de *branching* mencionadas devem ser implementadas. Pode-se utilizar um parâmetro passado como argumento para o executável para escolher a

estratégia a ser utilizada antes da execução do algoritmo, por exemplo. A Tabela 4.1 contém os valores ótimos e os tempos médios (em segundos) de resolução das instâncias em cada estratégia de *branching* após 10 execuções do algoritmo em um processador Intel® Core™ i7-7700 3.60GHz.

Tabela 4.1: Tempo e custo médios obtidos para cada instância.

Instância	Custo	Tempo		
		BFS	DFS	BBS
bayg29	1610	0.327	0.286	0.340
bays29	2020	1.142	0.979	1.115
burma14	3323	2.596	0.631	0.732
fri26	937	18.83	4.163	5.302
gr17	2085	22997	64.98	90.45
gr21	2707	0.001	0.001	0.001
gr24	1272	0.156	0.126	0.155
ulysses16	6859	992.1	12.70	16.22

5

Relaxação Lagrangiana

A relaxação lagrangiana procura facilitar a obtenção de LBs para um problema de otimização combinatória através da remoção de restrições dificultantes. As restrições removidas são, então, tratadas por meio de penalizações na função objetivo. O objetivo é obter uma versão do problema que seja mais fácil de resolver, e que sirva como uma boa relaxação, provendo LBs que sejam os mais próximos possíveis do valor ótimo do problema original. Este capítulo apresenta uma abordagem de resolução do TSP através do método da relaxação lagrangiana que, unido ao BnB através de uma regra de *branching* apropriada, supera o BnB combinatório do capítulo anterior em termos de performance, sendo capaz de resolver instâncias maiores.

5.1 Relaxação de um Problema Linear Inteiro

Sejam $\mathbf{A} \in \mathbb{R}^{m \times n}$ e $\mathbf{B} \in \mathbb{R}^{p \times n}$ matrizes, $\mathbf{b} \in \mathbb{R}^m$ e $\mathbf{d} \in \mathbb{R}^p$ vetores colunas, e $\mathbf{c} \in \mathbb{R}^n$ um vetor linha. Considere o Problema Linear Inteiro (*Integer Linear Program*, ILP)

$$\min_{\mathbf{x} \in X} \{ \mathbf{c}\mathbf{x} \mid \mathbf{A}\mathbf{x} \geq \mathbf{b}, \mathbf{B}\mathbf{x} \geq \mathbf{d} \}, \quad (\text{P})$$

em que $X = \mathbb{R}_+^{n_R} \mathbb{Z}_+^{n_I}$, i.e., o vetor coluna $\mathbf{x}$ representa um conjunto composto por n_R variáveis reais não negativas e n_I variáveis inteiras não negativas ($n_R + n_I = n$). Suponha, agora, que a presença das restrições $\mathbf{A}\mathbf{x} \geq \mathbf{b}$ impõem um desafio significativo ao problema. É possível retirar essas restrições e penalizar a função objetivo caso sejam violadas, conforme o problema

$$\min_{\mathbf{x} \in X} \{ \mathbf{c}\mathbf{x} + \boldsymbol{\lambda}(\mathbf{b} - \mathbf{A}\mathbf{x}) \mid \mathbf{B}\mathbf{x} \geq \mathbf{d} \}, \quad (\text{PR}_{\boldsymbol{\lambda}})$$

em que cada componente do vetor linha $\boldsymbol{\lambda} \in \mathbb{R}_+$ está associada a uma das restrições retiradas do problema original (P).

Embora não seja capaz de garantir que violações não ocorram, o termo adicional na função objetivo do problema (PR_λ) incentiva que as restrições $\mathbf{Ax} \geq \mathbf{b}$ sejam respeitadas. Para perceber isso, seja $\mathbf{a}_i$ a i -ésima linha da matriz $\mathbf{A}$. Note que o problema em questão é de minimização, e se $\mathbf{a}_i\mathbf{x} < b_i$ para alguma restrição (ou linha) $i \in \{1, \dots, m\}$, um número não negativo (i.e., uma penalização) $\lambda_i(b_i - \mathbf{a}_i\mathbf{x})$ será adicionado à função objetivo, aumentando-a.

Teorema 1. Para qualquer vetor $\boldsymbol{\lambda} \geq \mathbf{0}$, (PR_λ) é uma relaxação de (P) .

Demonstração. Já que (P) possui todas as restrições de (PR_λ) , então, o espaço de soluções de (P) está contido no espaço de soluções de (PR_λ) . Para mostrar que o valor ótimo de (PR_λ) é um LB para o valor ótimo de (P) , observe que, para qualquer solução viável $\mathbf{x}$ de (P) ,

$$(\mathbf{b} - \mathbf{Ax}) \leq \mathbf{0}.$$

Além disso, já que foi especificado que $\boldsymbol{\lambda} \geq \mathbf{0}$, então

$$\boldsymbol{\lambda}(\mathbf{b} - \mathbf{Ax}) \leq \mathbf{0},$$

e, por consequência,

$$\mathbf{cx} + \boldsymbol{\lambda}(\mathbf{b} - \mathbf{Ax}) \leq \mathbf{cx},$$

completando a demonstração. $\square$

Teorema 2. Se $\mathbf{x}^*$ é uma solução ótima de (PR_λ) para algum $\boldsymbol{\lambda} \geq \mathbf{0}$ e $\boldsymbol{\lambda}(\mathbf{b} - \mathbf{Ax}^*) = \mathbf{0}$ e $\mathbf{Ax}^* \geq \mathbf{b}$, então $\mathbf{x}^*$ também é uma solução ótima de (P) .

Demonstração. Primeiramente, $\mathbf{x}^*$ é viável para (P) , já que $\mathbf{Ax}^* \geq \mathbf{b}$. Além disso, o fato de que $\mathbf{x}^*$ é uma solução ótima para (PR_λ) significa que, para um $\mathbf{x}$ arbitrário e viável para (P) (e consequentemente para (PR_λ)),

$$\mathbf{cx}^* + \boldsymbol{\lambda}(\mathbf{b} - \mathbf{Ax}^*) \leq \mathbf{cx} + \boldsymbol{\lambda}(\mathbf{b} - \mathbf{Ax}),$$

mas $\boldsymbol{\lambda}(\mathbf{b} - \mathbf{Ax}^*) = \mathbf{0}$, então

$$\mathbf{cx}^* \leq \mathbf{cx} + \boldsymbol{\lambda}(\mathbf{b} - \mathbf{Ax}),$$

e de acordo com o Teorema 1,

$$\mathbf{cx} + \boldsymbol{\lambda}(\mathbf{b} - \mathbf{Ax}) \leq \mathbf{cx},$$

logo,

$$\mathbf{cx}^* \leq \mathbf{cx} + \boldsymbol{\lambda}(\mathbf{b} - \mathbf{Ax}) \leq \mathbf{cx},$$

e $\mathbf{x}^*$ é, de fato, uma solução ótima para (P) . $\square$

5.1.1 Exemplo com ILP simples

Considere o ILP

$$\min_{(x_1, x_2, x_3) \in \{0,1\}^3} \{5x_1 + 4x_2 + 2x_3 \mid 2x_1 + 3x_2 + 4x_3 \geq 5\}. \quad (\text{P1})$$

Já que o problema original (P1) possui apenas uma restrição, pode-se associá-la a um único penalizador λ (que desta vez é um número, e não um vetor). Ao aplicar a mesma técnica discutida acima, obtém-se o novo ILP

$$\min_{(x_1, x_2, x_3) \in \{0,1\}^3} \{5x_1 + 4x_2 + 2x_3 + \lambda(5 - (2x_1 + 3x_2 + 4x_3))\}. \quad (\text{PR}_\lambda 1)$$

A solução ótima de (PR_λ1) depende do valor do penalizador λ , que deve ser escolhido antes da resolução do problema. Se $\lambda = 0$, por exemplo, o valor ótimo é obtido fazendo-se $x_1 = x_2 = x_3 = 0$, tornando nulo o valor da função objetivo.

Para analisar o comportamento do problema em função de λ , convém reagrupar os termos da função objetivo de (PR_λ1), obtendo-se a expressão equivalente

$$(5 - 2\lambda)x_1 + (4 - 3\lambda)x_2 + (2 - 4\lambda)x_3 + 5\lambda. \quad (5.1)$$

Observando-se a Expressão (5.1), e sabendo-se que (PR_λ1) não possui restrições além do domínio das variáveis x_1 , x_2 e x_3 , é fácil perceber que é necessário fazer com que

$$x_1 = \begin{cases} 1 & \text{se } \lambda > 5/2 \\ 0 & \text{caso contrário.} \end{cases}$$

para que a solução ótima seja obtida. Em outras palavras, fazer $x_1 = 1$ só irá diminuir a função objetivo se $5 - 2\lambda$ (termo que multiplica x_1 na Expressão (5.1)) for menor que zero.

Seguindo-se uma lógica similar,

$$x_2 = \begin{cases} 1 & \text{se } \lambda > 4/3 \\ 0 & \text{caso contrário,} \end{cases}$$

e

$$x_3 = \begin{cases} 1 & \text{se } \lambda > 1/2 \\ 0 & \text{caso contrário.} \end{cases}$$

Dessa forma, com o intuito de determinar o valor ótimo de (PR_λ1), é possível saber, para cada valor de λ , quais variáveis assumem o valor 1, e quais assumem o valor 0. A Figura 5.1 ilustra o comportamento do valor ótimo de (PR_λ1) em função do penalizador λ . Observe que $\lambda = 4/3$ foi o penalizador que proveu a melhor estimativa do valor ótimo do problema original (P1).

A próxima seção formaliza a tarefa que consiste em encontrar os melhores penalizadores para problemas como (PR_λ1).

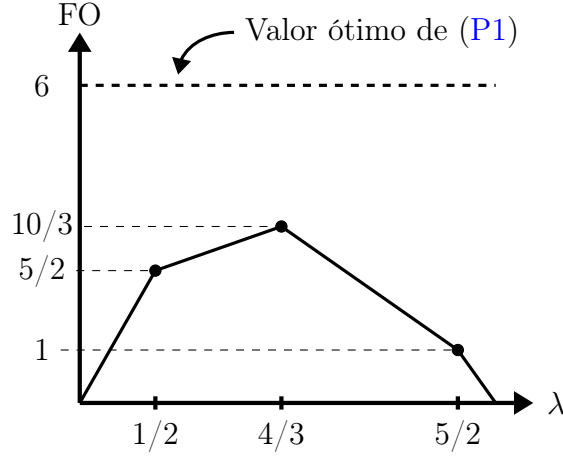


Figura 5.1: Valor da função objetivo de $(PR_\lambda 1)$ em função de λ .

5.2 Dual lagrangiano

Um tema introduzido no capítulo anterior — e recorrente no restante deste livro — é a redução do *gap* entre o LB provido pela relaxação e o valor ótimo do problema original. No exemplo anterior, uma análise feita a partir de um ILP revelou que o valor ótimo de sua relaxação depende diretamente do valor escolhido para o penalizador. Essa noção pode ser facilmente generalizada, e conclui-se que, ironicamente, encontrar os melhores penalizadores para (PR_λ) pode ser formulado como outro problema de otimização

$$\max_{\lambda \geq 0} \left\{ \min_{x \in X} \{ c x + \lambda(b - A x) \mid B x \geq d \} \right\}. \quad (D)$$

Por apresentar dois problemas de otimização interdependentes, o problema (D) pode parecer inicialmente confuso. Lembre-se, no entanto, de que como mostrado no exemplo anterior, o valor ótimo do problema interno depende do vetor de penalizadores λ . Em outras palavras, sendo $v(PR_\lambda)$ o valor objetivo da solução ótima de (PR_λ) , o problema (D) pode ser formulado alternativamente como

$$\max_{\lambda \geq 0} \{ v(PR_\lambda) \}. \quad (D)$$

Diferente de (P) e (PR_λ) , que possuem como variáveis as componentes de x , (D) possui como variáveis as componentes do vetor de penalizadores λ . Resolver (D) significa encontrar o melhor LB possível para o problema original (P) a partir de sua relaxação (PR_λ) . No exemplo anterior, a solução ótima para o dual lagrangiano associado a $(PR_\lambda 1)$ é $\lambda = 4/3$, e o valor objetivo referente à solução é $10/3$.

Resta, então, descobrir uma maneira de resolver o dual lagrangiano. Infelizmente, embora contínua, a função na Figura 5.1 claramente não é diferenciável, não permitindo o uso de técnicas do cálculo diferencial para determinar seus pontos máximos. No entanto, a função possui uma característica interessante: ela é côncava (i.e., a região abaixo da função forma um conjunto convexo) e, portanto, possui um ótimo global, ou seja, um único pico. Essas características, que são generalizadas para qualquer problema semelhante a (PR_λ) no seguinte teorema, serão exploradas mais adiante.

Teorema 3. *A função $v(\text{PR}_\lambda)$ é sempre côncava.*

Demonstração. Qualquer ILP pode ser reformulado em função da envoltória convexa de sua região viável. Por isso, o problema (PR_λ) pode ser reescrito como

$$\min_{\mathbf{x} \geq \mathbf{0}} \{ \mathbf{c}\mathbf{x} + \boldsymbol{\lambda}(\mathbf{b} - \mathbf{A}\mathbf{x}) \mid \mathbf{x} \in \text{conv} \{ \mathbf{x} \in X \mid \mathbf{B}\mathbf{x} \geq \mathbf{d} \} \},$$

em que $\text{conv}\{.\}$ denota a envoltória convexa do conjunto $\{.\}$. Sabe-se, ainda, que a solução ótima de um problema linear reside em um de seus pontos extremos. Assim, sendo $\{\mathbf{x}^1, \dots, \mathbf{x}^K\}$ o conjunto de pontos extremos de $\text{conv} \{ \mathbf{x} \in X \mid \mathbf{B}\mathbf{x} \geq \mathbf{d} \}$, (PR_λ) pode ser novamente reformulado como

$$\min_{k=1, \dots, K} \{ \mathbf{c}\mathbf{x}^k + \boldsymbol{\lambda}(\mathbf{b} - \mathbf{A}\mathbf{x}^k) \}.$$

Isso significa que a função $v(\text{PR}_\lambda)$ equivale ao mínimo de uma ou mais funções lineares de $\boldsymbol{\lambda}$, sendo, portanto, côncava, de acordo com um resultado conhecido da Análise Convexa. Um exemplo disso pode ser visto na Figura 5.2. $\square$

O Teorema 3 estabelece que resolver o dual lagrangiano se reduz a encontrar o máximo global de uma função contínua. As próximas seções descrevem os passos necessários para resolver esse problema de otimização usando uma teoria análoga às técnicas vistas em cursos de Cálculo.

5.3 Subgradiente

Dada uma função côncava $f : \mathbb{R}^m \mapsto \mathbb{R}$, o vetor coluna $\mathbf{g} \in \mathbb{R}^m$ é considerado um subgradiente no ponto $\boldsymbol{\lambda}_0 \in \mathbb{R}^m$ se

$$f(\boldsymbol{\lambda}) \leq \mathbf{g}(\boldsymbol{\lambda} - \boldsymbol{\lambda}_0) + f(\boldsymbol{\lambda}_0) \quad (5.2)$$

para qualquer $\boldsymbol{\lambda} \in \mathbb{R}^m$.

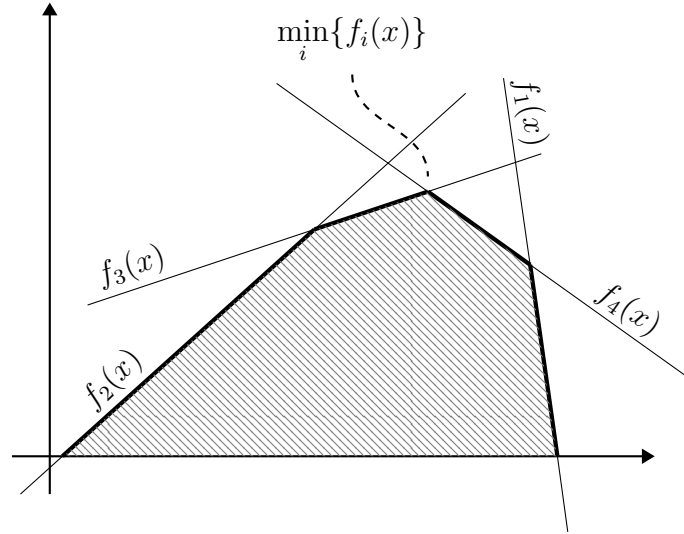


Figura 5.2: Exemplo de função cônica obtida calculando-se o mínimo entre quatro funções lineares quaisquer

De acordo com a Inequação (5.2), para que $\mathbf{g}$ seja um subgradiente válido da função $f(\boldsymbol{\lambda})$ no ponto $\boldsymbol{\lambda}_0$, o gráfico da função linear

$$h(\boldsymbol{\lambda}) = f(\boldsymbol{\lambda}_0) + \mathbf{g}(\boldsymbol{\lambda} - \boldsymbol{\lambda}_0)$$

precisa estar sempre acima do gráfico de $f(\boldsymbol{\lambda})$. Isso pode ser visto mais facilmente nas Figuras 5.3a–5.3b, que apresentam exemplos de três vetores subgradientes para uma função de uma variável. Dois dos três vetores são subgradientes válidos para o ponto $\lambda = 1/2$, enquanto o outro é válido para o ponto $\lambda = 5$. Na Figura 5.3a, as retas tracejadas representam o gráfico da função $h(\lambda)$ para cada vetor subgradiente. Note que tais funções obedecem à Inequação (5.2), sendo sempre maiores ou iguais à função $f(\lambda)$. Os vetores ortogonais a tais retas não são exatamente os subgradientes em si, mas extensões deles: já que $f(\lambda)$ é uma função de uma única variável, os subgradientes devem pertencer a $\mathbb{R}$. Os verdadeiros subgradientes, que são vetores unidimensionais no caso do exemplo, se encontram na Figura 5.3b, espalhados verticalmente para melhor visualização.

A informação mais importante a ser extraída da Figura 5.3b é que, olhando-se a partir do ponto λ_0 , o subgradiente associado “aponta” para o ótimo global de $f(\lambda)$ ($\lambda = 8/3$). Na verdade, se $f(\lambda)$ for uma função diferenciável de uma única variável, o subgradiente no ponto λ_0 é exatamente $df(\lambda_0)/d\lambda$. O vetor subgradiente nada mais é, portanto, que uma generalização do conceito de gradiente para funções côncavas não diferenciáveis. Dessa forma, uma maneira de resolver o dual lagrangiano é começar a partir de um ponto $\boldsymbol{\lambda}_0$ e “dar um pequeno passo” na direção do vetor subgradiente até o ponto $\boldsymbol{\lambda}_0 + \varepsilon \mathbf{g}$, em que ε é um número muito

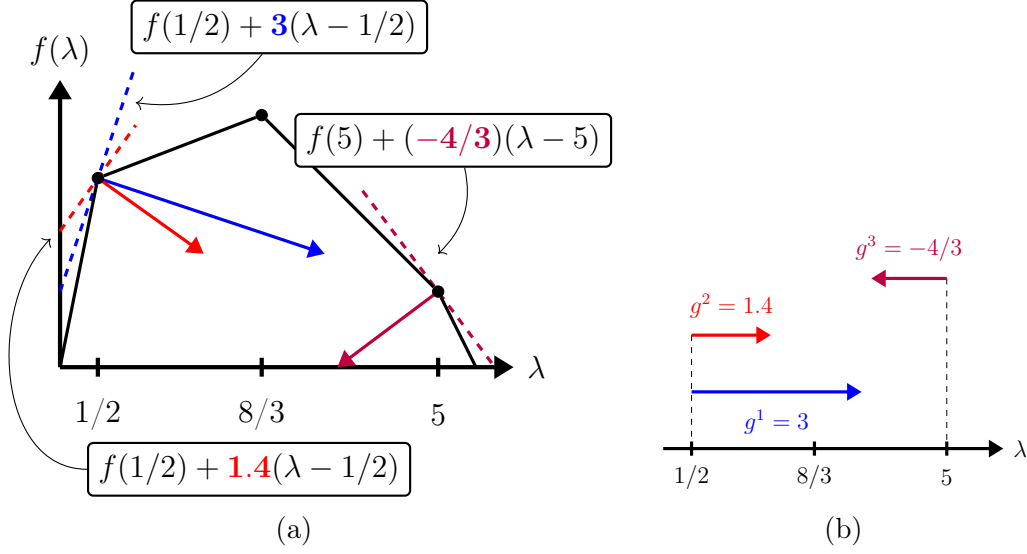


Figura 5.3: Exemplos de vetores subgradientes para uma função côncava de uma variável.

pequeno. Repetindo-se o processo até que o aumento em $f(\lambda)$ seja marginal, é possível chegar ao ótimo global de $f(\cdot)$.

Teorema 4. Se x^* é a solução ótima de (PR_λ) para $\lambda = \lambda_0$, então $b - Ax^*$ é subgradiente de (D) no ponto λ_0 .

Demonstração. Sejam $f(\lambda)$ e $f(\lambda_0)$ os valores de $v(PR_\lambda)$ para os penalizadores λ e λ_0 , respectivamente. Sabe-se que,

$$\begin{aligned} f(\lambda_0) &= \min_{x \in X} \{ cx + \lambda_0(b - Ax) \mid Bx \geq d \} \\ &= cx^* + \lambda_0(b - Ax^*). \end{aligned}$$

Além disso, embora a solução x^* seja ótima para (PR_λ) quando o penalizador é λ_0 , isso não necessariamente é verdade para o penalizador arbitrário λ . Portanto,

$$\begin{aligned} f(\lambda) &\leq cx^* + \lambda(b - Ax^*) \\ &\leq cx^* + \lambda(b - Ax^*) + \lambda_0(b - Ax^*) - \lambda_0(b - Ax^*) \\ &\leq f(\lambda_0) + \lambda(b - Ax^*) - \lambda_0(b - Ax^*) \\ &\leq (b - Ax^*)(\lambda - \lambda_0) + f(\lambda_0), \end{aligned}$$

e o vetor $g = (b - Ax^*)$ é, de fato, um subgradiente válido no ponto λ_0 , de acordo com a definição de subgradiente. $\square$

O Teorema 4 nos dá uma maneira simples e direta de obter um vetor subgradiente para qualquer ponto λ . Em posse disso, o procedimento para resolver o dual lagrangiano, que baseia-se na ideia descrita anteriormente, é apresentado na próxima seção.

5.4 Método do subgradiente

Como discutido na seção anterior, a ideia por trás do método de resolução do dual lagrangiano é caminhar em direção ao ótimo global de (D) através do vetor subgradiente. O procedimento depende, dentre outros fatores, de um cálculo de um tamanho de passo e de um critério de parada adequado, para que possa convergir numa velocidade razoável. Uma das maneiras de implementar esse procedimento, proposta em REF, é descrita no Algoritmo 3.

Algorithm 3: SUBGRADIENT METHOD.

Data: heuristic upper bound UB , $\varepsilon_{\min}$, $k_{\max}$, c , A , b

Result: Optimal penalizers λ^*

```

1 Procedure SolveLagrangianDual() is
2    $\lambda \leftarrow 0$ 
3    $\varepsilon \leftarrow 1$ 
4    $k \leftarrow 0$ 
5   while stop criterion not met do
6      $x^* \leftarrow$  optimal solution of (PR $_{\lambda}$ )
7      $w \leftarrow cx^* + \lambda(b - Ax^*)$ 
8     if  $w > w^*$  then
9        $w^* \leftarrow w$ 
10       $\lambda^* \leftarrow \lambda$ 
11       $k \leftarrow 0$ 
12    else
13       $k \leftarrow k + 1$ 
14      if  $k \geq k_{\max}$  then
15         $k \leftarrow 0$ 
16         $\varepsilon \leftarrow \varepsilon/2$ 
17       $\mu \leftarrow \varepsilon(UB - w) / ((b - Ax^*)(b - Ax^*))$ 
18       $\lambda \leftarrow \lambda + \mu(b - Ax^*)$ 
19      stop criterion  $\leftarrow \varepsilon < \varepsilon_{\min}$  or  $(Ax^* \geq b$  and  $\lambda(b - Ax^*) =$ 
20         $0)$  or  $w \geq UB$ 
  return  $\lambda^*$ 

```

O Algoritmo 3 recebe como um de seus parâmetros um UB obtido de maneira heurística. Após isso, a solução ótima de (PR_λ) , bem como o seu LB, são obtidos nas linhas 6–7. Se o LB obtido na iteração atual for melhor que o melhor LB w^* , então w^* e λ^* são atualizados, e o número de iterações k é resetado (linhas 8–11). Caso contrário, o número de iterações é incrementado na linha 13. Além disso, se o valor do LB não melhorar (i.e., o valor objetivo do dual lagrangiano não aumentar) após $k_{\max}$ iterações, o número de iterações é resetado e o valor de ε é reduzido pela metade (linhas 14–16), resultando em tamanhos de passo cada vez menores. Então, um pequeno passo na direção do subgradiente $\mathbf{b} - \mathbf{A}\mathbf{x}^*$ é dado na linha 18. O fator que determina o tamanho do passo é o número real μ , que é atualizado na linha 17.

O processo se repete até que o valor de ε seja menor que um valor muito pequeno $\varepsilon_{\min}$. Em tese, isso significa que o algoritmo convergiu, e a solução λ não pode ser melhorada de forma significativa, sendo suficientemente próxima do ótimo global. Outra possibilidade (mais rara) é a de que condições semelhantes às estabelecidas no Teorema 2 sejam atingidas, caso no qual o Algoritmo também encerra.

5.5 Qualidade do Dual Lagrangiano

O dual lagrangiano neste capítulo foi introduzido a partir de um ILP. Sendo assim, é natural se perguntar o quão bom o LB obtido ao resolver o problema é se comparado a outras relaxações. A relaxação mais intuitiva de (P) é, definitivamente, sua relaxação linear

$$\min_{\mathbf{x} \geq \mathbf{0}} \{ \mathbf{c}\mathbf{x} \mid \mathbf{A}\mathbf{x} \geq \mathbf{b}, \mathbf{B}\mathbf{x} \geq \mathbf{d} \}, \quad (\text{P}_{\text{lin}})$$

que difere de (P) apenas pelo fato de que todas as variáveis em $\mathbf{x}$ agora pertencem ao conjunto dos números reais.

Teorema 5. *O valor ótimo de (D) é maior ou igual ao valor ótimo de (P_{lin}) .*

Demonstração. O problema (PR_λ) se torna menos restrito retirando-se suas restrições de integralidade. Portanto,

$$\max_{\lambda \geq \mathbf{0}} \left\{ \min_{\mathbf{x} \in X} \{ \mathbf{c}\mathbf{x} + \lambda(\mathbf{b} - \mathbf{A}\mathbf{x}) \mid \mathbf{B}\mathbf{x} \geq \mathbf{d} \} \right\} \geq \max_{\lambda \geq \mathbf{0}} \left\{ \min_{\mathbf{x} \geq \mathbf{0}} \{ \mathbf{c}\mathbf{x} + \lambda(\mathbf{b} - \mathbf{A}\mathbf{x}) \mid \mathbf{B}\mathbf{x} \geq \mathbf{d} \} \right\}.$$

Observe, ainda, que o lado direito da inequação acima pode ser reescrito como

$$\max_{\lambda \geq \mathbf{0}} \left\{ \lambda \mathbf{b} + \min_{\mathbf{x} \geq \mathbf{0}} \{ (\mathbf{c} - \lambda \mathbf{A})\mathbf{x} \mid \mathbf{B}\mathbf{x} \geq \mathbf{d} \} \right\}.$$

Substituindo-se o problema de minimização interno pelo seu dual, o problema anterior pode ser reescrito novamente como

$$\max_{\lambda \geq 0} \left\{ \lambda \mathbf{b} + \max_{\mathbf{y} \geq 0} \{ \mathbf{y} \mathbf{d} \mid \mathbf{y} \mathbf{B} \leq (\mathbf{c} - \lambda \mathbf{A}) \} \right\} = \max_{\lambda \geq 0, \mathbf{y} \geq 0} \{ \lambda \mathbf{b} + \mathbf{y} \mathbf{d} \mid \mathbf{y} \mathbf{B} + \lambda \mathbf{A} \leq \mathbf{c} \}.$$

Por fim, substituindo-se o problema resultante pelo seu dual, obtém-se

$$\min_{\mathbf{x} \geq 0} \{ \mathbf{c} \mathbf{x} \mid \mathbf{A} \mathbf{x} \geq \mathbf{b}, \mathbf{B} \mathbf{x} \geq \mathbf{d} \},$$

que é exatamente o problema $(\mathbf{P}_{\text{lin}})$. $\square$

O Teorema 5 é muito importante, pois estabelece que, no pior dos casos, o LB fornecido pelo dual lagrangiano de um ILP é tão bom quanto o LB fornecido pela sua relaxação linear, podendo, por vezes, ser ainda melhor. Uma consequência direta disso é que se $(\mathbf{P})$ já for um problema linear (i.e., $X = \{ \mathbf{x} \in \mathbb{R}^n \mid \mathbf{x} \geq \mathbf{0} \}$), então o valor ótimo de $(\mathbf{D})$ é igual ao de $(\mathbf{P})$. O seguinte Teorema permite compreender mais profundamente a qualidade do LB associado ao dual lagrangiano.

Teorema 6. *O problema linear*

$$\min_{\mathbf{x} \geq 0} \{ \mathbf{c} \mathbf{x} \mid \mathbf{A} \mathbf{x} \geq \mathbf{b}, \mathbf{x} \in \text{conv} \{ \mathbf{B} \mathbf{x} \geq \mathbf{d} \mid \mathbf{x} \in X \} \} \quad (\text{CG})$$

possui o mesmo valor ótimo que $(\mathbf{D})$.

Demonstração. O dual lagrangiano de (CG) é

$$\max_{\lambda \geq 0} \left\{ \min_{\mathbf{x} \geq 0} \{ \mathbf{c} \mathbf{x} + \lambda (\mathbf{b} - \mathbf{A} \mathbf{x}) \mid \mathbf{x} \in \text{conv} \{ \mathbf{B} \mathbf{x} \geq \mathbf{d} \mid \mathbf{x} \in X \} \} \right\},$$

e pode ser reescrito como

$$\max_{\lambda \geq 0} \left\{ \min_{\mathbf{x} \in X} \{ \mathbf{c} \mathbf{x} + \lambda (\mathbf{b} - \mathbf{A} \mathbf{x}) \mid \mathbf{B} \mathbf{x} \geq \mathbf{d} \} \right\},$$

que é exatamente igual ao problema $(\mathbf{D})$. Observe que, apesar de em geral ser difícil, é possível escrever a envoltória convexa em (CG) como um conjunto de restrições lineares, logo (CG) é um problema puramente linear. Portanto, segue do Teorema 5 que, como discutido anteriormente, a solução ótima de $(\mathbf{D})$ possui o mesmo valor objetivo que a solução ótima de (CG) . $\square$

O resultado descrito no Teorema 6 estabelece uma equivalência entre $(\mathbf{D})$ e (CG) . Essencialmente, um problema é dual do outro. Observe que embora seja claramente uma relaxação de $(\mathbf{P})$, (CG) não é meramente uma relaxação linear de

(P). A diferença entre (P_{lin}) e (CG) é que o último captura os pontos inteiros do conjunto $\{Bx \geq d \mid x \in X\}$ por meio de uma envoltória convexa. Se, no entanto,

$$\{Bx \geq d \mid x \geq 0\} = \text{conv} \{Bx \geq d \mid x \in X\},$$

diz-se que o problema original (P) possui a Propriedade da Integralidade. Nesse caso, tanto (CG) quanto (D) e (P_{lin}) fornecerão exatamente o mesmo LB. Para um melhor entendimento, consulte a Figura 5.4, que resume as informações discutidas por meio de uma interpretação geométrica. De forma similar, a Figura 5.5 ilustra o caso em que a região formada pelas restrições $Bx \geq d$ é igual à sua própria envoltória convexa (i.e., as restrições $Bx \geq d$ obedecem à propriedade da integralidade).

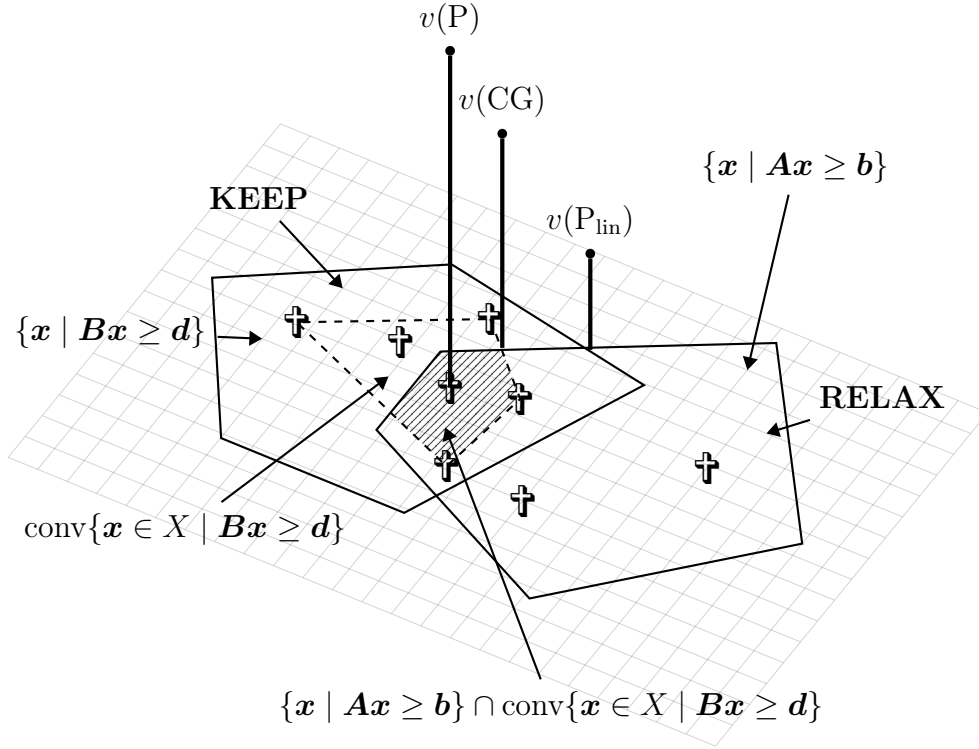


Figura 5.4: Representação geométrica das relaxações de um ILP.

5.6 Relaxação e dual lagrangiano do TSP

Suponha que a variável binária x_{ij} assume o valor 1 se uma aresta de um grafo completo $G = (V, E)$ for utilizada. A partir dessa variável, o TSP pode ser formulado

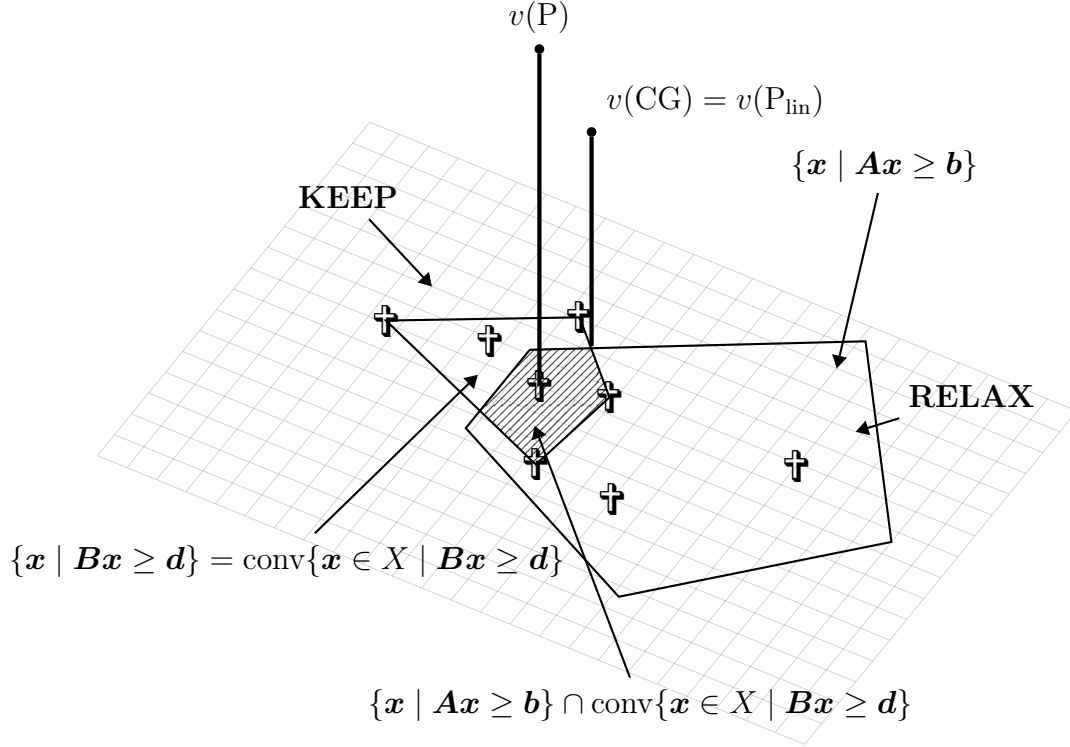


Figura 5.5: Representação geométrica do caso em que o problema obedece à propriedade da integralidade.

como o ILP

$$\begin{aligned} \min \quad & \sum_{i \in V, j > i} c_{ij} x_{ij} & (\text{TSP}) \\ \text{s.a.} \quad & \sum_{j > i} x_{ij} + \sum_{j < i} x_{ji} = 2, & i \in V, \\ & \sum_{i \in S} \sum_{j \notin S} x_{ij} \geq 1, & S \subset V, S \neq \emptyset, \\ & x_{ij} \in \{0, 1\}, & i, j \in V, j > i. \end{aligned} \quad (5.3)$$

$$\sum_{i \in S} \sum_{j \notin S} x_{ij} \geq 1, \quad S \subset V, S \neq \emptyset, \quad (5.4)$$

$$x_{ij} \in \{0, 1\}, \quad i, j \in V, j > i. \quad (5.5)$$

Na formulação (TSP), a função objetivo visa minimizar o custo total de viagem. As restrições (5.3) impõem que o grau de todos os vértices do grafo resultante seja igual a 2. As restrições (5.4), por sua vez, garantem que o grafo resultante será conexo, e portanto, formará um ciclo Hamiltoniano. Por fim, as restrições (5.5) descrevem a natureza das variáveis x_{ij} .

Neste caso, convém relaxar as restrições de grau para todos os vértices exceto o vértice $i = 0$. Isso significa que, na versão relaxada de (TSP) abordada neste

Relaxação Lagrangiana

capítulo, as restrições (5.3) serão removidas para todos os vértices $i \in V, i \neq 0$. Note, porém, que as restrições (5.3) são de igualdade. É possível, porém, reescrevê-las através de duas desigualdades como

$$\begin{aligned} \sum_{j>i} x_{ij} + \sum_{j<i} x_{ji} &\geq 2, i \in V, \\ -\sum_{j>i} x_{ij} - \sum_{j<i} x_{ji} &\geq -2, i \in V. \end{aligned} \tag{5.3}$$

A relaxação lagrangiana de (TSP) pode então ser escrita como

$$\begin{aligned} \min \sum_{i \in V, j>i} c_{ij} x_{ij} + \sum_{i>0} \lambda_i^+ \left(2 - \sum_{j>i} x_{ij} - \sum_{j<i} x_{ji} \right) \\ + \sum_{i>0} \lambda_i^- \left(\sum_{j>i} x_{ij} + \sum_{j<i} x_{ji} - 2 \right) \end{aligned} \tag{TSP}_\lambda \tag{5.6}$$

$$\text{s.a. } \sum_{j>0} x_{0j} = 2, \tag{5.6}$$

$$\sum_{i \in S} \sum_{j \notin S} x_{ij} \geq 1, \quad S \subset V, S \neq \emptyset, \tag{5.7}$$

$$x_{ij} \in \{0, 1\}, \quad i, j \in V, j > i. \tag{5.8}$$

A formulação (TSP_λ) possui dois vetores de penalização λ^+, λ^- não negativos, associados às versões com desigualdade das restrições (5.3). É possível simplificar a função objetivo significativamente reorganizando os termos e obtendo a expressão equivalente

$$\sum_{i \in V, j>i} c_{ij} x_{ij} + \sum_{i>0} \lambda_i^+ \left(2 - \sum_{j>i} x_{ij} - \sum_{j<i} x_{ji} \right) - \sum_{i>0} \lambda_i^- \left(2 - \sum_{j>i} x_{ij} - \sum_{j<i} x_{ji} \right)$$

ou, ainda,

$$\sum_{i \in V, j>i} c_{ij} x_{ij} + \sum_{i>0} (\lambda_i^+ - \lambda_i^-) \left(2 - \sum_{j>i} x_{ij} - \sum_{j<i} x_{ji} \right).$$

Já que $\lambda_i^+ \geq 0$ e $\lambda_i^- \geq 0$ para todo $i \in V \setminus \{0\}$, então $\lambda_i^+ - \lambda_i^-$ pode assumir qualquer valor real. Portanto, é possível substituir o termo $(\lambda_i^+ - \lambda_i^-)$ por um único termo irrestrito $\lambda_i \in \mathbb{R}$, obtendo-se a expressão

$$\sum_{i \in V, j>i} c_{ij} x_{ij} + \sum_{i>0} \lambda_i \left(2 - \sum_{j>i} x_{ij} - \sum_{j<i} x_{ji} \right),$$

o que faz sentido, já que diferente das restrições do tipo $\mathbf{Ax} \geq \mathbf{b}$, que podem ser violadas somente se $\mathbf{Ax} < \mathbf{b}$, as restrições de igualdade como (5.3) podem ser violadas tanto para “mais” quanto para “menos”, e é necessário penalizá-las conforme adequado. Reorganizando-se novamente, e supondo a existência de um penalizador “dummy” $\lambda_0 = 0$, obtém-se a expressão

$$\sum_{i \in V, j > i} c_{ij} x_{ij} - \sum_{i \in V} \lambda_i \left(\sum_{j > i} x_{ij} + \sum_{j < i} x_{ji} \right) + 2 \sum_{i > 0} \lambda_i, \quad (5.9)$$

Analisando-se a Expressão (5.9), pode-se perceber que variável x_{ij} é multiplicada não só pelo respectivo custo c_{ij} , mas também pelos penalizadores $-\lambda_i$ e $-\lambda_j$. Finalmente, o problema (TSP_λ) pode ser formulado como

$$\min \sum_{i \in V, j > i} (c_{ij} - \lambda_i - \lambda_j) x_{ij} + 2 \sum_{i > 0} \lambda_i \quad (\text{TSP}_\lambda)$$

$$\text{s.t.} \quad \sum_{j > 0} x_{0j} = 2, \quad (5.10)$$

$$\sum_{i \in S} \sum_{j \notin S} x_{ij} \geq 1, \quad S \subset V, S \neq \emptyset, \quad (5.11)$$

$$x_{ij} \in \{0, 1\}, \quad i, j \in V, j > i. \quad (5.12)$$

Em suma, (TSP_λ) se resume a encontrar um grafo conexo no conjunto de vértices $V' = \{1, 2, \dots, n\}$ e conectá-lo ao nó $i = 0$ por meio de exatamente duas arestas. O custo de cada aresta $\{i, j\}$ depende de c_{ij} e dos penalizadores λ_i e λ_j , e a soma total dos custos do grafo obtido deve ser mínima.

Exemplos de soluções para uma instância do (TSP_λ) podem ser vistas nas Figuras 5.6a–5.6c. Como esperado, em todos os exemplos, o nó $i = 0$ está sempre conectado a exatamente outros dois nós. As figuras diferem, principalmente, no número de nós que viola as restrições (5.3). Na Figura 5.6a, por exemplo, os nós 8, 3 e 7 possuem grau 3, enquanto os nós 5, 4 e 6 possuem grau 1. A Figura 5.6b, por outro lado, possui menos violações, com a maioria dos nós tendo grau 2, com exceção dos nós 8, 4, 7 e 2. Por fim, a Figura 5.6c mostra uma solução viável tanto para (TSP_λ) quanto para (TSP) , não violando nenhuma restrição de grau.

5.6.1 Resolução do MS1TP

Agora, resta encontrar uma maneira eficiente de resolver (TSP_λ) . Para isso, primeiramente, considere um outro problema parecido, intitulado Problema da Árvore Geradora Mínima (*Minimum Spanning Tree*, MST). O MST visa, dado um grafo com pesos G , encontrar um subgrafo conexo e sem ciclos, e cuja soma dos

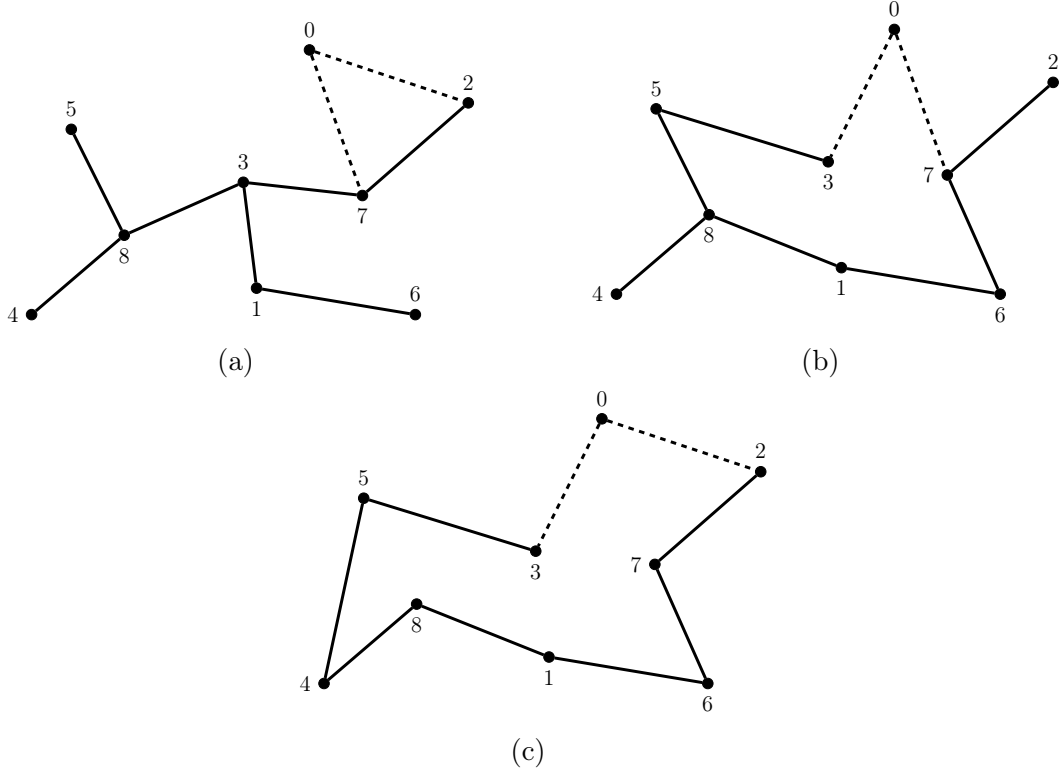


Figura 5.6: Exemplos de soluções para (TSP_λ) .

pesos das arestas é a menor possível. Uma solução para o MST em um grafo $G = (V = \{1, \dots, 13\}, E = V^2)$ é ilustrada na Figura 5.7a (desconsidere o nó 0 ilustrado).

Agora, considere que deseja-se resolver o (TSP_λ) para o grafo $G' = (V' = \{0, 1, \dots, 13\}, E' = V'^2)$, e que T é a MST do grafo G . Conforme ilustrado na Figura 5.7b, a 1-árvore mínima de G' pode ser encontrada conectando-se o vértice 0 aos dois vértices mais próximos em T . Em outras palavras, para resolver (TSP_λ) no grafo $G = (V = \{0, 1, \dots, n\}, E = V^2)$, é necessário apenas (i) encontrar a MST do grafo $G' = (V \setminus \{0\}, (V \setminus \{0\})^2)$ e (ii) conectar o nó 0 aos dois nós mais próximos, formando uma 1-árvore. Uma implementação do algoritmo de Kruskal, que encontra a MST de um grafo de forma eficiente, está disponível em <https://github.com/cvneves/kit-opt/tree/master/Relaxa%C3%A7%C3%A3o-Lagrangiana/min-spanning-tree>. Lembre-se que é necessário desconsiderar o nó 0 antes de obter a MST, então deve-se modificar levemente o algoritmo ou a matriz de distâncias. Em posse do método descrito acima, é possível resolver o dual lagrangiano associado ao TSP usando o método do subgradiente conforme o Algoritmo 4, fazendo-se $\varepsilon_{\min} = 10^{-5}$, e $k_{\max} = 30$ — o leitor

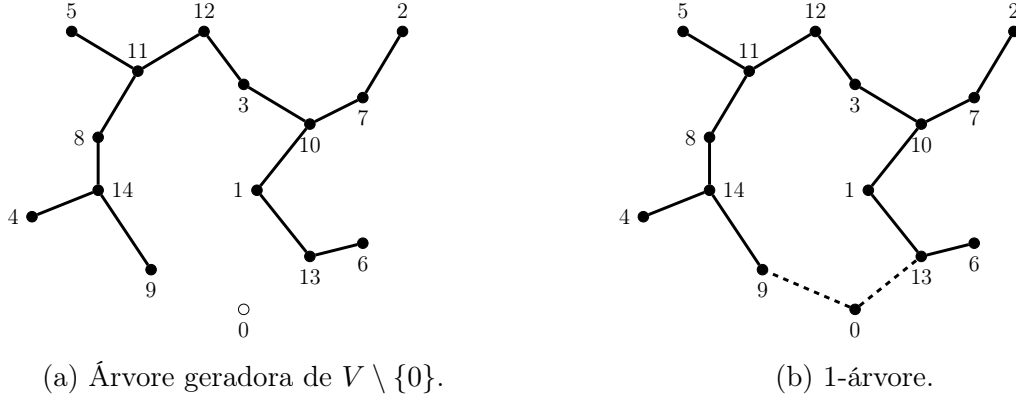


Figura 5.7: Obtendo a 1-árvore a partir da MST.

deve utilizar tal parametrização na implementação.

5.6.2 Exemplo numérico

Para ilustrar o procedimento descrito acima, considere um problema com 5 vértices, e cuja matriz de distâncias é

$$\begin{pmatrix} & 30 & 26 & 50 & 40 \\ & & 24 & 40 & 50 \\ & & & 24 & 26 \\ & & & & 30 \\ & & & & & 0 \end{pmatrix}.$$

Lembre-se que, conforme definido na formulação de (TSP_λ) , há um penalizador para cada vértice, e o penalizador do vértice 0 vale sempre 0. Inicializando o método com o vetor de penalizadores $\lambda = (\lambda_1, \lambda_2, \lambda_3, \lambda_4, \lambda_5) = (0, 0, 0, 0, 0)$ — aqui, ignora-se o penalizador *dummy* λ_0 —, a matriz de custos permanece intacta, e a 1-árvore encontrada com o método descrito acima é ilustrada na Figura 5.8a. Atualizando-se o vetor de penalizadores como na linha 22 do Algoritmo 4, obtém-se uma nova matriz de distâncias, e consequentemente uma 1-árvore diferente, ambos ilustrados na Figura 5.8b. Atualizando-se o vetor de penalizadores novamente, obtém-se a matriz de distâncias e a 1-árvore associada ilustradas na Figura 5.8c. Note que o número de nós de grau 2 é bem maior na última 1-árvore. Recomenda-se que o leitor considere $UB = 148$, e que, após isso, verifique que os valores dos penalizadores mostrados estão corretos calculando-os manualmente (note que, para uma linha $\mathbf{a}_i$ da matriz $\mathbf{A}$, tem-se que a expressão $b - \mathbf{a}_i \mathbf{x}^*$ equivale a $2 - \text{grau}(i)$).

É necessário atentar-se ao fato de que a melhor 1-árvore encontrada pelo Algoritmo 4 pode não ser a última encontrada. Isso pode ser visualizado na Figura 5.9,

Algorithm 4: TSP SUBGRADIENT METHOD.

Data: heuristic upper bound UB, tsp distance matrix C

Result: Optimal 1-Tree s^*

```

1 begin
2    $\lambda \leftarrow 0$ 
3    $\varepsilon \leftarrow 1$ 
4    $\varepsilon_{\min} \leftarrow 10^{-5}$ 
5    $k \leftarrow 0$ 
6    $k_{\max} \leftarrow 30$ 
7   while  $\varepsilon > \varepsilon_{\min}$  and a degree is not 2 do
8      $x \leftarrow \text{SolveMST}(\lambda)$ 
9      $s \leftarrow \text{Make1-Tree}(x)$ 
10     $w \leftarrow \text{1-TreeObjectiveValue}(s)$ 
11    if  $w > w^*$  then
12       $w^* \leftarrow w$ 
13       $\lambda^* \leftarrow \lambda$ 
14       $s^* \leftarrow s$ 
15       $k \leftarrow 0$ 
16    else
17       $k \leftarrow k + 1$ 
18      if  $k \geq k_{\max}$  then
19         $k \leftarrow 0$ 
20         $\varepsilon \leftarrow \varepsilon/2$ 
21       $\mu \leftarrow \varepsilon((\text{UB} - w) / \sum_{i=1}^n (2 - \text{degree}(i))^2)$ 
22       $\lambda \leftarrow \lambda + \mu(2 - \text{degree}(i))$ 
23  return  $s^*$ 

```

que mostra o valor objetivo da 1-árvore encontrada a cada iteração do Algoritmo 4. Note que embora esteja convergindo, o valor objetivo oscila bastante. Por isso, é necessário manter armazenada a melhor 1-árvore obtida, bem como o vetor de penalizadores associados a essa 1-árvore.

5.7 Unindo a Relaxação Lagrangiana ao BnB

Embora a resolução do dual lagrangiano não garanta uma solução ótima para o TSP, pode-se resolver o problema de maneira exata unindo o método ao Branch-and-Bound. Para isso, basta utilizar o método descrito no capítulo anterior, subs-

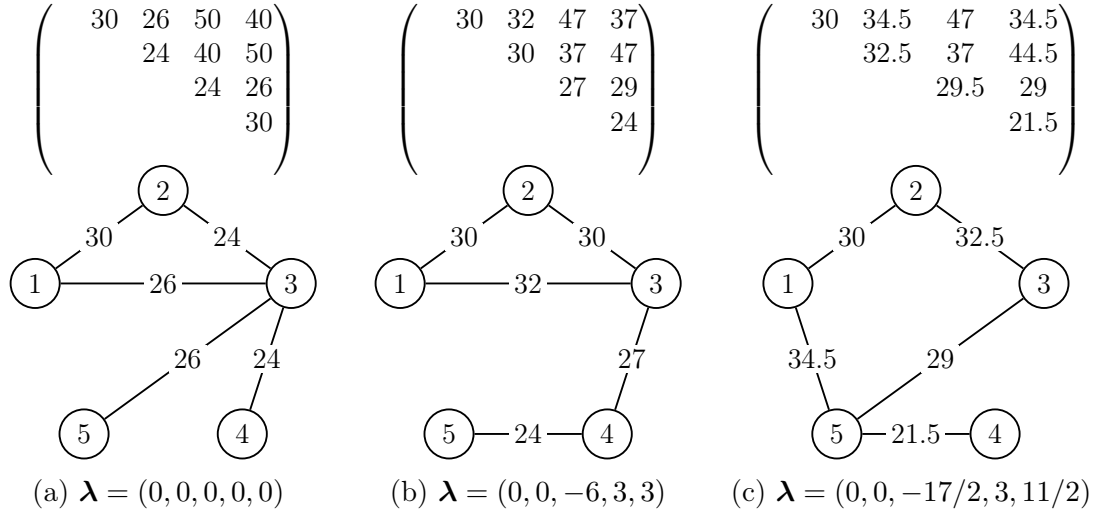


Figura 5.8: Soluções do (TSP_λ) .

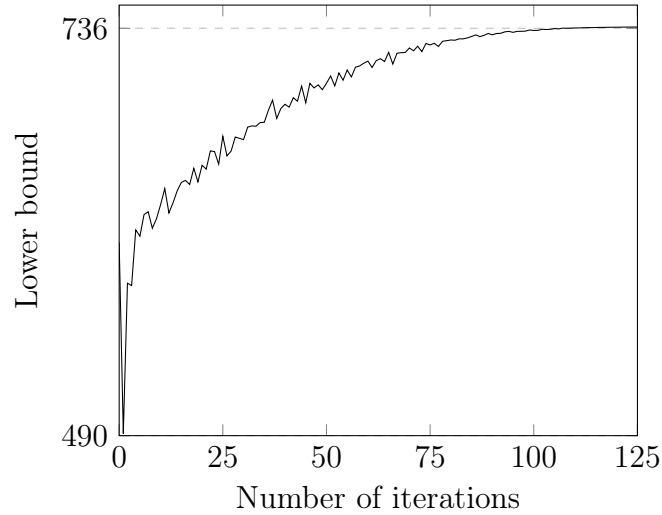


Figura 5.9: Valor objetivo do dual lagrangiano em função do número de iterações.

tituindo o AP_{TSP} — resolvido a cada nó da árvore do BnB — pelo dual lagrangiano do TSP. Desta vez, no entanto, as soluções obtidas em cada nó da árvore do BnB não contém *subtours*, portanto, a regra de branching também deve ser modificada como segue. Após a resolução do dual lagrangiano, caso a melhor 1-árvore obtida não seja um *tour*, deve-se encontrar o vértice com o maior grau e proibir as arestas associadas a esse vértice nos nós filhos. Isso é ilustrado na Figura 5.10. Observe que na melhor 1-árvore obtida no primeiro nó, o vértice 5 possui o maior grau, estando conectado aos vértices 1, 3 e 4. Portanto, o nó deve gerar três filhos,

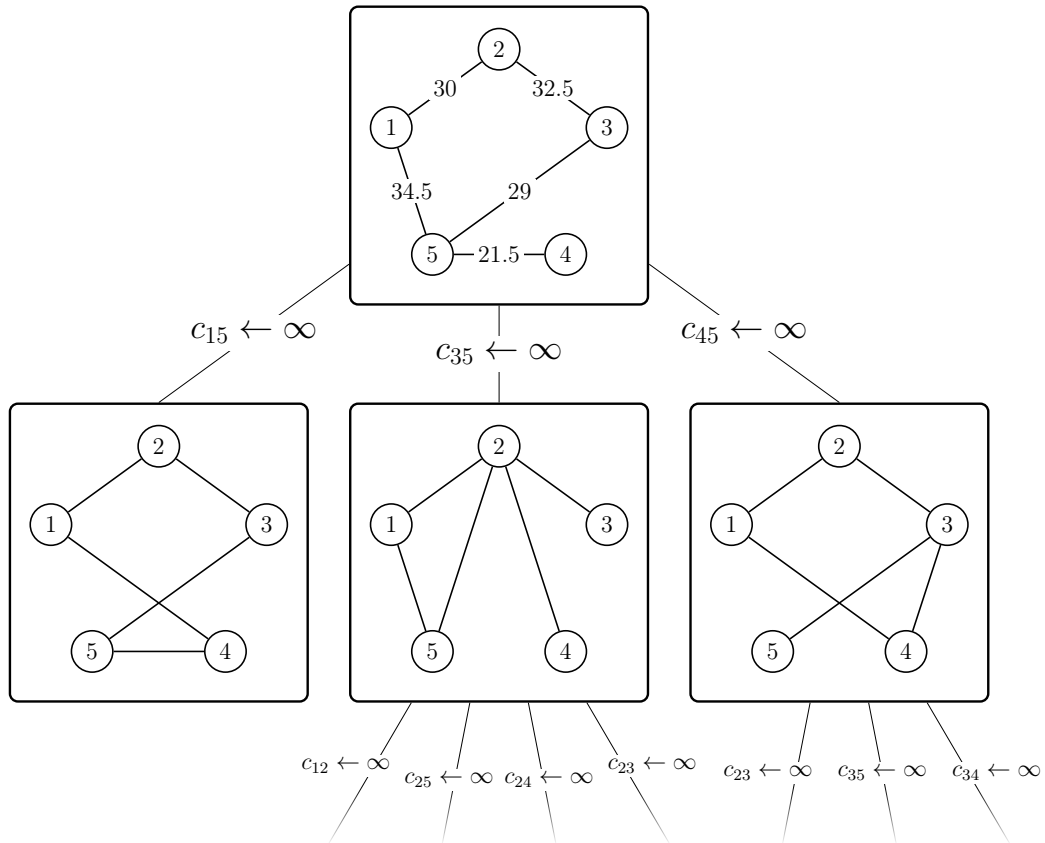


Figura 5.10: Representação em árvore do BnB com Relaxação Lagrangiana.

cada um associado a uma das arestas proibidas. De forma semelhante ao capítulo anterior, o nó filho deve herdar as arestas proibidas do pai. Além disso, os filhos também devem herdar os penalizadores associados à melhor 1-árvore do pai. Tais penalizadores devem ser utilizados como os penalizadores iniciais para resolver o dual lagrangiano. O leitor deve testar o algoritmo com as diferentes estratégias de busca descritas no capítulo anterior.

6

Planos de Corte

Neste capítulo, dois dos algoritmos mais importantes para resolver problemas de programação linear inteira serão abordados: (i) o *branch-and-bound* e (ii) o *cutting planes method* (método de planos de corte). O *branch-and-bound* é empregado na resolução de ILPs de maneira similar à abordada nos capítulos anteriores, i.e., particionando o conjunto de soluções em espaços cada vez menores — representados por meio de uma árvore — em uma busca exaustiva por soluções inteiras. O método dos planos de corte, por sua vez, atua resolvendo a relaxação linear de um ILP de maneira iterativa. Após resolver a relaxação linear do ILP pela primeira vez, um algoritmo é utilizado para detectar desigualdades válidas para o problema original que foram violadas. Se alguma delas for encontrada, ela é adicionada à relaxação linear, e o problema linear resultante é resolvido novamente. O processo se repete até que um certo número de iterações seja alcançado, ou até que uma solução inteira — que, neste caso, é garantidamente ótima — seja encontrada.

Embora os algoritmos *branch-and-bound* e *cutting planes* sejam independentes no sentido de que qualquer um deles pode ser usado de forma isolada para resolver um ILP, a junção de ambos resulta em um algoritmo ainda mais poderoso: o *branch-and-cut*. Ao fim deste capítulo, o leitor deve adquirir uma noção aprofundada do *branch-and-cut*, e aplicá-lo na resolução do TSP.

6.1 Resolvendo ILPs com o BnB

Considere o seguinte Mixed Integer Linear Program (MILP)¹

$$\min_{(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^n \times \mathbb{Z}^m} \{ \mathbf{c}\mathbf{x} + \mathbf{d}\mathbf{y} : \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{y} \leq \mathbf{b} \}, \quad (\text{MILP})$$

¹O termo *Mixed* é empregado porque nem todas as variáveis envolvidas são inteiras.

que possui como relaxação linear o Problema Linear (*Linear Program*, LP)

$$\min_{(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^{(n+m)}} \{ \mathbf{c}\mathbf{x} + \mathbf{d}\mathbf{y} : \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{y} \leq \mathbf{b} \}. \quad (\text{LR})$$

Observe que a única diferença entre (MILP) e (LR) é que em (LR), o conjunto de variáveis $\mathbf{y}$ não está mais restrito aos números inteiros, podendo agora assumir qualquer valor em $\mathbb{R}^m$. Em outras palavras, (LR) é um LP, e pode ser resolvido de forma eficiente, por exemplo, através do Método Simplex. Se, no entanto, a solução ótima $(\mathbf{x}^*, \mathbf{y}^*)$ de (LR) não pertencer a $\mathbb{R}^n \times \mathbb{Z}^m$, diz-se que ela não é viável para (MILP), e seu valor objetivo $\mathbf{c}\mathbf{x}^* + \mathbf{d}\mathbf{y}^*$ é apenas um limitante inferior para o valor ótimo de (MILP).

Suponha, agora, que deseja-se aplicar o *branch-and-bound* para encontrar a solução ótima de (MILP). Nesse contexto, assuma que uma solução ótima $(\mathbf{x}^*, \mathbf{y}^*)$ para o problema (LR) foi obtida no nó raiz da árvore associada ao *branch-and-bound*. Como discutido anteriormente — e de forma semelhante à vista em outras aplicações do *branch-and-bound* —, se $(\mathbf{x}^*, \mathbf{y}^*) \in \mathbb{R}^n \times \mathbb{Z}^m$, então a solução obtida no nó raiz é a solução ótima de (MILP), e pode-se encerrar o algoritmo. Suponha, no entanto, que há algum $i \in \{1, \dots, m\}$ tal que $y_i^* \notin \mathbb{Z}$, i.e., a i -ésima variável inteira y_i assumiu um valor fracionário, resultando em uma solução inviável para (MILP). Para lidar com isso, é possível derivar uma regra de *branching* válida a partir de uma simples observação: qualquer que seja a solução ótima de (MILP), é necessariamente verdade que $y_i \geq \lceil y_i^* \rceil$ ou $y_i \leq \lfloor y_i^* \rfloor$ ². A partir disso, os dois seguintes LPs são obtidos.

$$\min_{(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^{(n+m)}} \{ \mathbf{c}\mathbf{x} + \mathbf{d}\mathbf{y} : \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{y} \leq \mathbf{b}, y_i \geq \lceil y_i^* \rceil \}, \quad (\text{LR}_1)$$

$$\min_{(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^{(n+m)}} \{ \mathbf{c}\mathbf{x} + \mathbf{d}\mathbf{y} : \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{y} \leq \mathbf{b}, y_i \leq \lfloor y_i^* \rfloor \}. \quad (\text{LR}_2)$$

Note que a solução ótima $(\mathbf{x}^*, \mathbf{y}^*)$ obtida no nó raiz não é viável nem para (LR₁), e nem para (LR₂). Isso significa que o espaço de soluções de ambos os problemas lineares acima é estritamente menor que o espaço de soluções da relaxação linear original. Além disso, os espaços de soluções de ambos os problemas não possuem soluções em comum. Dessa forma, não é difícil concluir que a repetida aplicação do procedimento descrito acima resulta em um algoritmo correto, que sempre acabará encontrando a solução ótima de (MILP).

²De fato, para qualquer número inteiro y , e para qualquer número real z , exatamente uma das duas afirmações é verdade: $y \geq \lceil z \rceil$ ou $y \leq \lfloor z \rfloor$.

6.2 Resolvendo ILPs com *cutting planes*

Ainda tendo em mente o problema (MILP) descrito acima, considere, agora, o problema

$$\min_{(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^{(n+m)}} \left\{ \mathbf{c}\mathbf{x} + \mathbf{d}\mathbf{y} : \begin{array}{l} \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{y} \leq \mathbf{b} \\ \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{y} \leq \mathbf{f} \end{array} \right\}, \quad (\text{MILP}_{\text{conv}})$$

em que $\mathbf{C}$, $\mathbf{D}$ e $\mathbf{f}$ são tais que

$$\left\{ (\mathbf{x}, \mathbf{y}) \in \mathbb{R}^{(n+m)} : \begin{array}{l} \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{y} \leq \mathbf{b} \\ \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{y} \leq \mathbf{f} \end{array} \right\} = \text{conv} \{ (\mathbf{x}, \mathbf{y}) \in \mathbb{R}^n \times \mathbb{Z}^m : \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{y} \leq \mathbf{b} \}.$$

Ou seja, o problema (MILP_{conv}) foi obtido a partir da envoltória convexa das soluções inteiras de (MILP).

Observe que (MILP_{conv}) é um problema linear, pois não possui variáveis cujos domínios são restritos aos números inteiros, e todas as suas restrições são lineares. Sabe-se, portanto, que como consequência dos principais teoremas e resultados da programação linear, as soluções ótimas do problema (MILP_{conv}) consistem em pontos extremos de seu poliedro associado. Além disso, como definido acima, os pontos de extremos de (MILP_{conv}) são de fato soluções inteiras do problema original (MILP). Segue, finalmente, que toda solução ótima de (MILP_{conv}) também é ótima para (MILP), ou seja, resolver (MILP_{conv}) é equivalente a resolver (MILP).

Dado um problema (MILP) qualquer, e considerando o que foi concluído acima, o leitor pode se perguntar por que não apenas resolver (MILP_{conv}), que consiste em um LP e portanto, pode ser resolvido de maneira eficiente. Note, contudo, que nada foi comentado acerca da dificuldade de obter $\mathbf{C}$, $\mathbf{D}$ e $\mathbf{f}$. A verdade é que, alguns problemas inteiros exigiriam matrizes $\mathbf{C}$, $\mathbf{D}$ e um vetor coluna $\mathbf{f}$ com um número de linhas exponencial em $(m + n)$, caso fossem convertidos em sua forma (MILP_{conv}). É provado matematicamente que uma formulação matemática para o TSP envolvendo exclusivamente variáveis binárias x_{ij} para representar a inclusão (ou exclusão) da aresta $\{i, j\} \in E$ no ciclo hamiltoniano, por exemplo, exigiria um número de restrições exponencial na quantidade de vértices $|V|$ para ser representada como um LP. Dessa maneira, uma simples iteração do Método Simplex exigiria um número exponencial de operações, inviabilizando a resolução do problema, mesmo que ele seja um LP.

Felizmente, sob certas condições, os conceitos abordados acima ainda podem ser aproveitados para elaborar algoritmos exatos para resolver MILPs. Para isso, é necessário compreender o conceito de *oráculo de separação*, discutido a seguir.

6.2.1 O Problema de Separação

Suponha que estejamos em posse de um vetor $\mathbf{x}^*$ qualquer, e que desejamos saber se $\mathbf{x}^* \notin P$ — i.e., $\mathbf{x}^*$ está fora de P —, sendo $P := \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} \leq \mathbf{b}\}$ um poliedro qualquer. Uma maneira de descobrir isso é encontrando alguma desigualdade que é válida para todo $\mathbf{x} \in P$, mas que é violada por $\mathbf{x}^*$. Mais precisamente, seria necessário encontrar um vetor linha $\boldsymbol{\alpha} \in \mathbb{R}^n$ e $\beta \in \mathbb{R}$ tais que $P \subseteq \{\mathbf{x} \in \mathbb{R}^n : \boldsymbol{\alpha}\mathbf{x} \leq \beta\}$ e $\boldsymbol{\alpha}\mathbf{x}^* > \beta$. O problema que consiste em determinar a existência de $\boldsymbol{\alpha}$ e β — que não existem caso $\mathbf{x}^* \in P$ — e encontrá-los é denominado *problema de separação*. Um algoritmo que toma como dados de entrada P e $\mathbf{x}^*$ e é sempre capaz de retornar $\boldsymbol{\alpha}$ e β , caso existam — e que não retorna nada caso $\boldsymbol{\alpha}$ e β não existam — é denominado *oráculo de separação* ou *algoritmo de separação*. O resultado da aplicação de um algoritmo de separação é exemplificado na Figura 6.1.

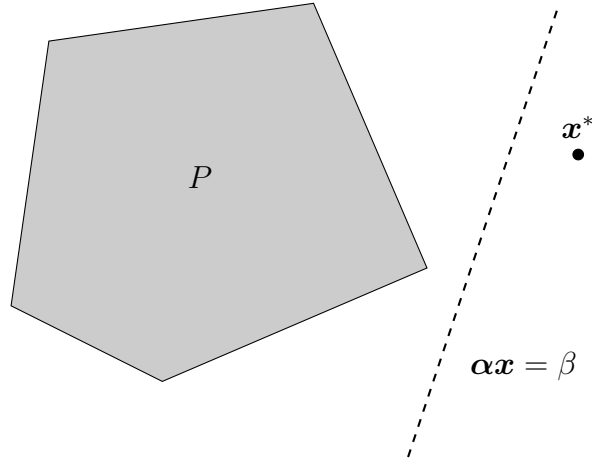


Figura 6.1: A desigualdade $\boldsymbol{\alpha}\mathbf{x} \leq \beta$ é válida para todo ponto $\mathbf{x} \in P$, mas não é válida para o ponto $\mathbf{x}^*$. Isso é uma prova de que $\mathbf{x}^* \notin P$.

Suponha, por exemplo, que

$$P = \left\{ \mathbf{x} \in \mathbb{R}^n : \sum_{i \in S} x_i - \sum_{i \notin S} x_i \leq 1, \forall S \subseteq \{1, \dots, n\} \right\}$$

$$= \left\{ \mathbf{x} \in \mathbb{R}^n : \begin{array}{l} x_1 + x_2 + \dots + x_{n-1} + x_n \leq 1 \\ x_1 + x_2 + \dots + x_{n-1} - x_n \leq 1 \\ x_1 + x_2 + \dots - x_{n-1} + x_n \leq 1 \\ x_1 + x_2 + \dots - x_{n-1} - x_n \leq 1 \\ \vdots \\ -x_1 - x_2 - \dots - x_{n-1} + x_n \leq 1 \\ -x_1 - x_2 - \dots - x_{n-1} - x_n \leq 1 \end{array} \right\}.$$

A princípio, elaborar um algoritmo de separação eficiente para P pode parecer uma tarefa simples: bastaria iterar por todas as restrições do poliedro P e verificar se alguma delas foi violada por $\mathbf{x}^*$. Note, contudo, que P possui exatamente 2^n restrições. Por consequência, o algoritmo descrito exigiria cerca de $2^n \times n$ operações para determinar a existência de uma desigualdade válida violada por um ponto $\mathbf{x}^*$ qualquer. Felizmente, é possível obter um algoritmo de separação significativamente melhor aproveitando-se da estrutura especial de P :

1. Verificar se $\sum_{i=1}^n |x_i^*| > 1$
2. Se sim, retornar o vetor $\boldsymbol{\alpha} := (\text{sgn}(x_1^*), \dots, \text{sgn}(x_n^*))$ — em que $\text{sgn}(x) = 1$ se $x \geq 0$, e $\text{sgn}(x) = -1$ caso contrário — e o escalar $\beta := 1$
3. Caso contrário, o vetor $\mathbf{x}^*$ não viola nenhuma desigualdade de P , e $\mathbf{x}^* \in P$

Além de ser correto, o algoritmo descrito exige um número de operações linear em n . A ideia do algoritmo é ilustrada na Figura 6.2.

6.2.2 Descrição do Algoritmo

Embora seja simples, o exemplo anterior revela uma ideia interessante e fundamental: ainda que um poliedro P exija um grande número de restrições para ser descrito, é possível que exista um oráculo de separação que seja capaz determinar de forma eficiente se um ponto $\mathbf{x}^*$ pertence a P . A consequência disso é que, caso disponhamos de um oráculo de separação eficiente para o poliedro associado a $(\text{MILP}_{\text{conv}})$, seria possível resolver $(\text{MILP}_{\text{conv}})$ de maneira iterativa, começando a partir da resolução da relaxação linear (LR) e verificando se sua solução ótima $(\mathbf{x}^*, \mathbf{y}^*)$ viola alguma restrição $\boldsymbol{\alpha}\mathbf{x} + \boldsymbol{\pi}\mathbf{y} \leq \beta$ de $(\text{MILP}_{\text{conv}})$. Se esse for o caso,

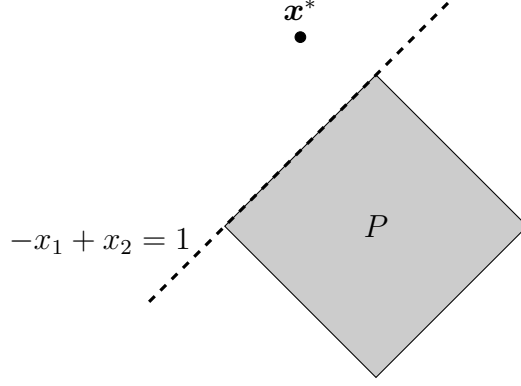


Figura 6.2: O ponto $\mathbf{x}^* = (-1/2, 5/4)$ não pertence a P por violar a desigualdade $\text{sgn}(-1/2)x_1 + \text{sgn}(5/4)x_2 = -x_1 + x_2 \leq 1$.

um novo problema

$$\min_{(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^{(n+m)}} \left\{ \mathbf{c}\mathbf{x} + \mathbf{d}\mathbf{y} : \begin{array}{l} \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{y} \leq \mathbf{b} \\ \boldsymbol{\alpha}\mathbf{x} + \boldsymbol{\pi}\mathbf{y} \leq \beta \end{array} \right\}$$

é gerado adicionando-se a restrição violada ao problema inicial, e o oráculo de separação deve ser novamente aplicado na solução do novo problema até que uma solução viável — que é garantidamente ótima, diferente do caso do *branch-and-bound* — seja encontrada.

Algorithm 5: THE CUTTING PLANES METHOD

Data: A polyhedron P_{LR} such that $P_{\text{MILPconv}} \subseteq P_{\text{LR}}$ and a cost vector $\mathbf{c}$

Result: An optimal solution $\mathbf{x}^* \in P_{\text{MILPconv}}$

1 **Procedure** CuttingPlanes() **is**

2 **while** *true* **do**

3 $\mathbf{x}^* \leftarrow \text{Solve}(\min\{\mathbf{c}\mathbf{x} \mid \mathbf{x} \in P_{\text{LR}}\})$

4 $(\boldsymbol{\alpha}, \beta) \leftarrow \text{Separate}(\mathbf{x}^*, P_{\text{MILPconv}})$ // Find violated ineq.

5 **if** $(\boldsymbol{\alpha}, \beta) = (\mathbf{0}, -1)$ **then** // Is $\mathbf{x}^*$ feasible?

6 **return** $\mathbf{x}^*$

7 **else**

8 $P_{\text{LR}} \leftarrow P_{\text{LR}} \cap \{\mathbf{x} \in \mathbb{R}^d \mid \boldsymbol{\alpha}\mathbf{x} \leq \beta\}$ // Add ineq. to P_{LR}

A ideia é descrita de maneira mais precisa no Algoritmo 5. O algoritmo inicia com o poliedro P_{LR} associado à relaxação linear (LR) de (MILP). Em seguida, uma solução ótima $\mathbf{x}^*$ é encontrada para (LR) (linha 3). Após isso, um oráculo de separação é utilizado para encontrar uma desigualdade válida para P_{MILPconv}

Planos de Corte

que foi violada por $\mathbf{x}^*$ (linha 4), sendo $P_{\text{MILP}_{\text{conv}}}$ o poliedro associado ao problema ($\text{MILP}_{\text{conv}}$). Se nenhuma desigualdade for encontrada, a solução $\mathbf{x}^*$ além de ser viável para (MILP), é ótima, e o algoritmo é então encerrado retornando-a (linhas 5–6). Se, por outro lado, uma desigualdade for encontrada, ela é adicionada ao poliedro P_{LR} (linha 8). O procedimento se repete até que nenhuma desigualdade violada seja encontrada. Um exemplo de execução do algoritmo é ilustrado na Figura 6.3.

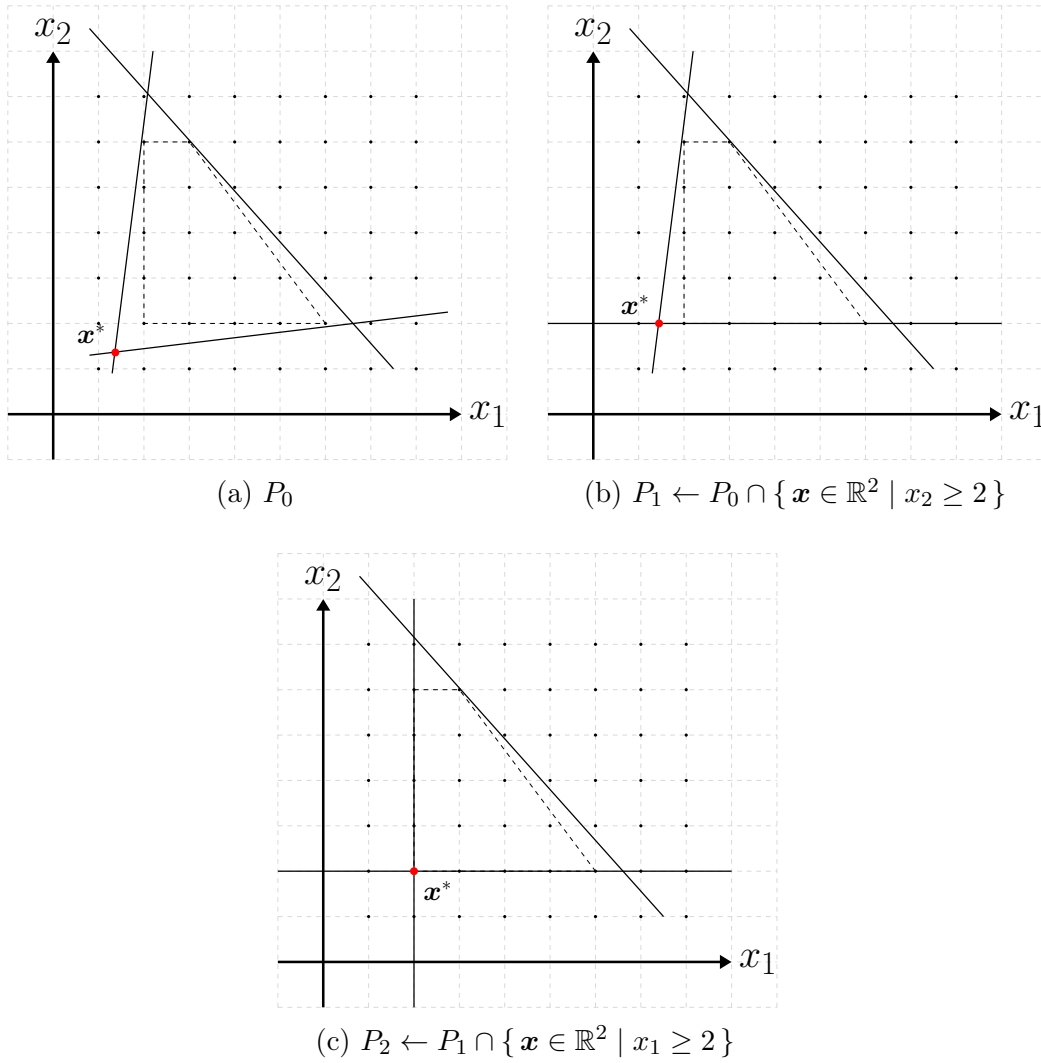


Figura 6.3: A região tracejada é a envoltória convexa das soluções inteiras do problema original. Note que a solução ótima foi encontrada em apenas duas iterações, e antes que o poliedro se tornasse igual ao do problema original.

6.3 O Algoritmo *Branch-and-Cut*

Suponha que dispõe-se de uma versão modificada do Algoritmo 5. Nela, o laço da linha 2 é limitado a um certo número fixo de iterações. Além disso, o algoritmo de separação empregado na linha 4 é heurístico, i.e., ele não garante que uma desigualdade violada será encontrada caso ela exista. O algoritmo descrito é uma heurística, e não é capaz de garantir que a solução ótima de $(\text{MILP}_{\text{conv}})$ seja encontrada. Contudo, essa heurística possui o potencial de encontrar múltiplas desigualdades válidas violadas, adicionando-as à formulação atual e aproximando-a cada vez mais de $(\text{MILP}_{\text{conv}})$. Note, ainda, que essa heurística pode ser aplicada em um nó arbitrário da árvore associada ao *branch-and-bound* descrito na Seção 6.1, resultando em um algoritmo mais poderoso denominado *branch-and-cut*. Nesse contexto, o método de planos de corte tende a assumir um papel heurístico, fornecendo algumas desigualdades válidas violadas pela solução corrente quando possível, e consequentemente acelerando a convergência do método. A tarefa de garantir a otimalidade por meio de uma busca exaustiva, por sua vez, é concedida ao *branch-and-bound*.

O *Concorde TSP Solver* [\[link\]](#), por exemplo, é o melhor resolvidor exato de TSPs do mundo, e emprega como uma de suas rotinas o algoritmo *branch-and-cut*. Nas próximas seções, uma aplicação simples do método *branch-and-cut* ao TSP é descrita. Desta vez, o leitor deverá fazer uso de um resolvidor exato de MILPs como o CPLEX ou o Gurobi, que já dispõem de implementações prontas do algoritmo *branch-and-bound*, bem como de alguns algoritmos de separação genéricos para certas famílias de desigualdades válidas. Por consequência, o leitor deve dedicar a maior parte do tempo à implementação de dois algoritmos de separação específicos para o TSP, que serão descritos a seguir.

6.4 O problema da Separação para o TSP

Seja $G = (V, E)$ o grafo completo associado a uma instância qualquer do TSP, em que cada aresta $e \in E$ possui um peso $c_e \in \mathbb{R}^+$. Considere, além disso, que dado um subconjunto de vértices S , a função definida como $\delta(S) := \{\{i, j\} \in E : i \in S, j \notin S\}$ informa quais arestas em E possuem um nó em S e outro nó fora de S . Define-se, ainda, que x_e é uma variável binária que representa a escolha de incluir a aresta $e \in E$ no ciclo hamiltoniano. Dessa forma, resolver o

TSP equivale a resolver o ILP

$$\min_{\mathbf{x} \in \mathbb{R}^{|E|}} \left\{ \sum_{e \in E} c_e x_e : \begin{array}{ll} \sum_{e \in \delta(\{i\})} x_e = 2 & i \in V \\ \sum_{e \in \delta(S)} x_e \geq 2 & S \subset V, S \neq \emptyset \\ x_e \in \{0, 1\} & e \in E \end{array} \right\}, \quad (\text{TSP}_{\text{ILP}})$$

em que o primeiro conjunto de restrições garante que todo vértice do grafo resultante possui grau igual a 2, enquanto o segundo conjunto de restrições garante que não há *subtours*. O problema $(\text{TSP}_{\text{ILP}})$ possui como relaxação linear o LP

$$\min_{\mathbf{x} \in \mathbb{R}^{|E|}} \left\{ \sum_{e \in E} c_e x_e : \begin{array}{ll} \sum_{e \in \delta(\{i\})} x_e = 2 & i \in V \\ \sum_{e \in \delta(S)} x_e \geq 2 & S \subset V, S \neq \emptyset \\ 0 \leq x_e \leq 1 & e \in E \end{array} \right\}. \quad (\text{TSP}_{\text{LR}})$$

Observe que até mesmo (TSP_{LR}) possui um número exponencial de restrições, pois há $2^{|V|} - 2$ subconjuntos S de V tais que $S \neq \emptyset$ e $S \neq V$. Portanto, para tamanhos moderados de $|V|$, é impossível resolver sequer a relaxação linear do problema original usando o Método Simplex.

Uma alternativa a resolver (TSP_{LR}) seria resolver o problema mais simples

$$\min_{\mathbf{x} \in [0, 1]^{|E|}} \left\{ \sum_{e \in E} c_e x_e : \sum_{e \in \delta(\{i\})} x_e = 2 \quad i \in V \right\}, \quad (\text{TSP}_{\text{init}})$$

e aplicar o método de planos de corte, empregando um oráculo de separação para encontrar, caso exista, uma restrição de *subtour* que foi violada. A restrição seria então adicionada a $(\text{TSP}_{\text{init}})$ e o problema resultante seria resolvido novamente, repetindo-se o processo até que a solução ótima de (TSP_{LR}) seja encontrada. Resta, portanto, descobrir como separar um ponto $\mathbf{x} \in [0, 1]^{|E|}$ do poliedro

$$P_{\text{sub}} = \left\{ \mathbf{x} \in [0, 1]^{|E|} : \sum_{e \in \delta(S)} x_e \geq 2 \quad S \subset V, S \neq \emptyset \right\}.$$

Uma tática comum para lidar com problemas de separação como esse — i.e., que envolvem um conjunto específico de desigualdades — é imaginá-los como problemas de otimização. Mais especificamente, suponha que $\mathbf{x}^* \in [0, 1]^{|E|}$ é um ponto qualquer, e considere o problema de otimização

$$\min_{S \subset V, S \neq \emptyset} \left\{ \sum_{e \in \delta(S)} x_e^* \right\}, \quad (\text{MCP})$$

obtido diretamente a partir do lado esquerdo das restrições de *subtour*. Seja S a solução ótima de (MCP). Se $\sum_{e \in \delta(S)} x_e^* \geq 2$, então todas as restrições de *subtour* foram respeitadas, já que S é um minimizador. Caso contrário, a restrição $\sum_{e \in \delta(S)} x_e \geq 2$ foi violada por $\mathbf{x}^*$.

Para compreender melhor o problema (MCP) e como resolvê-lo, considere o grafo $H = (V, E)$ no qual cada aresta $e \in E$ possui peso $x_e^* \in [0, 1]$. A solução ótima de (MCP) consiste em um conjunto $S \subset V$ tal que a soma dos pesos das arestas que possuem uma “ponta” em S e outra fora de S seja a menor possível. O problema (MCP) é chamado de Problema do Corte Mínimo (*Minimum Cut Problem*), e recebe esse nome porque o conjunto S pode ser entendido como uma divisão (ou corte) de V nos dois conjuntos S e $V \setminus S$. Uma solução ótima de (MCP) é um *corte mínimo*, ou *min-cut*, enquanto soluções não necessariamente ótimas podem ser chamadas de *cortes*. A função objetivo do problema é chamada de *valor do corte*. Dois exemplos de cortes em grafos são apresentados na Figura 6.4.

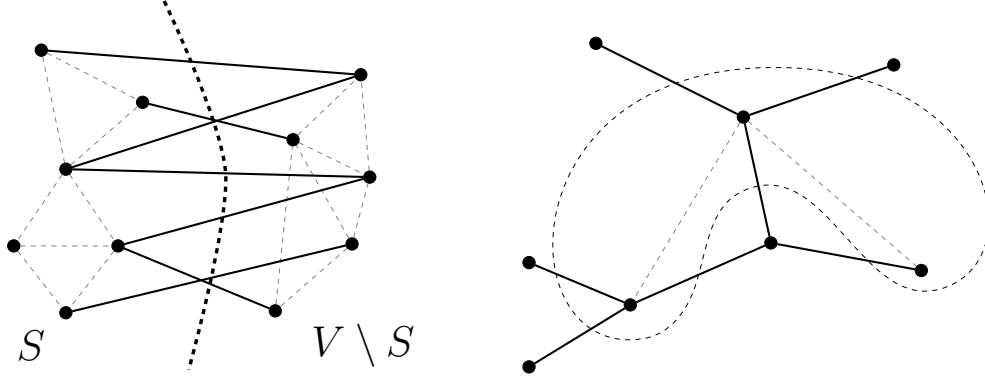


Figura 6.4: Exemplos de cortes em grafos. Na primeira figura, os vértices foram divididos em lados esquerdo e direito. Na segunda figura, três vértices quaisquer compõem o conjunto S . Em ambas as figuras, as arestas escuras são aquelas que possuem uma ponta em S , e outra fora de S .

A Figura 6.5a, por sua vez, mostra uma possível solução de (TSP_{LR}). Observe que a soma dos pesos das arestas conectadas a qualquer um dos vértices da solução é 2. Isso significa que todas as restrições de grau foram respeitadas. Por outro lado, a Figura 6.5b ilustra um corte do grafo que possui valor $0.75 + 0.75 = 1.5$. Lembre-se, no entanto, que as restrições de *subtour* impõem que todo corte no grafo cujos pesos são $\mathbf{x}$ deve possuir valor maior ou igual a 2. Isso significa que ao menos uma restrição de *subtour* (especificamente aquela que envolve o conjunto S

associado ao corte ilustrado) foi violada. Tal restrição poderia ser então adicionada à formulação, resultando em uma formulação mais forte e que poderia ser resolvida novamente.

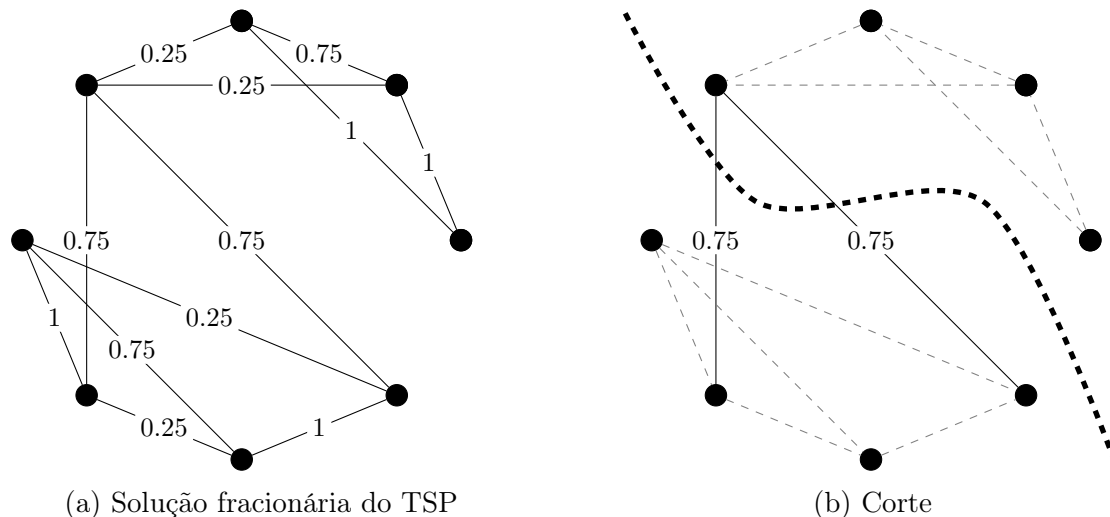


Figura 6.5: Solução fracionária do TSP

Nas próximas seções, dois algoritmos (um heurístico e outro exato) para encontrar um min-cut de um grafo são descritos. O leitor deve implementar ambos.

6.5 Heurística *Max-Back*

A heurística Max-Back [REF](#) é simples, e consiste em um método guloso. Em suma, dada uma solução possivelmente fracionária x^* de (TSP_{init}) e um conjunto inicial de vértices $S_0 \subset V$, obtém-se um novo conjunto $S_1 := S_0 \cup i$, em que $i \in V \setminus S_0$ é o vértice mais “conectado” a S_0 , e o processo se repete até a iteração k , quando $S_k = V$. O algoritmo retorna o conjunto $S \in \{S_0, \dots, S_{k-1}\}$ tal que o corte induzido por S é o menor possível. A Figura [6.7](#) ilustra algumas iterações da heurística em uma solução inteira de (TSP_{init}) .

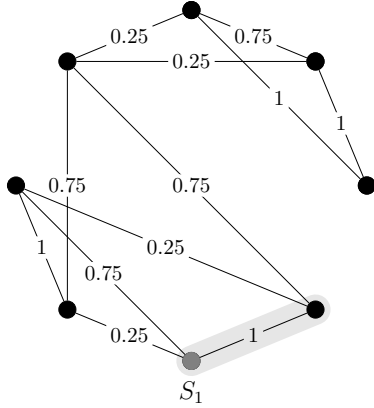
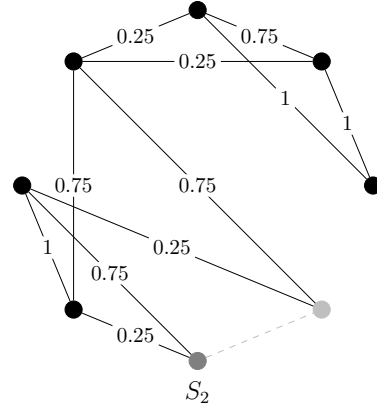
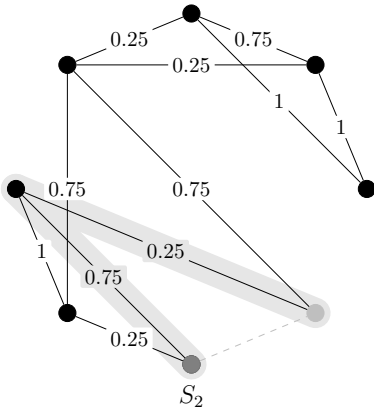
O procedimento é descrito em mais detalhes no Algoritmo [6](#). [Descrever linha a linha...](#)

Algorithm 6: MAX-BACK HEURISTIC**Data:** A subset of vertices $S_0 \subset V$ and a solution $\mathbf{x}^*$ of (TSP_{init})**Result:** A (cut-inducing) subset $S \subset V$

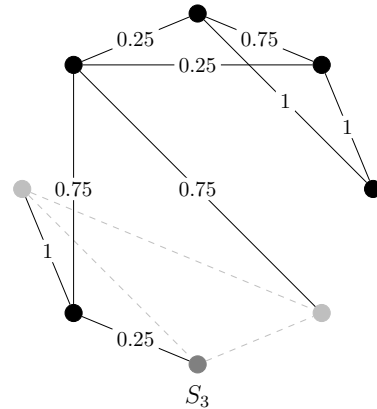
```

1 Procedure Max-Back() is
2   maxback_val[j]  $\leftarrow \sum_{i \in S_0} x_{ij}^*$  for all  $j \in V$ 
3   cut_val  $\leftarrow \sum_{i \in S_0} \sum_{j \in V \setminus S_0} x_{ij}^*$ 
4   mincut_val  $\leftarrow$  cut_val
5    $S \leftarrow S_0$ 
6   for  $k = 1, \dots, |V| - |S_0|$  do
7      $i \leftarrow j \in V \setminus S_{k-1}$  such that maxback_val[j] is maximal
8      $S_k \leftarrow S_{k-1} \cup \{i\}$ 
9     cut_val  $\leftarrow$  cut_val +  $\delta(i) - 2 \times \text{maxback\_val}[i]$ 
10    for  $j \in V \setminus S_k$  do
11      maxback_val[j]  $\leftarrow$  maxback_val[j] +  $x_{ij}^*$ 
12    if cut_val < mincut_val then
13      mincut_val  $\leftarrow$  cut_val
14       $S \leftarrow S_k$ 
15  return mincut_val,  $S, V \setminus S_{|V|-|S_0|-1}$ 

```

(a) $\text{max_back} = 1$ (b) $\text{cut_val} = 2$ (c) $\text{max_back} = 1$

80

(d) $\text{cut_val} = 2$

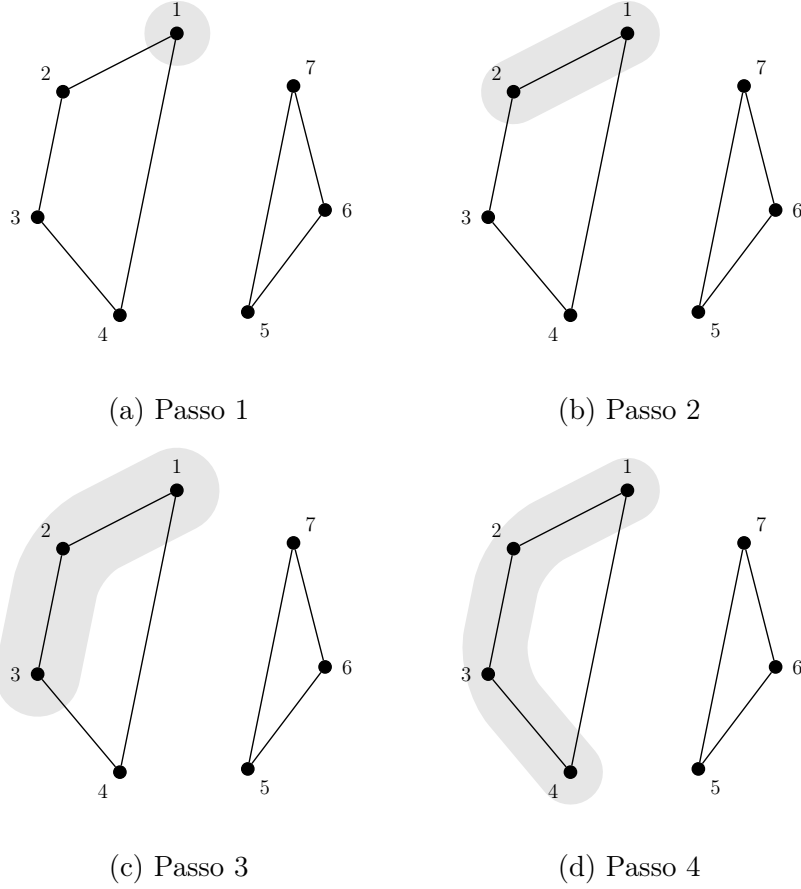


Figura 6.7: As quatro primeiras iterações da heurística *Max-back* em uma solução inteira de $(\text{TSP}_{\text{init}})$, em que os vértices destacados consistem no conjunto S_k corrente (i.e., $S_0 = \{1\}$).

Teorema 7. Se $\mathbf{x}^* \in \{0, 1\}^{|E|}$ e $\sum_{\delta(\{i\})} x_e^* = 2$ para todo $i \in V$, então o corte fornecido pelo Algoritmo 6 é o *min-cut*.

Demonstração. Suponha que $H = (V, E^*)$ é o grafo cuja matriz de adjacência é $\mathbf{x}^*$, i.e., uma aresta e pertence a E^* se e somente se $x_e^* = 1$. As condições estabelecidas no teorema impõem que H é um grafo em que todos os nós possuem grau 2. Se H não contiver *subtours*, então qualquer corte em H — incluindo o corte obtido ao fim do Algoritmo 6 — possui valor 2.

Suponha, por outro lado, que H possui um subtour cujo conjunto de vértices é $V_0 \subset V$, e que — sem perda de generalidade — o Algoritmo 6 inicia a partir do conjunto $S_0 \subset V_0$. Seja $S_0, S_1, \dots, S_{|V|-|S_0|}$ a sequência de cortes gerados pelo algoritmo. Agora, resta apenas mostrar que $S_k = V_0$ para algum $k \geq 0$, pois V_0

induz um ciclo em H , o valor do corte associado a S_k é

$$\sum_{e \in \delta(S_k)} x_e = \sum_{e \in \delta(V_0)} x_e = 0,$$

que é o menor valor que algum corte em H poderia ter, já que não há arestas com peso negativo.

Para mostrar que $S_k = V_0$ para algum $k \geq 0$, observe que

$$\sum_{j \in V \setminus V_0} x_{ij}^* = 0$$

para qualquer nó $i \in S_0$, e que

$$\sum_{i \in S_0, j \in V_0 \setminus S_0} x_{ij}^* > 0.$$

Essas duas propriedades garantem que o próximo vértice a ser adicionado a S_0 está em V_0 . Além disso, observe que a escolha de S_0 foi arbitrária — qualquer subconjunto de V_0 poderia ter sido escolhido como o conjunto inicial do algoritmo. Portanto, em alguma iteração $k \geq 0$, o subconjunto de vértices corrente é $S_k = V_0$, concluindo a prova. $\square$

O Teorema 7 garante que a heurística max-back sempre encontra subtours em soluções inteiras, caso existam. Portanto, se a formulação (TSP_{init}) for utilizada no nó raiz da árvore do *branch-and-bound* e a heurística max-back for executada em todo nó da árvore no qual uma solução inteira $\mathbf{x}^*$ foi encontrada pelo resolvidor, é garantido que a solução ótima de (TSP_{ILP}) será encontrada ao fim do algoritmo. Em outras palavras, a heurística max-back unida ao *branch-and-bound* é suficiente para garantir otimalidade no TSP. Ao leitor, recomenda-se que a implemente primeiro, e que teste o algoritmo resultante em instâncias envolvendo até 300 vértices.

6.5.1 Um Algoritmo Exato para obter o min-cut

Para todo $\{i, j\} \in V_{\text{new}}$,

$$x_{ij} := \begin{cases} x_{sj}^* + x_{tj}^* & \text{se } i = \{s, t\} \\ x_{is}^* + x_{it}^* & \text{se } j = \{s, t\} \\ x_{ij}^* & \text{caso contrário.} \end{cases} \quad (6.1)$$

Algorithm 7: SIMPLE MIN-CUT

Data: A graph $G = (V, E)$ and a solution $\mathbf{x}^*$ of $(\text{TSP}_{\text{init}})$

Result: A minimum cut $S \subset V$

```

1 Procedure Simple-Min-Cut( $G, \mathbf{x}^*$ ) is
2   mincut_val  $\leftarrow +\infty$ 
3   for  $k = 1, \dots, |V| - 1$  do
4     cut_val,  $\{s, t\} \leftarrow \text{Max-Back}(G, \{1\}, \mathbf{x}^*)$ 
5     if cut_val < mincut_val then
6       mincut_val  $\leftarrow$  cut_val
7        $S \leftarrow t$ 
8      $G, \mathbf{x}^* \leftarrow \text{Shrink}(G, \mathbf{x}^*, s, t)$ 
9   return mincut_val,  $S$ 

10 Procedure Shrink( $G = (V, E), \mathbf{x}^*, s, t$ ) is
11    $V_{\text{new}} \leftarrow (V \setminus \{s, t\}) \cup \{\{s, t\}\}$ 
12    $E_{\text{new}} \leftarrow \{\{i, j\} \in V_{\text{new}} : i \neq j\}$ 
13    $\mathbf{x}_{\text{new}}^* \leftarrow \mathbf{x} \in \mathbb{R}^{|E_{\text{new}}|}$  s.t.  $x_{ij} = \begin{cases} x_{ij}^* & \text{if } i \neq \{s, t\} \text{ and } j \neq \{s, t\} \\ x_{sj}^* + x_{tj}^* & \text{otherwise} \end{cases}$ 
14   return  $G_{\text{new}} = (V_{\text{new}}, E_{\text{new}}), \mathbf{x}_{\text{new}}^*$ 

```

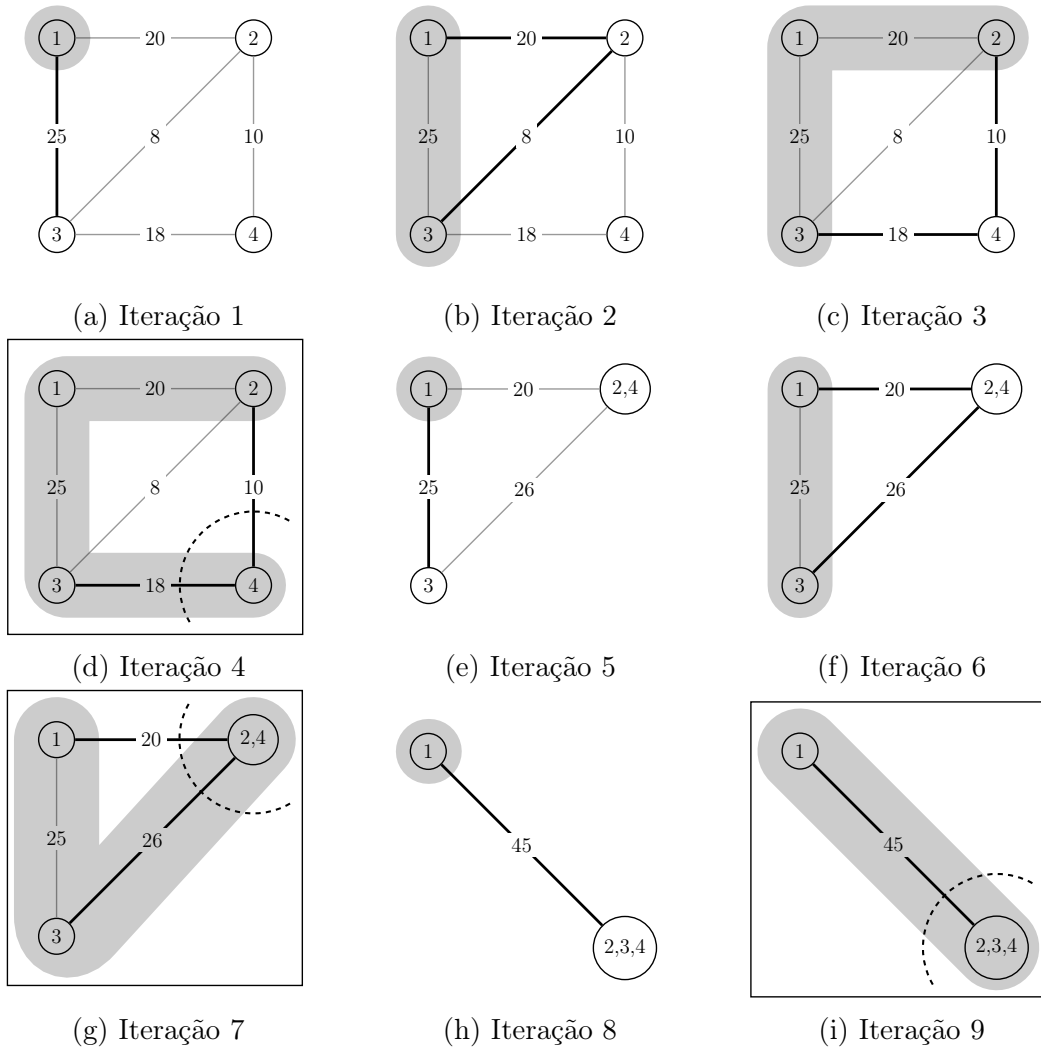


Figura 6.8: Iterações do *Min-cut*

Planos de Corte

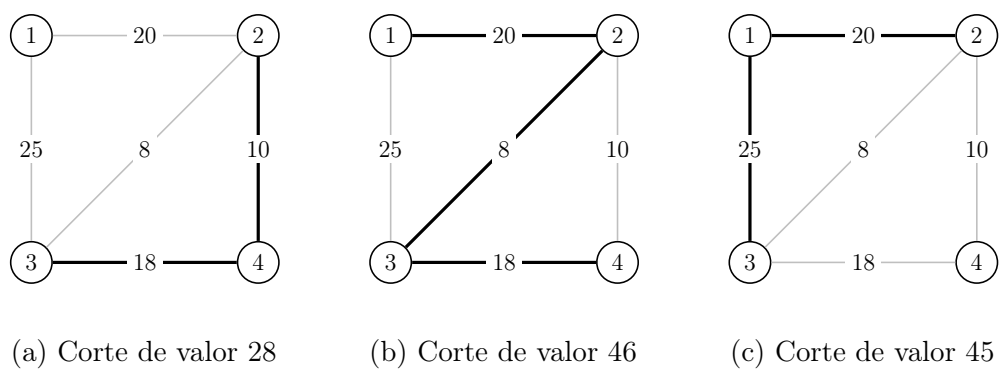


Figura 6.9: Exemplos de corte no TSP

Referências Bibliográficas

- [1] Lagrangean relaxation. <https://github.com/carlosvinicius01/kit-opt/blob/master/Relaxa%C3%A7%C3%A3o-Lagrangeana/LagrangianRelaxation.pdf>.
- [2] Tsp 1 tree. <https://github.com/carlosvinicius01/kit-opt/blob/master/Relaxa%C3%A7%C3%A3o-Lagrangeana/TSP-1-tree.pdf>.
- [3] Tsp 1 tree. <https://github.com/carlosvinicius01/kit-opt/blob/master/Relaxa%C3%A7%C3%A3o-Lagrangeana/relaxlag-2p.pdf>.
- [4] Teobaldo Bulhões. Exemplo de solução para o BPP, 2019. Notas de aula da disciplina Pesquisa Operacional, oferecida pelo Centro de Informática da Universidade Federal da Paraíba.
- [5] Maxence Delorme, Manuel Iori, and Silvano Martello. Bpplib: a library for bin packing and cutting stock problems. *Optimization Letters*, 12:1–16, 03 2018.
- [6] Monique Guignard. Lagrangean relaxation. *Top*, 11(2):151–200, December 2003.
- [7] ILOG CPLEX Optimization Studio (IBM). What are callbacks? <https://www.ibm.com/docs/en/icos/20.1.0?topic=callbacks-what-are>, 2021.
- [8] L. V. Kantorovich. Mathematical methods of organizing and planning production. *Management Science*, 6(4):366–422, 1960.
- [9] Nelson Maculan and Marcia H Costa Fampa. *Otimização Linear*. 2004.
- [10] Denis Naddef. Polyhedral theory and branch- and-cut algorithms for the symmetric tsp. chapter 2, pages 84–85. Laboratoire Informatique et Distribution, Institut National Polytechnique de Grenoble, France.

REFERÊNCIAS BIBLIOGRÁFICAS

- [11] Marcos Melo Silva, Anand Subramanian, Thibaut Vidal, and Luiz Satoru Ochi. A simple and effective metaheuristic for the minimum latency problem. *European Journal of Operational Research*, 221(3):513 – 520, 2012.
- [12] Marccone Jamilson Freitas Souza. Inteligência computacional para otimização. Instituto de Ciências Exatas e Biológicas, Universidade Federal de Ouro Preto, 35400-000 Ouro Preto, MG, jan 2011.
- [13] Mechthild Stoer and Frank Wagner. A simple min-cut algorithm. *J. ACM*, 44(4):585–591, July 1997.